

STAR RX72N - rx_ds18b20 Library
1.0.0

Generated by Doxygen 1.16.1

1 Topic Index	1
1.1 Topics	1
2 Directory Hierarchy	3
2.1 Directories	3
3 File Index	5
3.1 File List	5
4 Topic Documentation	7
4.1 Internal Constants	7
4.1.1 Detailed Description	7
4.1.2 Variable Documentation	7
4.1.2.1 s_ds18b20_conversion_time_invalid_u32	7
4.1.2.2 s_ds18b20_family_code	8
4.1.2.3 s_ds18b20_reserved_byte_value	8
4.1.2.4 s_tag	8
4.1.2.5 s_temp_conversion_divisor	8
5 Directory Documentation	9
5.1 libs/rx_ds18b20/inc Directory Reference	9
5.2 libs Directory Reference	9
5.3 libs/rx_ds18b20 Directory Reference	10
5.4 libs/rx_ds18b20/src Directory Reference	10
6 File Documentation	11
6.1 libs/rx_ds18b20/inc/rx_ds18b20.h File Reference	11
6.1.1 Detailed Description	13
6.1.2 Overview	13
6.1.3 Key Features	13
6.1.3.1 Measurement Capabilities	13
6.1.3.2 Resolution vs. Precision vs. Conversion Time Trade-off	13
6.1.3.3 Advanced Features	14
6.1.4 System Architecture Context	14
6.1.5 Design Rationale	14
6.1.5.1 Dependency Injection Pattern	14
6.1.5.2 Blocking API Design	15
6.1.5.3 Error Handling Strategy	15
6.1.6 Implementation Approach	15
6.1.6.1 Temperature Conversion Process	15
6.1.6.2 Memory Layout and Scratchpad Structure	15
6.1.7 Performance Characteristics	16
6.1.7.1 Timing Analysis	16
6.1.7.2 Memory Usage	16

6.1.8 Hardware Requirements	17
6.1.8.1 Typical Wiring (Single Sensor)	18
6.1.9 Example Usage Patterns	18
6.1.9.1 Example 1: Basic Single-Sensor Configuration	18
6.1.9.2 Example 2: Multi-Sensor Configuration (ROM Matching)	19
6.1.9.3 Example 3: Error Handling and Diagnostics	19
6.1.9.4 Example 4: Runtime Resolution Change	19
6.1.10 Thread Safety	19
6.1.11 Known Limitations	20
6.1.12 References and Standards	20
6.1.13 Data Structure Documentation	22
6.1.13.1 struct rx_ds18b20_config_t	22
6.1.13.2 struct rx_ds18b20_handle_t	22
6.1.14 Enumeration Type Documentation	23
6.1.14.1 ds18b20_command_t	23
6.1.14.2 ds18b20_config_bits_t	24
6.1.14.3 ds18b20_conversion_constants_t	24
6.1.14.4 ds18b20_conversion_time_ms_t	25
6.1.14.5 ds18b20_init_values_t	25
6.1.14.6 ds18b20_power_mode_t	25
6.1.14.7 ds18b20_resolution_t	26
6.1.14.8 ds18b20_scratchpad_index_t	26
6.1.14.9 ds18b20_temp_mask_t	28
6.1.14.10 ds18b20_write_idx_t	29
6.1.15 Function Documentation	29
6.1.15.1 rx_ds18b20_deinit()	29
6.1.15.2 rx_ds18b20_get_conversion_time_ms()	31
6.1.15.3 rx_ds18b20_get_resolution()	32
6.1.15.4 rx_ds18b20_init()	34
6.1.15.5 rx_ds18b20_read_power_mode()	36
6.1.15.6 rx_ds18b20_read_scratchpad()	39
6.1.15.7 rx_ds18b20_read_temperature()	41
6.1.15.8 rx_ds18b20_read_temperature_raw()	44
6.1.15.9 rx_ds18b20_recall_config()	47
6.1.15.10 rx_ds18b20_save_config()	49
6.1.15.11 rx_ds18b20_set_resolution()	52
6.1.15.12 rx_ds18b20_trigger_conversion()	56
6.2 rx_ds18b20.h	59
6.3 libs/rx_ds18b20/src/rx_ds18b20.c File Reference	70
6.3.1 Detailed Description	72
6.3.1.1 Architecture	72
6.3.1.2 NASA Power of 10 Compliance	72

6.3.1.3 SOLID Principles	73
6.3.1.4 Performance	73
6.3.1.5 Memory Layout	74
6.3.1.6 Error Handling	74
6.3.2 Data Structure Documentation	76
6.3.2.1 struct ds18b20_scratchpad_write_t	76
6.3.3 Macro Definition Documentation	77
6.3.3.1 CHECK_DS18B20_HANDLE	77
6.3.4 Function Documentation	78
6.3.4.1 internal_ds18b20_get_temp_mask()	78
6.3.4.2 internal_ds18b20_raw_to_celsius()	79
6.3.4.3 internal_ds18b20_read_scratchpad_raw()	79
6.3.4.4 internal_ds18b20_resolution_to_config()	83
6.3.4.5 internal_ds18b20_select_device()	83
6.3.4.6 internal_ds18b20_validate_config()	86
6.3.4.7 internal_ds18b20_verify_device_presence()	88
6.3.4.8 internal_ds18b20_write_scratchpad()	90
6.3.4.9 rx_ds18b20_deinit()	94
6.3.4.10 rx_ds18b20_get_conversion_time_ms()	95
6.3.4.11 rx_ds18b20_get_resolution()	96
6.3.4.12 rx_ds18b20_init()	98
6.3.4.13 rx_ds18b20_read_power_mode()	100
6.3.4.14 rx_ds18b20_read_scratchpad()	102
6.3.4.15 rx_ds18b20_read_temperature()	104
6.3.4.16 rx_ds18b20_read_temperature_raw()	106
6.3.4.17 rx_ds18b20_recall_config()	109
6.3.4.18 rx_ds18b20_save_config()	111
6.3.4.19 rx_ds18b20_set_resolution()	113
6.3.4.20 rx_ds18b20_trigger_conversion()	116
6.4 rx_ds18b20.c	118

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

Internal Constants	7
--------------------	---

Chapter 2

Directory Hierarchy

2.1 Directories

inc	9
rx_ds18b20.h	11
libs	9
rx_ds18b20	10
inc	9
rx_ds18b20.h	11
src	10
rx_ds18b20.c	70
rx_ds18b20	10
inc	9
rx_ds18b20.h	11
src	10
rx_ds18b20.c	70
src	10
rx_ds18b20.c	70

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

libs/rx_ds18b20/inc/ rx_ds18b20.h	
DS18B20 1-Wire Digital Temperature Sensor Driver API	11
libs/rx_ds18b20/src/ rx_ds18b20.c	
DS18B20 1-Wire Digital Temperature Sensor Driver Implementation	70

Chapter 4

Topic Documentation

4.1 Internal Constants

Variables

- static const char [s_tag](#) [] = "DS18B20"
Logging tag for DS18B20 driver.
- static const uint8_t [s_ds18b20_family_code](#) = 0x28U
DS18B20 family code (byte 0 of ROM).
- static const float [s_temp_conversion_divisor](#) = 16.0f
Temperature conversion divisor (raw to Celsius).
- static const uint8_t [s_ds18b20_reserved_byte_value](#) = 0xFFU
Reserved byte expected value in scratchpad.
- static const uint32_t [s_ds18b20_conversion_time_invalid_u32](#) = 0U
Invalid conversion time sentinel.

4.1.1 Detailed Description

4.1.2 Variable Documentation

4.1.2.1 [s_ds18b20_conversion_time_invalid_u32](#)

```
const uint32_t s_ds18b20_conversion_time_invalid_u32 = 0U [static]
```

Invalid conversion time sentinel.

Returned by [rx_ds18b20_get_conversion_time_ms\(\)](#) on error.

Definition at line 239 of file [rx_ds18b20.c](#).

Referenced by [rx_ds18b20_get_conversion_time_ms\(\)](#).

4.1.2.2 s_ds18b20_family_code

```
const uint8_t s_ds18b20_family_code = 0x28U [static]
```

DS18B20 family code (byte 0 of ROM).

Definition at line 221 of file [rx_ds18b20.c](#).

Referenced by [internal_ds18b20_validate_config\(\)](#).

4.1.2.3 s_ds18b20_reserved_byte_value

```
const uint8_t s_ds18b20_reserved_byte_value = 0xFFU [static]
```

Reserved byte expected value in scratchpad.

Byte 5 of scratchpad should always be 0xFF per datasheet.

Definition at line 233 of file [rx_ds18b20.c](#).

Referenced by [internal_ds18b20_read_scratchpad_raw\(\)](#).

4.1.2.4 s_tag

```
const char s_tag[] = "DS18B20" [static]
```

Logging tag for DS18B20 driver.

Definition at line 218 of file [rx_ds18b20.c](#).

Referenced by [internal_ds18b20_raw_to_celsius\(\)](#), [internal_ds18b20_read_scratchpad_raw\(\)](#), [internal_ds18b20_select_device\(\)](#), [internal_ds18b20_validate_config\(\)](#), [internal_ds18b20_verify_device_presence\(\)](#), [internal_ds18b20_write_scratchpad\(\)](#), [rx_ds18b20_deinit\(\)](#), [rx_ds18b20_get_resolution\(\)](#), [rx_ds18b20_init\(\)](#), [rx_ds18b20_read_power_mode\(\)](#), [rx_ds18b20_read_scratchpad\(\)](#), [rx_ds18b20_read_temperature\(\)](#), [rx_ds18b20_read_temperature_raw\(\)](#), [rx_ds18b20_recall_config\(\)](#), [rx_ds18b20_save_config\(\)](#), [rx_ds18b20_set_resolution\(\)](#), and [rx_ds18b20_trigger_conversion\(\)](#).

4.1.2.5 s_temp_conversion_divisor

```
const float s_temp_conversion_divisor = 16.0f [static]
```

Temperature conversion divisor (raw to Celsius).

DS18B20 stores temperature in 1/16degC units. Divide by 16 to get Celsius.

Definition at line 227 of file [rx_ds18b20.c](#).

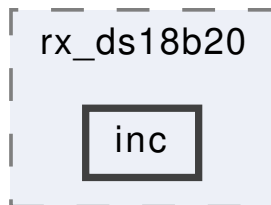
Referenced by [internal_ds18b20_raw_to_celsius\(\)](#).

Chapter 5

Directory Documentation

5.1 libs/rx_ds18b20/inc Directory Reference

Directory dependency graph for inc:

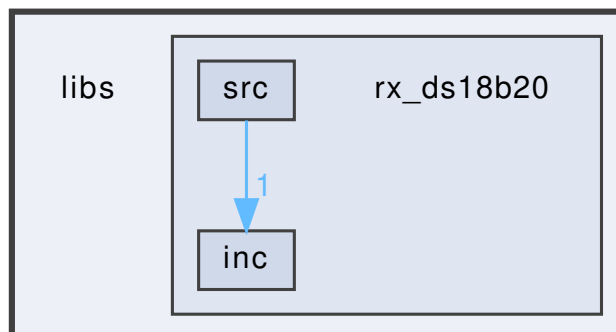


Files

- file [rx_ds18b20.h](#)
DS18B20 1-Wire Digital Temperature Sensor Driver API.

5.2 libs Directory Reference

Directory dependency graph for libs:

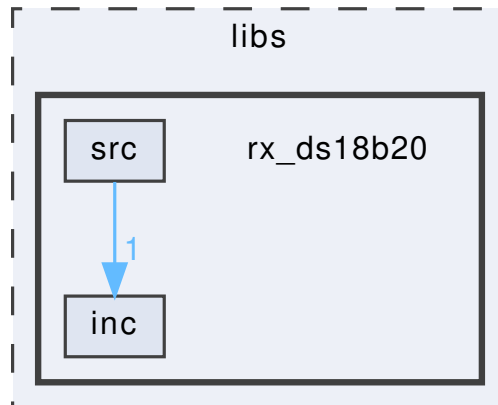


Directories

- directory [rx_ds18b20](#)

5.3 libs/rx_ds18b20 Directory Reference

Directory dependency graph for rx_ds18b20:

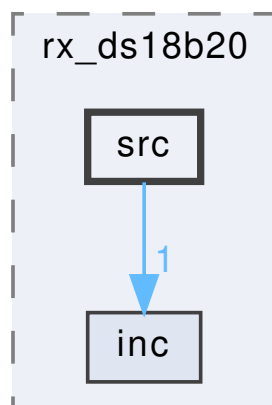


Directories

- directory [inc](#)
- directory [src](#)

5.4 libs/rx_ds18b20/src Directory Reference

Directory dependency graph for src:



Files

- file [rx_ds18b20.c](#)
DS18B20 1-Wire Digital Temperature Sensor Driver Implementation.

Chapter 6

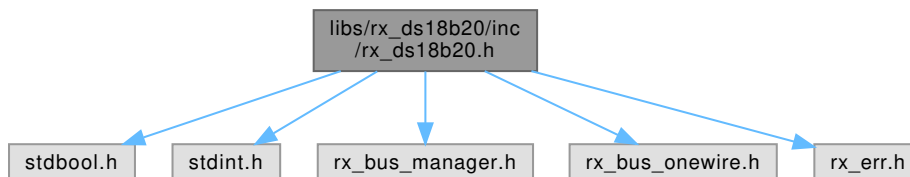
File Documentation

6.1 libs/rx_ds18b20/inc/rx_ds18b20.h File Reference

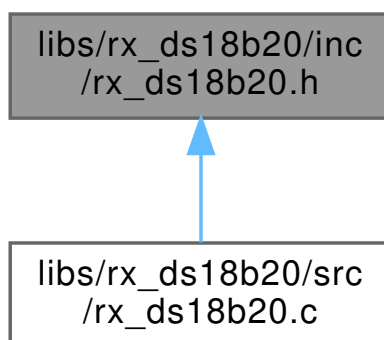
DS18B20 1-Wire Digital Temperature Sensor Driver API.

```
#include <stdbool.h>
#include <stdint.h>
#include "rx_bus_manager.h"
#include "rx_bus_1wire.h"
#include "rx_err.h"
```

Include dependency graph for rx_ds18b20.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [rx_ds18b20_config_t](#)
DS18B20 configuration structure.
- struct [rx_ds18b20_handle_t](#)
DS18B20 driver handle structure.

Enumerations

- enum `ds18b20_command_t`: `uint8_t` {
`k_ds18b20_cmd_convert_t` = 0x44 , `k_ds18b20_cmd_write_scratch` = 0x4E , `k_ds18b20_cmd_read_scratch` = 0xBE , `k_ds18b20_cmd_copy_scratch` = 0x48 ,
`k_ds18b20_cmd_recall_eeprom` = 0xB8 , `k_ds18b20_cmd_read_power` = 0xB4 }
DS18B20 function commands per datasheet specification.
- enum `ds18b20_scratchpad_index_t`: `uint8_t` {
`k_ds18b20_scratch_temp_lsb` = 0 , `k_ds18b20_scratch_temp_msb` = 1 , `k_ds18b20_scratch_th_reg` = 2 ,
`k_ds18b20_scratch_tl_reg` = 3 ,
`k_ds18b20_scratch_config` = 4 , `k_ds18b20_scratch_reserved1` = 5 , `k_ds18b20_scratch_reserved2` = 6 ,
`k_ds18b20_scratch_reserved3` = 7 ,
`k_ds18b20_scratch_crc` = 8 , `k_ds18b20_scratchpad_bytes` = 9 }
DS18B20 scratchpad memory layout indices.
- enum `ds18b20_config_bits_t`: `uint8_t` { `k_ds18b20_config_r0_bit` = 5 , `k_ds18b20_config_r1_bit` = 6 }
DS18B20 configuration register bit positions.
- enum `ds18b20_resolution_t`: `uint8_t` { `k_ds18b20_resolutionobit` = 0 , `k_ds18b20_resolution bit` = 1 ,
`k_ds18b20_resolution bit` = 2 , `k_ds18b20_resolution bit` = 3 }
DS18B20 temperature resolution modes.
- enum `ds18b20_conversion_time_ms_t`: `uint16_t` { `k_ds18b20_conv_timeobit_ms` = 100 , `k_ds18b20_conv_time bit_ms` = 200 , `k_ds18b20_conv_time bit_ms` = 400 , `k_ds18b20_conv_time bit_ms` = 800 }
DS18B20 conversion times (milliseconds).
- enum `ds18b20_temp_mask_t`: `uint16_t` { `k_ds18b20_temp_maskobit` = 0xFFFF8 , `k_ds18b20_temp_mask bit` = 0xFFFC , `k_ds18b20_temp_mask bit` = 0xFFFE , `k_ds18b20_temp_mask bit` = 0xFFFF }
DS18B20 temperature masks for resolution.
- enum `ds18b20_conversion_constants_t`: `int32_t` {
`k_ds18b20_temp_shift` = 4 , `k_ds18b20_sign_bit` = 0x8000 , `k_ds18b20_crc_bytes` = 8 , `k_ds18b20_shift_byte` = 8 ,
`k_ds18b20_scratchpad_write_bytes` = 3 , `k_ds18b20_temp_min_raw` = -880 , `k_ds18b20_temp_max_raw` = 2000 }
DS18B20 temperature conversion constants.
- enum `ds18b20_write_idx_t`: `uint8_t` { `k_ds18b20_write_idx_th` = 0 , `k_ds18b20_write_idx_tl` = 1 ,
`k_ds18b20_write_idx_config` = 2 }
DS18B20 scratchpad write buffer indices.
- enum `ds18b20_init_values_t`: `uint8_t` { `k_ds18b20_config_register_cleared` = 0 }
DS18B20 initialization values.
- enum `ds18b20_power_mode_t`: `uint8_t` { `k_ds18b20_power_parasitic` = 0 , `k_ds18b20_power_external` = 1 }
DS18B20 power supply modes.

Functions

- `rx_err_t rx_ds18b20_init` (`rx_ds18b20_handle_t` *handle, const `rx_ds18b20_config_t` *config)
Initialize DS18B20 sensor.
- `rx_err_t rx_ds18b20_deinit` (`rx_ds18b20_handle_t` *handle)
Deinitialize DS18B20 sensor.
- `rx_err_t rx_ds18b20_trigger_conversion` (const `rx_ds18b20_handle_t` *handle)
Trigger temperature conversion.
- `rx_err_t rx_ds18b20_read_temperature` (`rx_ds18b20_handle_t` *handle, float *temperature_celsius)
Read temperature in Celsius.
- `rx_err_t rx_ds18b20_read_temperature_raw` (`rx_ds18b20_handle_t` *handle, `int16_t` *raw_temp)
Read raw temperature value.
- `rx_err_t rx_ds18b20_set_resolution` (`rx_ds18b20_handle_t` *handle, `ds18b20_resolution_t` resolution)

- Set temperature resolution.*
- rx_err_t [rx_ds18b20_get_resolution](#) (const rx_ds18b20_handle_t *handle, ds18b20_resolution_t *resolution)
- Get current temperature resolution.*
- rx_err_t [rx_ds18b20_save_config](#) (const rx_ds18b20_handle_t *handle)
- Save configuration to EEPROM.*
- rx_err_t [rx_ds18b20_recall_config](#) (const rx_ds18b20_handle_t *handle)
- Recall configuration from EEPROM.*
- rx_err_t [rx_ds18b20_read_power_mode](#) (const rx_ds18b20_handle_t *handle, bool *external_power)
- Read power supply mode.*
- rx_err_t [rx_ds18b20_read_scratchpad](#) (rx_ds18b20_handle_t *handle, uint8_t scratchpad[k_ds18b20_scratchpad_bytes])
- Read scratchpad memory.*
- uint32_t [rx_ds18b20_get_conversion_time_ms](#) (const rx_ds18b20_handle_t *handle)
- Get conversion time for current resolution.*

6.1.1 Detailed Description

DS18B20 1-Wire Digital Temperature Sensor Driver API.

6.1.2 Overview

Platform-independent driver for Maxim DS18B20 programmable resolution 1-Wire digital thermometer with CRC-8 data integrity checking. Provides high-level API for temperature measurement, resolution configuration, and EEPROM persistence with automatic error handling and validation.

This driver abstracts the DS18B20's complex 1-Wire protocol into simple init, trigger, and read operations while maintaining full access to advanced features like ROM matching (multi-drop buses) and alarm thresholds.

6.1.3 Key Features

6.1.3.1 Measurement Capabilities

- **Temperature range:** -55degC to +125degC with +/-0.5degC accuracy (-10degC to +85degC)
- **Programmable resolution:** 9, 10, 11, or 12 bits
- **Conversion times:** 94ms (9-bit) to 750ms (12-bit)
- **CRC-8 validation:** Detects transmission errors (polynomial $x^8 + x^5 + x^4 + 1$)

6.1.3.2 Resolution vs. Precision vs. Conversion Time Trade-off

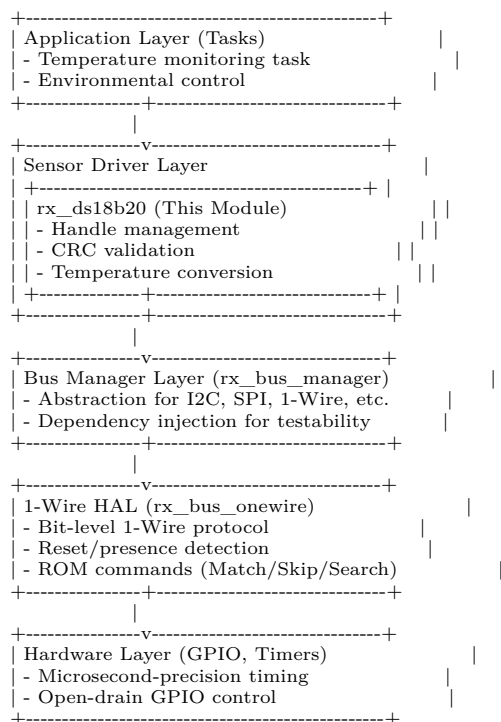
Resolution	Precision	Conversion Time	Use Case
9-bit	0.5degC	~94ms	Fast updates, low precision needed
10-bit	0.25degC	~188ms	Balanced for most applications
11-bit	0.125degC	~375ms	High precision, moderate speed
12-bit	0.0625degC	~750ms	Maximum precision, slow updates

6.1.3.3 Advanced Features

- **ROM matching:** Supports multi-device 1-Wire buses (up to 255 sensors)
- **Skip ROM mode:** Optimized for single-sensor configurations
- **EEPROM storage:** Persist resolution/alarm settings across power cycles
- **Power mode detection:** Identify parasitic vs. external power operation
- **Alarm thresholds:** Programmable TH/TL registers (not exposed in this API)

6.1.4 System Architecture Context

The DS18B20 driver sits at the sensor layer of the STAR firmware stack:



6.1.5 Design Rationale

6.1.5.1 Dependency Injection Pattern

The driver accepts a `rx_bus_manager_t*` instead of directly accessing GPIO, enabling:

- **Unit testing** with mock bus implementations (no hardware required)
- **Hardware independence** - works on any 1-Wire HAL implementation
- **Multi-bus support** - single driver code for multiple 1-Wire buses

This follows the **Dependency Inversion Principle** (high-level modules depend on abstractions, not concrete implementations).

6.1.5.2 Blocking API Design

All operations are blocking (caller must wait for conversion). Rationale:

- DS18B20 conversions are inherently slow (up to 750ms)
- Non-blocking would require state machines and polling infrastructure
- Simplicity favored: caller can use ThreadX semaphores if async needed
- Typical usage: Dedicated temperature monitoring task with periodic reads

6.1.5.3 Error Handling Strategy

- **All functions return `rx_err_t`** for consistent error propagation
- **CRC validation on every read** - detect bus noise/corruption
- **Range checking** - reject impossible temperatures (-880 to +2000 raw)
- **nullptr checks** - fail fast on API misuse
- **State validation** - ensure init before use, prevent double-init

6.1.6 Implementation Approach

6.1.6.1 Temperature Conversion Process

The DS18B20 uses a two-phase measurement cycle:

1. **Trigger Phase** (`rx_ds18b20_trigger_conversion()`):
 - Send Convert T command (0x44)
 - DS18B20 begins analog-to-digital conversion
 - Returns immediately (non-blocking)
2. **Wait Phase** (Caller's responsibility):
 - Wait for conversion time based on resolution
 - Use `rx_ds18b20_get_conversion_time_ms()` to get exact time
 - ThreadX: `tx_thread_sleep(time_ms * TX_TIMER_TICKS_PER_SECOND / 1000)`
3. **Read Phase** (`rx_ds18b20_read_temperature()`):
 - Read 9-byte scratchpad (includes CRC)
 - Validate CRC-8 checksum
 - Extract 16-bit raw temperature
 - Apply resolution mask (clear undefined bits)
 - Convert to Celsius (raw / 16.0)

6.1.6.2 Memory Layout and Scratchpad Structure

The DS18B20's scratchpad RAM is 9 bytes:

Byte	Name	Description
0	Temp LSB	Temperature low byte

Byte	Name	Description
1	Temp MSB	Temperature high byte (+ sign)
2	TH Register	High alarm threshold
3	TL Register	Low alarm threshold
4	Config	Resolution bits [6:5]
5	Reserved	0xFF
6	Reserved	Factory value
7	Reserved	Factory value
8	CRC	CRC-8 of bytes 0-7

Configuration register format (byte 4):

```

Bit:  7  6  5  4  3  2  1  0
      0  R1 R0  1  1  1  1  1
      +---+ Resolution: 00=9b, 01=10b, 10=11b, 11=12b

```

6.1.7 Performance Characteristics

6.1.7.1 Timing Analysis

Operation	Duration	Notes
Init	~10ms	Presence detect + read
Trigger conversion	~1ms	Command transmission
Conversion (9-bit)	93.75ms (spec)	100ms recommended
Conversion (10-bit)	187.5ms (spec)	200ms recommended
Conversion (11-bit)	375ms (spec)	400ms recommended
Conversion (12-bit)	750ms (spec)	800ms recommended
Read scratchpad	~2ms	9 bytes + CRC validation
Set resolution	~5ms	Read + modify + write

Note: Recommended times include 10% safety margin above datasheet specs.

6.1.7.2 Memory Usage

Component	Size (bytes)	Location
rx_ds18b20_handle_t	28	Caller stack
rx_ds18b20_config_t	24	Caller stack
Scratchpad buffer	9	Stack (temp)
Driver code (.text)	~2KB	Flash
Constant data (.rodata)	~100 bytes	Flash

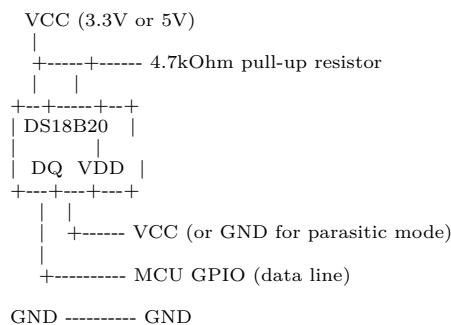
Total RAM per sensor: 28 bytes (handle only, no dynamic allocation)

6.1.8 Hardware Requirements

Resource	Requirement
CPU	Any (architecture-independent)

Resource	Requirement
RAM	28 bytes per sensor + 9-byte temp buffer
Flash	~2KB code + ~100 bytes constants
GPIO	1 pin (open-drain or push-pull with resistor)
Timing	Microsecond-precision delays (1-Wire protocol)
Pull-up	4.7kOhm resistor to VCC (external)
Power	3.0V-5.5V (parasitic mode: data pin can power)

6.1.8.1 Typical Wiring (Single Sensor)



6.1.9 Example Usage Patterns

6.1.9.1 Example 1: Basic Single-Sensor Configuration

```

#include "rx_ds18b20.h"
#include "rx_bus_manager.h"
#include "tx_api.h"

void temperature_task_entry(ULONG input) {
    rx_ds18b20_handle_t sensor;
    rx_ds18b20_config_t config = {
        .bus_manager = &g_bus_manager,
        .bus_name = "onewire0",
        .resolution = k_ds18b20_resolution_bit, // 0.0625degC precision
        .use_rom_matching = false,              // Skip ROM (single sensor)
    };

    rx_err_t err = rx_ds18b20_init(&sensor, &config);
    if (err != k_rx_ok) {
        // Handle error: check bus wiring, power, pull-up resistor
        return;
    }

    while (1) {
        // Trigger conversion
        err = rx_ds18b20_trigger_conversion(&sensor);
        if (err != k_rx_ok) continue;

        // Wait for conversion (800ms for 12-bit)
        tx_thread_sleep(80); // 800ms at 100 Hz tick rate

        // Read temperature
        float temp_celsius;
        err = rx_ds18b20_read_temperature(&sensor, &temp_celsius);
        if (err == k_rx_ok) {
            // Process temperature (e.g., log, control, transmit)
            printf("Temperature: %.2fdegC\n", temp_celsius);
        } else if (err == k_rx_err_crc_mismatch) {
            // CRC error: retry or log fault
        }

        // Repeat every 5 seconds
        tx_thread_sleep(500);
    }
}

```


6.1.9.2 Example 2: Multi-Sensor Configuration (ROM Matching)

```
// Pre-discovered ROM codes (use ROM search to find these)
uint8_t rom_sensor1[8] = {0x28, 0xFF, 0x12, 0x34, 0x56, 0x78, 0x90, 0xAB};
uint8_t rom_sensor2[8] = {0x28, 0xFF, 0xAB, 0xCD, 0xEF, 0x01, 0x23, 0x45};

rx_ds18b20_handle_t sensors[2];
rx_ds18b20_config_t config_template = {
    .bus_manager = &g_bus_manager,
    .bus_name = "onewire0",
    .resolution = k_ds18b20_resolution_bit,
    .use_rom_matching = true, // Enable ROM matching for multi-drop
};

// Init sensor 1
memcpy(config_template.rom, rom_sensor1, 8);
rx_ds18b20_init(&sensors[0], &config_template);

// Init sensor 2
memcpy(config_template.rom, rom_sensor2, 8);
rx_ds18b20_init(&sensors[1], &config_template);

// Trigger both simultaneously (broadcast not shown, or trigger individually)
rx_ds18b20_trigger_conversion(&sensors[0]);
rx_ds18b20_trigger_conversion(&sensors[1]);

tx_thread_sleep(40); // Wait 400ms for 11-bit

// Read both
float temps[2];
rx_ds18b20_read_temperature(&sensors[0], &temps[0]);
rx_ds18b20_read_temperature(&sensors[1], &temps[1]);
```

6.1.9.3 Example 3: Error Handling and Diagnostics

```
rx_err_t err = rx_ds18b20_read_temperature(&sensor, &temp_c);

switch (err) {
    case k_rx_ok:
        // Success: use temp_c
        break;

    case k_rx_err_crc_mismatch:
        // CRC error: possible bus noise, retry
        retry_count++;
        break;

    case k_rx_err_invalid_state:
        // Sensor disconnected or not responding
        // Check: wiring, power, pull-up resistor
        break;

    case k_rx_err_out_of_range:
        // Temperature outside -55degC to +125degC
        // Sensor fault or extreme environment
        break;

    default:
        // Unexpected error
        break;
}
```

6.1.9.4 Example 4: Runtime Resolution Change

```
// Switch to 9-bit for faster updates (0.5degC precision)
rx_ds18b20_set_resolution(&sensor, k_ds18b20_resolution_bit);

// Persist to EEPROM (survives power cycle)
rx_ds18b20_save_config(&sensor);

// Now trigger with shorter wait time
rx_ds18b20_trigger_conversion(&sensor);
tx_thread_sleep(10); // 100ms for 9-bit
rx_ds18b20_read_temperature(&sensor, &temp_c);
```

6.1.10 Thread Safety

Not thread-safe. Caller must serialize access if multiple threads use the same sensor handle. Use ThreadX mutexes:

```
TX_MUTEX sensor_mutex;
tx_mutex_create(&sensor_mutex, "DS18B20", TX_NO_INHERIT);
```

```
// Thread 1:
tx_mutex_get(&sensor_mutex, TX_WAIT_FOREVER);
rx_ds18b20_trigger_conversion(&sensor);
tx_mutex_put(&sensor_mutex);
```

```
// Thread 2:
tx_mutex_get(&sensor_mutex, TX_WAIT_FOREVER);
rx_ds18b20_read_temperature(&sensor, &temp_c);
tx_mutex_put(&sensor_mutex);
```

Note: Different sensor handles (different physical sensors) CAN be accessed concurrently without synchronization, even on the same 1-Wire bus (ROM matching addresses each sensor independently).

6.1.11 Known Limitations

1. **No alarm threshold API:** TH/TL registers accessible via scratchpad but not exposed. Users can read/write scratchpad directly if needed.
2. **No broadcast/simultaneous conversion:** Must trigger each sensor individually when using ROM matching. Skip ROM mode allows broadcast but only for single-sensor.
3. **No ROM search:** Must provide ROM codes manually for multi-sensor. Consider adding `rx_ds18b20_discover()` helper in future.
4. **Blocking API only:** No asynchronous/callback interface. Caller must use RTOS primitives (tasks, timers) for non-blocking behavior.

6.1.12 References and Standards

- **DS18B20 Datasheet:** Maxim Integrated (now Analog Devices) <https://www.analog.com/media/en/technical-documentation/data-sheets/DS18B20.pdf>
- **1-Wire Protocol:** Book of iButton Standards, Maxim Integrated
- **CRC-8 Polynomial:** $x^8 + x^5 + x^4 + 1$ (standard for 1-Wire devices)
- **ROM Structure:** 8-byte format (1-byte family + 6-byte serial + 1-byte CRC)
- **IEEE 802.3 Ethernet CRC:** Different from DS18B20's CRC-8, not applicable here

See also

```
rx_bus_manager.h - Bus manager abstraction layer
rx_bus_onewire.h - 1-Wire protocol implementation
rx_crc.h - CRC calculation utilities (CRC-8 for DS18B20)
docs/sections/03_hardware_pinout.tex - GPIO pin assignments
```

NASA Power of 10 Compliance:

- **Rule 1** (No goto): [OK] All control flow uses structured statements
- **Rule 2** (Bounded loops): [OK] All loops have compile-time or runtime bounds
- **Rule 3** (No heap): [OK] Zero dynamic allocation (static handles only)
- **Rule 4** (Functions ≤ 60 lines): [OK] All public functions < 60 lines
- **Rule 5** (Assertions): [OK] 2+ preconditions per function (NULL checks, state)
- **Rule 6** (Data scope): [OK] Variables declared at smallest scope
- **Rule 7** (Return checking): [OK] All HAL returns validated
- **Rule 8** (Preprocessor limits): [OK] C23 typed enums only, minimal macros
- **Rule 9** (Pointer restrictions): [WARN] Intentional deviation for DIP (bus_manager)
- **Rule 10** (Compiler warnings): [OK] -Wall -Wextra -Werror, zero warnings

SOLID Principles:

- **S (Single Responsibility)**: Module handles ONLY DS18B20 sensor operations
- **O (Open/Closed)**: Extensible via bus_manager (add new buses w/o changes)
- **L (Liskov Substitution)**: Any rx_bus_manager_t* is interchangeable
- **I (Interface Segregation)**: Focused API (temp read, no unrelated operations)
- **D (Dependency Inversion)**: Depends on rx_bus_manager_t abstraction, not GPIO

Module Dependencies:

```
rx_ds18b20
+-- rx_bus_manager (bus abstraction, injected)
+-- rx_bus_onewire (1-Wire protocol layer)
+-- rx_crc (CRC-8 validation)
+-- rx_err (error codes)
+-- rx_log (diagnostic logging)
+-- rx_check (nullptr validation macros)
```

Author

STAR Team

Date

2026-01-30

Version

1.0.0

Copyright

MIT License

Definition in file [rx_ds18b20.h](#).

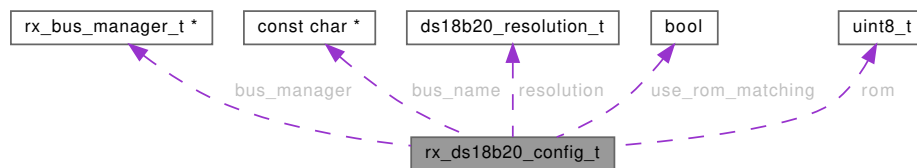
6.1.13 Data Structure Documentation

6.1.13.1 struct rx_ds18b20_config_t

DS18B20 configuration structure.

Definition at line 690 of file [rx_ds18b20.h](#).

Collaboration diagram for rx_ds18b20_config_t:



Data Fields

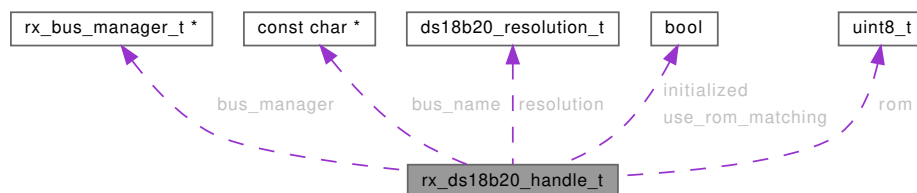
rx_bus_manager_t *	bus_manager	Bus manager instance (required)
const char *	bus_name	OneWire bus name (required)
ds18b20_resolution_t	resolution	Temperature resolution
uint8_t	rom[k_owewire_rom_bytes]	Device ROM (if use_rom_matching=true)
bool	use_rom_matching	Use ROM matching (true) or skip ROM (false)

6.1.13.2 struct rx_ds18b20_handle_t

DS18B20 driver handle structure.

Definition at line 701 of file [rx_ds18b20.h](#).

Collaboration diagram for rx_ds18b20_handle_t:



Data Fields

rx_bus_manager_t *	bus_manager	Bus manager reference (not owned)
const char *	bus_name	OneWire bus name (not owned)
bool	initialized	True after successful init
ds18b20_resolution_t	resolution	Current temperature resolution

uint8_t	rom[k_owewire_rom_bytes]	Device ROM code
bool	use_rom_matching	ROM matching enabled

6.1.14 Enumeration Type Documentation

6.1.14.1 ds18b20_command_t

enum `ds18b20_command_t` : `uint8_t`

DS18B20 function commands per datasheet specification.

Command codes sent after ROM selection (Match ROM or Skip ROM) to initiate DS18B20 operations. Each command triggers a specific sensor function.

Protocol Sequence:

1. Reset pulse + presence detection
2. ROM command (Match ROM 0x55 or Skip ROM 0xCC)
3. Function command (one of these values)
4. Additional data bytes (if required by command)

Command Details:

Command	Code	Data Bytes	Duration	Description
Convert T	0x44	0	94-750ms	Start temperature conversion
Write Scratchpad	0x4E	3	~2ms	Write TH, TL, Config registers
Read Scratchpad	0xBE	0	~2ms	Read 9-byte scratchpad + CRC
Copy Scratchpad	0x48	0	~10ms	Save scratchpad to EEPROM
Recall E ²	0xB8	0	~1ms	Restore EEPROM to scratchpad
Read Power Supply	0xB4	0	<1ms	Check parasitic vs. external

Usage Example:

```
// Trigger conversion
rx_bus_owewire_reset(bus_mgr, "owewire0", &presence);
rx_bus_owewire_skip_rom(bus_mgr, "owewire0");
rx_bus_owewire_write_byte(bus_mgr, "owewire0", k_ds18b20_cmd_convert_t);
// Wait conversion time...
```

See also

DS18B20 datasheet section "Function Commands" for timing specifications

rx_bus_owewire.h for ROM command functions

Since

Version 1.0.0

Enumerator

k_ds18b20_cmd_convert_t	Trigger temperature conversion (ADC operation)
-------------------------	--

k_ds18b20_cmd_write_scratch	Write to scratchpad memory (TH, TL, Config bytes)
k_ds18b20_cmd_read_scratch	Read from scratchpad memory (9 bytes: data + CRC)
k_ds18b20_cmd_copy_scratch	Copy scratchpad to EEPROM (non-volatile storage)
k_ds18b20_cmd_recall_eeprom	Recall EEPROM to scratchpad (restore saved values)
k_ds18b20_cmd_read_power	Read power supply mode (0=parasitic, 1=external)

Definition at line 494 of file [rx_ds18b20.h](#).

```

00494     : uint8_t {
00495     k_ds18b20_cmd_convert_t    = 0x44, /**< Trigger temperature conversion (ADC operation) */
00496     k_ds18b20_cmd_write_scratch = 0x4E, /**< Write to scratchpad memory (TH, TL, Config bytes) */
00497     k_ds18b20_cmd_read_scratch  = 0xBE, /**< Read from scratchpad memory (9 bytes: data + CRC) */
00498     k_ds18b20_cmd_copy_scratch  = 0x48, /**< Copy scratchpad to EEPROM (non-volatile storage) */
00499     k_ds18b20_cmd_recall_eeprom = 0xB8, /**< Recall EEPROM to scratchpad (restore saved values) */
00500     k_ds18b20_cmd_read_power    = 0xB4, /**< Read power supply mode (0=parasitic, 1=external) */
00501 } ds18b20_command_t;

```

6.1.14.2 ds18b20_config_bits_t

enum [ds18b20_config_bits_t](#) : uint8_t

DS18B20 configuration register bit positions.

Enumerator

k_ds18b20_config_r0_bit	Resolution bit 0
k_ds18b20_config_r1_bit	Resolution bit 1

Definition at line 604 of file [rx_ds18b20.h](#).

```

00604     : uint8_t {
00605     k_ds18b20_config_r0_bit = 5, /**< Resolution bit 0 */
00606     k_ds18b20_config_r1_bit = 6, /**< Resolution bit 1 */
00607 } ds18b20_config_bits_t;

```

6.1.14.3 ds18b20_conversion_constants_t

enum [ds18b20_conversion_constants_t](#) : int32_t

DS18B20 temperature conversion constants.

Enumerator

k_ds18b20_temp_shift	Shift to get integer temperature
k_ds18b20_sign_bit	Sign bit in 16-bit temperature
k_ds18b20_crc_bytes	Number of bytes for CRC calculation
k_ds18b20_shift_byte	Shift for byte positioning
k_ds18b20_scratchpad_write_bytes	Scratchpad write size (TH, TL, Config)
k_ds18b20_temp_min_raw	Minimum temperature (-55degC raw value)
k_ds18b20_temp_max_raw	Maximum temperature (+125degC raw value)

Definition at line 648 of file [rx_ds18b20.h](#).

```

00648         : int32_t {
00649     k_ds18b20_temp_shift      = 4,    /**< Shift to get integer temperature */
00650     k_ds18b20_sign_bit       = 0x8000, /**< Sign bit in 16-bit temperature */
00651     k_ds18b20_crc_bytes      = 8,    /**< Number of bytes for CRC calculation */
00652     k_ds18b20_shift_byte     = 8,    /**< Shift for byte positioning */
00653     k_ds18b20_scratchpad_write_bytes = 3, /**< Scratchpad write size (TH, TL, Config) */
00654     k_ds18b20_temp_min_raw   = -880, /**< Minimum temperature (-55degC raw value) */
00655     k_ds18b20_temp_max_raw   = 2000, /**< Maximum temperature (+125degC raw value) */
00656 } ds18b20_conversion_constants_t;

```

6.1.14.4 ds18b20_conversion_time_ms_t

```
enum ds18b20_conversion_time_ms_t : uint16_t
```

DS18B20 conversion times (milliseconds).

Maximum conversion time for each resolution. Add margin for safety (spec values + 10%).

Enumerator

k_ds18b20_conv_timeobit_ms	9-bit conversion: 100ms
k_ds18b20_conv_time bit_ms	10-bit conversion: 200ms
k_ds18b20_conv_time bit_ms	11-bit conversion: 400ms
k_ds18b20_conv_time bit_ms	12-bit conversion: 800ms

Definition at line 625 of file [rx_ds18b20.h](#).

```

00625         : uint16_t {
00626     k_ds18b20_conv_time bit_ms = 100, /**< 9-bit conversion: 100ms */
00627     k_ds18b20_conv_time bit_ms = 200, /**< 10-bit conversion: 200ms */
00628     k_ds18b20_conv_time bit_ms = 400, /**< 11-bit conversion: 400ms */
00629     k_ds18b20_conv_time bit_ms = 800, /**< 12-bit conversion: 800ms */
00630 } ds18b20_conversion_time_ms_t;

```

6.1.14.5 ds18b20_init_values_t

```
enum ds18b20_init_values_t : uint8_t
```

DS18B20 initialization values.

Enumerator

k_ds18b20_config_register_cleared	Cleared config register value
-----------------------------------	-------------------------------

Definition at line 670 of file [rx_ds18b20.h](#).

```

00670         : uint8_t {
00671     k_ds18b20_config_register_cleared = 0, /**< Cleared config register value */
00672 } ds18b20_init_values_t;

```

6.1.14.6 ds18b20_power_mode_t

```
enum ds18b20_power_mode_t : uint8_t
```

DS18B20 power supply modes.

Enumerator

k_ds18b20_power_parasitic	Parasitic power mode
---------------------------	----------------------

k_ds18b20_power_external	External power mode
--------------------------	---------------------

Definition at line 677 of file [rx_ds18b20.h](#).

```
00677 : uint8_t {
00678 k_ds18b20_power_parasitic = 0, /**< Parasitic power mode */
00679 k_ds18b20_power_external = 1, /**< External power mode */
00680 } ds18b20_power_mode_t;
```

6.1.14.7 ds18b20_resolution_t

enum [ds18b20_resolution_t](#) : uint8_t

DS18B20 temperature resolution modes.

Enumerator

k_ds18b20_resolutionnobit	9-bit: 0.5degC, 93.75ms
k_ds18b20_resolution bit	10-bit: 0.25degC, 187.5ms
k_ds18b20_resolution bit	11-bit: 0.125degC, 375ms
k_ds18b20_resolution bit	12-bit: 0.0625degC, 750ms

Definition at line 612 of file [rx_ds18b20.h](#).

```
00612 : uint8_t {
00613 k_ds18b20_resolution bit = 0, /**< 9-bit: 0.5degC, 93.75ms */
00614 k_ds18b20_resolution bit = 1, /**< 10-bit: 0.25degC, 187.5ms */
00615 k_ds18b20_resolution bit = 2, /**< 11-bit: 0.125degC, 375ms */
00616 k_ds18b20_resolution bit = 3, /**< 12-bit: 0.0625degC, 750ms */
00617 } ds18b20_resolution_t;
```

6.1.14.8 ds18b20_scratchpad_index_t

enum [ds18b20_scratchpad_index_t](#) : uint8_t

DS18B20 scratchpad memory layout indices.

The DS18B20's 9-byte scratchpad RAM contains temperature data, alarm thresholds, configuration, and a CRC-8 checksum. Use these indices to access specific bytes when reading via Read Scratchpad command (0xBE).

Memory Layout:

Index	Byte	Name	Description	Access
0	0	Temp LSB	Temperature low byte (bits 0-7)	Read-only
1	1	Temp MSB	Temperature high byte (sign + bits 8-11)	Read-only
2	2	TH Register	High alarm threshold (-55 to +125degC)	R/W
3	3	TL Register	Low alarm threshold (-55 to +125degC)	R/W
4	4	Config	Resolution bits [6:5]: 00=9b, 01=10b, 10=11b, 11=12b	R/W
5	5	Reserved	Always 0xFF (ignore)	Read-only
6	6	Reserved	Factory-programmed (ignore)	Read-only

Index	Byte	Name	Description	Access
7	7	Reserved	Factory-programmed (ignore)	Read-only
8	8	CRC	CRC-8 checksum of bytes 0-7	Read-only

Temperature Format (Bytes 0-1):

16-bit signed integer in 1/16degC units (two's complement):

MSB (byte 1): S S S S S T10 T9 T8

LSB (byte 0): T7 T6 T5 T4 T3 T2 T1 T0

+-----+ Temperature bits (12-bit for 12-bit resolution)

- **S:** Sign bits (all 0 for positive, all 1 for negative)
- **T10-T0:** Temperature bits (11 bits used, T0-T2 undefined for lower resolutions)
- **Value range:** -880 to +2000 (represents -55.0degC to +125.0degC)
- **Conversion:** $\text{temp_celsius} = \text{raw_value} / 16.0$

Configuration Register (Byte 4):

Format: 0 R1 R0 1 1 1 1 1 where R1:R0 = resolution bits

R1	R0	Resolution	Precision	Conversion Time
0	0	9-bit	0.5degC	93.75ms
0	1	10-bit	0.25degC	187.5ms
1	0	11-bit	0.125degC	375ms
1	1	12-bit	0.0625degC	750ms

CRC-8 Calculation (Byte 8):

- **Polynomial:** $x^8 + x^5 + x^4 + 1$ (0x8C)
- **Initial value:** 0x00
- **Data:** Bytes 0-7 (inclusive)
- **Purpose:** Detect transmission errors on 1-Wire bus

Usage Example:

```
uint8_t scratchpad[k_ds18b20_scratchpad_bytes];
rx_ds18b20_read_scratchpad(&sensor, scratchpad);

// Extract temperature
uint16_t temp_raw = scratchpad[k_ds18b20_scratch_temp_lsb] |
    (scratchpad[k_ds18b20_scratch_temp_msb] « 8);

// Validate CRC
uint8_t crc_calc = rx_crc8_maxim(scratchpad, k_ds18b20_crc_bytes);
if (crc_calc != scratchpad[k_ds18b20_scratch_crc]) {
    // CRC mismatch: retry read
}

// Read alarm thresholds
int8_t th = (int8_t)scratchpad[k_ds18b20_scratch_th_reg];
int8_t tl = (int8_t)scratchpad[k_ds18b20_scratch_tl_reg];
```

Invariant

Byte 5 (reserved1) is always 0xFF

CRC (byte 8) is valid for bytes 0-7

Temperature bytes (0-1) represent value in range [-880, +2000]

See also

[rx_ds18b20_read_scratchpad\(\)](#) Read entire scratchpad with CRC validation

[rx_crc8_maxim\(\)](#) CRC-8 calculation function

DS18B20 datasheet section "Memory" for detailed scratchpad specification

Since

Version 1.0.0

Enumerator

k_ds18b20_scratch_temp_lsb	Temperature LSB (bits 0-7 of 16-bit value)
k_ds18b20_scratch_temp_msb	Temperature MSB (sign bits + bits 8-11)
k_ds18b20_scratch_th_reg	TH (high alarm) register (signed byte, -55 to +125)
k_ds18b20_scratch_tl_reg	TL (low alarm) register (signed byte, -55 to +125)
k_ds18b20_scratch_config	Configuration register (resolution bits at [6:5])
k_ds18b20_scratch_reserved1	Reserved byte (always 0xFF, do not write)
k_ds18b20_scratch_reserved2	Reserved byte (factory-programmed, do not write)
k_ds18b20_scratch_reserved3	Reserved byte (factory-programmed, do not write)
k_ds18b20_scratch_crc	CRC-8 of bytes 0-7 (polynomial 0x8C)
k_ds18b20_scratchpad_bytes	Total scratchpad size in bytes

Definition at line 588 of file [rx_ds18b20.h](#).

```

00588      : uint8_t {
00589  k_ds18b20_scratch_temp_lsb = 0, /**< Temperature LSB (bits 0-7 of 16-bit value) */
00590  k_ds18b20_scratch_temp_msb = 1, /**< Temperature MSB (sign bits + bits 8-11) */
00591  k_ds18b20_scratch_th_reg   = 2, /**< TH (high alarm) register (signed byte, -55 to +125) */
00592  k_ds18b20_scratch_tl_reg   = 3, /**< TL (low alarm) register (signed byte, -55 to +125) */
00593  k_ds18b20_scratch_config    = 4, /**< Configuration register (resolution bits at [6:5]) */
00594  k_ds18b20_scratch_reserved1 = 5, /**< Reserved byte (always 0xFF, do not write) */
00595  k_ds18b20_scratch_reserved2 = 6, /**< Reserved byte (factory-programmed, do not write) */
00596  k_ds18b20_scratch_reserved3 = 7, /**< Reserved byte (factory-programmed, do not write) */
00597  k_ds18b20_scratch_crc       = 8, /**< CRC-8 of bytes 0-7 (polynomial 0x8C) */
00598  k_ds18b20_scratchpad_bytes  = 9, /**< Total scratchpad size in bytes */
00599 } ds18b20_scratchpad_index_t;

```

6.1.14.9 ds18b20_temp_mask_t

enum [ds18b20_temp_mask_t](#) : uint16_t

DS18B20 temperature masks for resolution.

Lower bits are undefined for resolutions < 12-bit and must be masked. This prevents garbage bits from corrupting the temperature reading.

Enumerator

k_ds18b20_temp_maskobit	Mask for 9-bit (discard bits 0-2)
-------------------------	-----------------------------------

k_ds18b20_temp_mask bit	Mask for 10-bit (discard bits 0-1)
k_ds18b20_temp_mask bit	Mask for 11-bit (discard bit 0)
k_ds18b20_temp_mask bit	Mask for 12-bit (all bits valid)

Definition at line 638 of file [rx_ds18b20.h](#).

```

00638      : uint16_t {
00639  k_ds18b20_temp_mask bit = 0xFFFF8, /**< Mask for 9-bit (discard bits 0-2) */
00640  k_ds18b20_temp_mask bit = 0xFFFFC, /**< Mask for 10-bit (discard bits 0-1) */
00641  k_ds18b20_temp_mask bit = 0xFFFFE, /**< Mask for 11-bit (discard bit 0) */
00642  k_ds18b20_temp_mask bit = 0xFFFFF, /**< Mask for 12-bit (all bits valid) */
00643 } ds18b20_temp_mask_t;

```

6.1.14.10 ds18b20_write_idx_t

enum [ds18b20_write_idx_t](#) : uint8_t

DS18B20 scratchpad write buffer indices.

Enumerator

k_ds18b20_write_idx_th	TH (high alarm) register index
k_ds18b20_write_idx_tl	TL (low alarm) register index
k_ds18b20_write_idx_config	Configuration register index

Definition at line 661 of file [rx_ds18b20.h](#).

```

00661      : uint8_t {
00662  k_ds18b20_write_idx_th = 0, /**< TH (high alarm) register index */
00663  k_ds18b20_write_idx_tl = 1, /**< TL (low alarm) register index */
00664  k_ds18b20_write_idx_config = 2, /**< Configuration register index */
00665 } ds18b20_write_idx_t;

```

6.1.15 Function Documentation

6.1.15.1 rx_ds18b20_deinit()

rx_err_t rx_ds18b20_deinit (
 [rx_ds18b20_handle_t](#) * handle) [nodiscard]

Deinitialize DS18B20 sensor.

Parameters

in	<i>handle</i>	Pointer to initialized handle. Must not be nullptr.
----	---------------	---

Returns

`k_rx_ok` on success
`k_rx_err_null_ptr` if handle is nullptr
`k_rx_err_invalid_state` if not initialized

Deinitialize DS18B20 sensor.

Tears down a previously initialized DS18B20 handle by zeroing all internal state via `memset`. After this call the handle is safe to reinitialize with `rx_ds18b20_init()`.

Algorithm steps:

1. Validate handle is not nullptr
2. Verify handle is currently initialized
3. Clear the entire handle structure with `memset` (zeros all fields)
4. Log deinitialization success

Parameters

in,out	<i>handle</i>	Sensor handle to deinitialize (must be initialized, non-null)
--------	---------------	---

Returns

`rx_err_t` Error code

Return values

<i>k_rx_ok</i>	Handle successfully cleared
<i>k_rx_err_null_ptr</i>	handle pointer is nullptr
<i>k_rx_err_invalid_state</i>	handle was not previously initialized

Precondition

handle must be a valid non-null pointer
handle must have been successfully initialized via `rx_ds18b20_init()`

Postcondition

All handle fields are zeroed (initialized flag is false)
handle may be safely reused with `rx_ds18b20_init()`

Note

Not thread-safe; caller must ensure no concurrent access to handle

Example:

```
rx_ds18b20_handle_t sensor;
rx_ds18b20_init(&sensor, &config);
// ... use sensor ...
rx_err_t err = rx_ds18b20_deinit(&sensor);
if (err != k_rx_ok) {
    rx_log_error("app", "Failed to deinit DS18B20");
}
```

See also

[rx_ds18b20_init\(\)](#) Initialize the handle before use

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

Definition at line 441 of file [rx_ds18b20.c](#).

```
00442 {
00443     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00444
00445     if (!handle->initialized) {
00446         rx_log_warn(s_tag, "DS18B20 not initialized");
00447         return k_rx_err_invalid_state;
00448     }
00449
00450     /* Clear handle */
00451     *handle = (rx_ds18b20_handle_t){0};
00452
00453     rx_log_info(s_tag, "DS18B20 deinitialized");
00454     return k_rx_ok;
00455 }
```

References [rx_ds18b20_handle_t::initialized](#), and [s_tag](#).

6.1.15.2 rx_ds18b20_get_conversion_time_ms()

```
uint32_t rx_ds18b20_get_conversion_time_ms (
    const rx\_ds18b20\_handle\_t * handle)
```

Get conversion time for current resolution.

Returns the maximum conversion time in milliseconds for the configured resolution. Caller should wait this long after triggering conversion before reading temperature.

Parameters

in	<i>handle</i>	Pointer to initialized handle. Must not be nullptr.
----	---------------	---

Returns

Conversion time in milliseconds, or 0 if handle is nullptr or not initialized

Get conversion time for current resolution.

Returns the required wait time after triggering conversion based on the configured resolution (9-bit: 94ms, 10-bit: 188ms, 11-bit: 375ms, 12-bit: 750ms).

Parameters

in	<i>handle</i>	Sensor handle
----	---------------	---------------

Returns

Conversion time in milliseconds, or invalid value if handle is nullptr/uninitialized

Definition at line 1204 of file [rx_ds18b20.c](#).

```

01205 {
01206     if (handle == nullptr || !handle->initialized) {
01207         return s_ds18b20_conversion_time_invalid_u32;
01208     }
01209
01210     switch (handle->resolution) {
01211     case k_ds18b20_resolution_bit:
01212         return k_ds18b20_conv_time_bit_ms;
01213     case k_ds18b20_resolution_bit:
01214         return k_ds18b20_conv_time_bit_ms;
01215     case k_ds18b20_resolution_bit:
01216         return k_ds18b20_conv_time_bit_ms;
01217     case k_ds18b20_resolution_bit:
01218         return k_ds18b20_conv_time_bit_ms;
01219     default:
01220         return s_ds18b20_conversion_time_invalid_u32;
01221     }
01222 }
```

References [rx_ds18b20_handle_t::initialized](#), [k_ds18b20_conv_time_bit_ms](#), [k_ds18b20_conv_time_bit_ms](#), [k_ds18b20_conv_time_bit_ms](#), [k_ds18b20_conv_timeobit_ms](#), [k_ds18b20_resolution_bit](#), [k_ds18b20_resolution_bit](#), [k_ds18b20_resolution_bit](#), [k_ds18b20_resolutionbit](#), [rx_ds18b20_handle_t::resolution](#), and [s_ds18b20_conversion_time_invalid_u32](#)

6.1.15.3 rx_ds18b20_get_resolution()

```

rx_err_t rx_ds18b20_get_resolution (
    const rx\_ds18b20\_handle\_t * handle,
    ds18b20\_resolution\_t * resolution) [nodiscard]
```

Get current temperature resolution.

Parameters

in	<i>handle</i>	Pointer to initialized handle. Must not be nullptr.
out	<i>resolution</i>	Pointer to store resolution. Must not be nullptr.

Returns

`k_rx_ok` on success
`k_rx_err_null_ptr` if any pointer is nullptr
`k_rx_err_invalid_state` if not initialized

Get current temperature resolution.

Returns the resolution value stored in the handle, which reflects the last successfully applied resolution (set during initialization or via [rx_ds18b20_set_resolution\(\)](#)). This is a cached read from the handle state; it does not issue a 1-Wire bus transaction.

Algorithm steps:

1. Validate handle and output pointer are non-null
2. Verify handle is initialized
3. Copy handle->resolution to *resolution

Parameters

in	<i>handle</i>	Sensor handle (must be initialized, non-null)
out	<i>resolution</i>	Receives the currently configured ds18b20_resolution_t value; undefined on error

Returns

`rx_err_t` Error code

Return values

<i>k_rx_ok</i>	Resolution successfully returned
<i>k_rx_err_null_ptr</i>	handle or resolution pointer is nullptr
<i>k_rx_err_invalid_state</i>	handle not initialized (CHECK_DS18B20_HANDLE fails)

Precondition

handle must be initialized via [rx_ds18b20_init\(\)](#)
resolution must be a valid non-null pointer

Postcondition

*resolution contains the current resolution value on `k_rx_ok`
handle state is unchanged

Note

Not thread-safe; caller must ensure handle is not concurrently modified
Returns cached value from handle – does not issue 1-Wire bus transaction

Example:

```
ds18b20_resolution_t res;
rx_err_t err = rx_ds18b20_get_resolution(&sensor, &res);
if (err == k_rx_ok) {
    uint32_t conv_ms = rx_ds18b20_get_conversion_time_ms(&sensor);
    rx_log_info("app", "Resolution %d, conv time %u ms", (int)res, (unsigned)conv_ms);
}
```

See also

[rx_ds18b20_set_resolution\(\)](#) Change the ADC resolution

[rx_ds18b20_get_conversion_time_ms\(\)](#) Get conversion time for current resolution

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

Definition at line 858 of file [rx_ds18b20.c](#).

```
00860 {
00861     CHECK_DS18B20_HANDLE(handle, s_tag);
00862     RX_CHECK_NULL_PTR(resolution, s_tag, "resolution is nullptr");
00863
00864     *resolution = handle->resolution;
00865     return k_rx_ok;
00866 }
```

References [CHECK_DS18B20_HANDLE](#), [rx_ds18b20_handle_t::resolution](#), and [s_tag](#).

6.1.15.4 rx_ds18b20_init()

```
rx_err_t rx_ds18b20_init (
    rx_ds18b20_handle_t * handle,
    const rx_ds18b20_config_t * config) [nodiscard]
```

Initialize DS18B20 sensor.

Initializes the DS18B20 sensor and configures resolution. If ROM matching is disabled, uses Skip ROM (single device mode).

Parameters

out	<i>handle</i>	Pointer to DS18B20 handle. Must not be nullptr.
in	<i>config</i>	Pointer to configuration. Must not be nullptr.

Returns

k_rx_ok on success
 k_rx_err_null_ptr if handle or config is nullptr
 k_rx_err_invalid_arg if bus_manager or bus_name is nullptr
 k_rx_err_invalid_state if device not responding
 k_rx_err_crc if scratchpad CRC check fails

Definition at line 356 of file [rx_ds18b20.c](#).

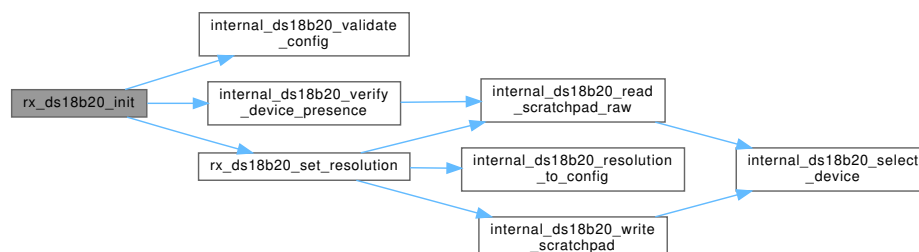
```

00357 {
00358     /* Validate inputs */
00359     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00360
00361     rx_err_t err = internal_ds18b20_validate_config(config, handle);
00362     if (err != k_rx_ok) {
00363         return err;
00364     }
00365
00366     /* Initialize handle */
00367     *handle = (rx_ds18b20_handle_t){0};
00368     handle->bus_manager = config->bus_manager;
00369     handle->bus_name = config->bus_name;
00370     handle->resolution = config->resolution;
00371     handle->use_rom_matching = config->use_rom_matching;
00372
00373     if (config->use_rom_matching) {
00374         memcpy(handle->rom, config->rom, k_onewire_rom_bytes);
00375     }
00376
00377     /* Verify device presence */
00378     err = internal_ds18b20_verify_device_presence(handle);
00379     if (err != k_rx_ok) {
00380         return err;
00381     }
00382
00383     /* Configure resolution */
00384     handle->initialized = true;
00385     err = rx_ds18b20_set_resolution(handle, config->resolution);
00386     if (err != k_rx_ok) {
00387         handle->initialized = false;
00388         return err;
00389     }
00390
00391     rx_log_info(s_tag, "DS18B20 initialized successfully");
00392     return k_rx_ok;
00393 }

```

References [rx_ds18b20_config_t::bus_manager](#), [rx_ds18b20_handle_t::bus_manager](#), [rx_ds18b20_config_t::bus_name](#), [rx_ds18b20_handle_t::bus_name](#), [rx_ds18b20_handle_t::initialized](#), [internal_ds18b20_validate_config\(\)](#), [internal_ds18b20_verify_device_presence\(\)](#), [rx_ds18b20_config_t::resolution](#), [rx_ds18b20_handle_t::resolution](#), [rx_ds18b20_config_t::rom](#), [rx_ds18b20_handle_t::rom](#), [rx_ds18b20_set_resolution\(\)](#), [s_tag](#), [rx_ds18b20_config_t::use_rom_matching](#), and [rx_ds18b20_handle_t::use_rom_matching](#).

Here is the call graph for this function:



6.1.15.5 rx_ds18b20_read_power_mode()

```
rx_err_t rx_ds18b20_read_power_mode (
    const rx_ds18b20_handle_t * handle,
    bool * external_power) [nodiscard]
```

Read power supply mode.

Checks if sensor is using parasitic power or external power.

Parameters

in	<i>handle</i>	Pointer to initialized handle. Must not be nullptr.
----	---------------	---

out	<i>external_power</i>	True if external power, false if parasitic. Must not be nullptr.
-----	-----------------------	--

Returns

- k_rx_ok* on success
- k_rx_err_null_ptr* if any pointer is nullptr
- k_rx_err_invalid_state* if not initialized or device not present

Read power supply mode.

Issues the Read Power Supply (0xB4) command over the 1-Wire bus and reads back a single bit to determine whether the DS18B20 is operating on an external voltage supply or in parasitic (bus-powered) mode.

Per the DS18B20 datasheet: the sensor pulls the bus low for one read time slot to indicate parasitic power mode (0), or releases the bus (1) to indicate external power mode.

Algorithm steps:

1. Validate handle and output pointer are non-null; handle is initialized
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Read Power Supply command byte (0xB4)
5. Read one bit from the bus (0 = parasitic, 1 = external)
6. Write result to **external_power*

Parameters

in	<i>handle</i>	Sensor handle (must be initialized, non-null)
out	<i>external_power</i>	Set to true if device uses external power supply, false if device uses parasitic (bus-powered) mode; undefined on error

Returns

- rx_err_t* Error code

Return values

<i>k_rx_ok</i>	Power mode successfully read
<i>k_rx_err_null_ptr</i>	handle or external_power pointer is nullptr
<i>k_rx_err_invalid_state</i>	handle not initialized or device not present on bus
<i>k_rx_err_bus_error</i>	1-Wire bus reset, write, or read operation failed

Precondition

handle must be initialized via [rx_ds18b20_init\(\)](#)
 DS18B20 device must be physically connected to the bus

Postcondition

*external_power reflects the device power supply mode on k_rx_ok
 handle state is unchanged

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Some operations (Copy Scratchpad) are unreliable in parasitic power mode

Example:

```
bool ext_power = false;
rx_err_t err = rx_ds18b20_read_power_mode(&sensor, &ext_power);
if (err == k_rx_ok && !ext_power) {
    rx_log_warn("app", "DS18B20 in parasitic mode: EEPROM writes may fail");
}
```

See also

[rx_ds18b20_save_config\(\)](#) EEPROM write (requires external power)
[rx_ds18b20_init\(\)](#) Initialize sensor handle

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null checks/initialized, device present), 2 postconditions

Definition at line 1083 of file [rx_ds18b20.c](#).

```
01084 {
01085     CHECK_DS18B20_HANDLE(handle, s_tag);
01086     RX_CHECK_NULL_PTR(external_power, s_tag, "external_power is nullptr");
01087
01088     /* Reset and check presence */
01089     bool presence = false;
01090     rx_err_t err = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
01091     if (err != k_rx_ok || !presence) {
01092         rx_log_error(s_tag, "Device not present for power mode read");
01093         return k_rx_err_invalid_state;
01094     }
01095
01096     /* Select device */
01097     err = internal_ds18b20_select_device(handle);
01098     if (err != k_rx_ok) {
01099         return err;
01100     }
01101
01102     /* Send Read Power Supply command */
```

```

01103 err = rx_bus_owewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_read_power);
01104 if (err != k_rx_ok) {
01105     rx_log_error(s_tag, "Failed to send read power command");
01106     return err;
01107 }
01108
01109 /* Read power bit (1 = external, 0 = parasitic) */
01110 bool power_bit = false;
01111 err = rx_bus_owewire_read_bit(handle->bus_manager, handle->bus_name, &power_bit);
01112 if (err != k_rx_ok) {
01113     rx_log_error(s_tag, "Failed to read power bit");
01114     return err;
01115 }
01116
01117 *external_power = power_bit;
01118 return k_rx_ok;
01119 }

```

References [rx_ds18b20_handle_t::bus_manager](#), [rx_ds18b20_handle_t::bus_name](#), [CHECK_DS18B20_HANDLE](#), [internal_ds18b20_select_device\(\)](#), [k_ds18b20_cmd_read_power](#), and [s_tag](#).

Here is the call graph for this function:



6.1.15.6 rx_ds18b20_read_scratchpad()

```

rx_err_t rx_ds18b20_read_scratchpad (
    rx_ds18b20_handle_t * handle,
    uint8_t scratchpad[k_ds18b20_scratchpad_bytes]) [nodiscard]

```

Read scratchpad memory.

Reads the entire 9-byte scratchpad (8 data bytes + 1 CRC byte). Performs CRC validation.

Parameters

in	<i>handle</i>	Pointer to initialized handle. Must not be nullptr.
out	<i>scratchpad</i>	Pointer to 9-byte buffer. Must not be nullptr.

Returns

- `k_rx_ok` on success
- `k_rx_err_null_ptr` if any pointer is nullptr
- `k_rx_err_invalid_state` if not initialized or device not present
- `k_rx_err_crc` if CRC check fails

Read scratchpad memory.

Reads all nine bytes of the DS18B20 scratchpad memory and returns the raw buffer to the caller. The scratchpad layout is:

Byte	Field	Description
0	Temperature LSB	Low byte of temperature register

Byte	Field	Description
1	Temperature MSB	High byte of temperature register
2	TH Register	High alarm threshold
3	TL Register	Low alarm threshold
4	Config Register	Resolution configuration byte
5-7	Reserved	Reserved (0xFF, 0x10, 0x10)
8	CRC	CRC-8 of bytes 0-7

CRC is verified internally by [internal_ds18b20_read_scratchpad_raw\(\)](#) before the buffer is returned. This function is a thin public wrapper around the internal helper.

Algorithm steps:

1. Validate handle and scratchpad buffer pointer are non-null
2. Verify handle is initialized
3. Delegate to [internal_ds18b20_read_scratchpad_raw\(\)](#) which issues the 1-Wire reset, Match/Skip ROM selection, Read Scratchpad (0xBE) command, reads 9 bytes, and verifies CRC-8

Parameters

in,out	<i>handle</i>	Sensor handle (must be initialized, non-null)
out	<i>scratchpad</i>	Buffer of exactly <code>k_ds18b20_scratchpad_bytes</code> (9) bytes to receive the raw scratchpad contents; undefined on error

Returns

`rx_err_t` Error code

Return values

<i>k_rx_ok</i>	Scratchpad successfully read and CRC verified
<i>k_rx_err_null_ptr</i>	handle or scratchpad buffer is nullptr
<i>k_rx_err_invalid_state</i>	handle not initialized
<i>k_rx_err_crc_failure</i>	Scratchpad CRC-8 check failed (data corrupted)
<i>k_rx_err_bus_error</i>	1-Wire bus communication failure

Precondition

handle must be initialized via [rx_ds18b20_init\(\)](#)

scratchpad must point to a buffer of at least `k_ds18b20_scratchpad_bytes` bytes

Postcondition

scratchpad[0..8] contains valid data on `k_rx_ok`

CRC of scratchpad bytes 0-7 matches byte 8 on `k_rx_ok`

Note

Not thread-safe; caller must serialize access to the same handle

Use [rx_ds18b20_read_temperature\(\)](#) for converted Celsius value

Example:

```
uint8_t raw[k_ds18b20_scratchpad_bytes];
rx_err_t err = rx_ds18b20_read_scratchpad(&sensor, raw);
if (err == k_rx_ok) {
    uint8_t config_reg = raw[4];
}
```

See also

[rx_ds18b20_read_temperature\(\)](#) High-level temperature read returning float Celsius

[rx_ds18b20_read_temperature_raw\(\)](#) Read raw 16-bit temperature with masking

Since

Version 1.0.0

NASA Power of 10 Compliance:

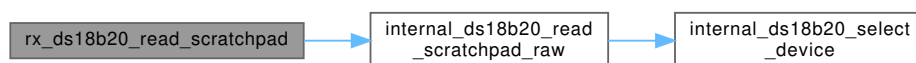
- Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

Definition at line 1185 of file [rx_ds18b20.c](#).

```
01187 {
01188     CHECK_DS18B20_HANDLE(handle, s_tag);
01189     RX_CHECK_NULL_PTR(scratchpad, s_tag, "scratchpad is nullptr");
01190
01191     return internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
01192 }
```

References [CHECK_DS18B20_HANDLE](#), [internal_ds18b20_read_scratchpad_raw\(\)](#), [k_ds18b20_scratchpad_bytes](#), and [s_tag](#).

Here is the call graph for this function:



6.1.15.7 rx_ds18b20_read_temperature()

```
rx_err_t rx_ds18b20_read_temperature (
    rx_ds18b20_handle_t * handle,
    float * temperature_celsius) [nodiscard]
```

Read temperature in Celsius.

Reads the scratchpad and converts raw temperature to Celsius. Temperature conversion must complete before calling (see conversion times).

Parameters

in	<i>handle</i>	Pointer to initialized handle. Must not be nullptr.
----	---------------	---

out	<i>temperature_celsius</i>	Pointer to store temperature in degC. Must not be nullptr.
-----	----------------------------	--

Returns

k_rx_ok on success
 k_rx_err_null_ptr if any pointer is nullptr
 k_rx_err_invalid_state if not initialized or device not present
 k_rx_err_crc if scratchpad CRC check fails

Read temperature in Celsius.

Reads the full 9-byte scratchpad from the DS18B20, extracts the 16-bit raw temperature value, applies the resolution mask to clear undefined low-order bits, validates the result is within the sensor's physical range, and converts to a floating-point Celsius value using the DS18B20 fixed-point encoding (1 LSB = 1/16 degC).

Algorithm steps:

1. Validate handle and output pointer are non-null
2. Call [rx_ds18b20_read_temperature_raw\(\)](#) to obtain masked raw value
3. Convert raw signed 16-bit value to float: $celsius = raw / 16.0f$
4. Write result to *temperature_celsius

Parameters

in,out	<i>handle</i>	Sensor handle (must be initialized)
out	<i>temperature_celsius</i>	Converted temperature in degrees Celsius, valid range [-55.0, +125.0]; undefined on error

Returns

rx_err_t Error code

Return values

<i>k_rx_ok</i>	Temperature read and converted successfully
<i>k_rx_err_null_ptr</i>	handle or temperature_celsius is nullptr
<i>k_rx_err_invalid_state</i>	handle not initialized
<i>k_rx_err_out_of_range</i>	raw temperature outside [-55degC, +125degC]
<i>k_rx_err_crc_failure</i>	Scratchpad CRC mismatch
<i>k_rx_err_bus_error</i>	1-Wire bus communication failure

Precondition

handle must be initialized via [rx_ds18b20_init\(\)](#)
[rx_ds18b20_trigger_conversion\(\)](#) must have been called and conversion time elapsed

Postcondition

*temperature_celsius contains valid temperature in [-55.0, +125.0] on k_rx_ok
 handle->stats.read_count incremented (via internal_ds18b20_read_scratchpad_raw)

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Returns stale data if conversion time has not elapsed after trigger

Example:

```
float temp_celsius = 0.0f;
rx_err_t err = rx_ds18b20_read_temperature(&sensor, &temp_celsius);
if (err == k_rx_ok) {
    rx_log_info("app", "Temperature: %.4f C", (double)temp_celsius);
}
```

See also

[rx_ds18b20_trigger_conversion\(\)](#) Must be called before reading
[rx_ds18b20_read_temperature_raw\(\)](#) Read the raw 16-bit value
[rx_ds18b20_get_conversion_time_ms\(\)](#) Determine required wait time

Since

Version 1.0.0

NASA Power of 10 Compliance:

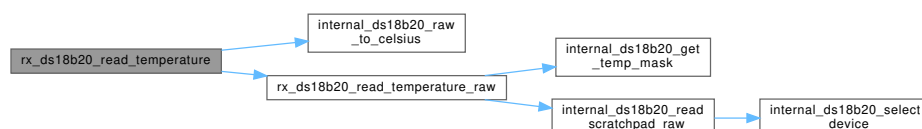
- Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

Definition at line 595 of file rx_ds18b20.c.

```
00596 {
00597     CHECK_DS18B20_HANDLE(handle, s_tag);
00598     RX_CHECK_NULL_PTR(temperature_celsius, s_tag, "temperature_celsius is nullptr");
00599
00600     /* Read raw temperature */
00601     int16_t raw_temp = 0;
00602     rx_err_t err = rx_ds18b20_read_temperature_raw(handle, &raw_temp);
00603     if (err != k_rx_ok) {
00604         return err;
00605     }
00606
00607     /* Convert to Celsius */
00608     *temperature_celsius = internal_ds18b20_raw_to_celsius(raw_temp);
00609
00610     return k_rx_ok;
00611 }
```

References [CHECK_DS18B20_HANDLE](#), [internal_ds18b20_raw_to_celsius\(\)](#), [rx_ds18b20_read_temperature_raw\(\)](#), and [s_tag](#).

Here is the call graph for this function:



6.1.15.8 rx_ds18b20_read_temperature_raw()

```
rx_err_t rx_ds18b20_read_temperature_raw (
    rx_ds18b20_handle_t * handle,
    int16_t * raw_temp) [nodiscard]
```

Read raw temperature value.

Reads the raw 16-bit temperature from scratchpad without conversion. Useful for debugging or custom temperature processing.

Parameters

in	<i>handle</i>	Pointer to initialized handle. Must not be nullptr.
out	<i>raw_temp</i>	Pointer to store raw temperature. Must not be nullptr.

Returns

- k_rx_ok on success
- k_rx_err_null_ptr if any pointer is nullptr
- k_rx_err_invalid_state if not initialized or device not present
- k_rx_err_crc if scratchpad CRC check fails

Read raw temperature value.

Reads the 9-byte DS18B20 scratchpad register, extracts the two temperature bytes (LSB at offset 0, MSB at offset 1), combines them into a 16-bit unsigned value, applies the resolution-dependent mask to clear undefined low-order bits, casts to signed int16_t, and range-validates the result.

The DS18B20 temperature encoding uses a signed fixed-point format where 1 LSB = 1/16 degC (12-bit mode). The valid raw range is:

- Minimum: -55 degC = 0xFC90 (int16: -880)
- Maximum: +125 degC = 0x07D0 (int16: +2000)

Resolution masks (clears undefined bits in lower-resolution modes):

- 9-bit: 0xFFFF8 (3 undefined bits)
- 10-bit: 0xFFFFC (2 undefined bits)
- 11-bit: 0xFFFFE (1 undefined bit)
- 12-bit: 0xFFFF (no undefined bits)

Algorithm steps:

1. Validate handle and output pointer are non-null and handle is initialized
2. Read raw scratchpad bytes via [internal_ds18b20_read_scratchpad_raw\(\)](#)
3. Combine temperature LSB and MSB bytes into uint16_t
4. Apply resolution mask from [internal_ds18b20_get_temp_mask\(\)](#)
5. Cast to int16_t and validate range [k_ds18b20_temp_min_raw, k_ds18b20_temp_max_raw]
6. Write validated result to *raw_temp

Parameters

in,out	<i>handle</i>	Sensor handle (must be initialized)
--------	---------------	-------------------------------------

out	<i>raw_temp</i>	Signed 16-bit raw temperature in 1/16 degC units, valid range [k_ds18b20_temp_min_raw, k_ds18b20_temp_max_raw]; undefined on error
-----	-----------------	--

Returns

`rx_err_t` Error code

Return values

<i>k_rx_ok</i>	Raw temperature read, masked, and validated successfully
<i>k_rx_err_null_ptr</i>	handle or raw_temp is nullptr
<i>k_rx_err_invalid_state</i>	handle not initialized
<i>k_rx_err_out_of_range</i>	raw value outside physical sensor range
<i>k_rx_err_crc_failure</i>	Scratchpad CRC check failed
<i>k_rx_err_bus_error</i>	1-Wire bus communication failure

Precondition

handle must be initialized via [rx_ds18b20_init\(\)](#)

[rx_ds18b20_trigger_conversion\(\)](#) must have been called and conversion time elapsed

Postcondition

*raw_temp is in [k_ds18b20_temp_min_raw, k_ds18b20_temp_max_raw] on k_rx_ok

Resolution mask has been applied; undefined bits are cleared

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Returns stale data if conversion time has not elapsed after trigger

Example:

```
int16_t raw = 0;
rx_err_t err = rx_ds18b20_read_temperature_raw(&sensor, &raw);
if (err == k_rx_ok) {
    float celsius = (float)raw / 16.0f;
}
```

See also

[rx_ds18b20_read_temperature\(\)](#) Convenience wrapper returning float Celsius

[rx_ds18b20_trigger_conversion\(\)](#) Must be called before reading

[rx_ds18b20_set_resolution\(\)](#) Configure ADC resolution and mask

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions (range, mask)

Definition at line 680 of file `rx_ds18b20.c`.

```

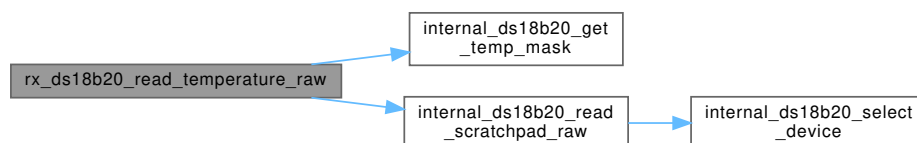
00681 {
00682     CHECK_DS18B20_HANDLE(handle, s_tag);
00683     RX_CHECK_NULL_PTR(raw_temp, s_tag, "raw_temp is nullptr");
00684
00685     /* Read scratchpad */
00686     uint8_t scratchpad[k_ds18b20_scratchpad_bytes];
00687     rx_err_t err = internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
00688     if (err != k_rx_ok) {
00689         return err;
00690     }
00691
00692     /* Extract temperature (LSB + MSB) */
00693     uint16_t temp = (uint16_t)scratchpad[k_ds18b20_scratch_temp_lsb];
00694     temp |= ((uint16_t)scratchpad[k_ds18b20_scratch_temp_msb]) « k_ds18b20_shift_byte;
00695
00696     /* Apply resolution mask to clear undefined bits */
00697     const uint16_t mask = internal_ds18b20_get_temp_mask(handle->resolution);
00698     temp &= mask;
00699
00700     /* Post-condition: Validate temperature is within sensor range */
00701     *raw_temp = (int16_t)temp;
00702     if (*raw_temp < k_ds18b20_temp_min_raw || *raw_temp > k_ds18b20_temp_max_raw) {
00703         rx_log_error(s_tag, "Temperature out of sensor range (-55degC to +125degC)");
00704         return k_rx_err_out_of_range;
00705     }
00706
00707     return k_rx_ok;
00708 }

```

References [CHECK_DS18B20_HANDLE](#), [internal_ds18b20_get_temp_mask\(\)](#), [internal_ds18b20_read_scratchpad_raw\(\)](#), [k_ds18b20_scratch_temp_lsb](#), [k_ds18b20_scratch_temp_msb](#), [k_ds18b20_scratchpad_bytes](#), [k_ds18b20_shift_byte](#), [k_ds18b20_temp_max_raw](#), [rx_ds18b20_handle_t::resolution](#), and [s_tag](#).

Referenced by [rx_ds18b20_read_temperature\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.1.15.9 rx_ds18b20_recall_config()

```
rx_err_t rx_ds18b20_recall_config (  
    const rx_ds18b20_handle_t * handle)  [nodiscard]
```

Recall configuration from EEPROM.

Restores scratchpad bytes 2-4 (TH, TL, Config) from EEPROM. Useful after power-up to restore saved settings.

Parameters

in	<i>handle</i>	Pointer to initialized handle. Must not be nullptr.
----	---------------	---

Returns

k_rx_ok on success
k_rx_err_null_ptr if handle is nullptr
k_rx_err_invalid_state if not initialized or device not present

Recall configuration from EEPROM.

Sends the Recall E2 (0xB8) command over the 1-Wire bus, which instructs the DS18B20 to copy the TH alarm, TL alarm, and configuration register bytes from non-volatile EEPROM back into volatile scratchpad RAM. This is the same operation the device performs automatically on power-up.

Use this function to discard unsaved scratchpad changes and revert to the last saved EEPROM state without cycling power.

Algorithm steps:

1. Validate handle is initialized and non-null
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Recall E2 command byte (0xB8)

Parameters

in	<i>handle</i>	Sensor handle (must be initialized, non-null)
----	---------------	---

Returns

rx_err_t Error code

Return values

<i>k_rx_ok</i>	Recall E2 command successfully issued
<i>k_rx_err_null_ptr</i>	handle is nullptr

<code>k_rx_err_invalid_state</code>	handle not initialized or device not present on bus
<code>k_rx_err_bus_error</code>	1-Wire bus reset or write operation failed

Precondition

handle must be initialized via [rx_ds18b20_init\(\)](#)

DS18B20 device must be physically connected to the bus

Postcondition

DS18B20 scratchpad TH, TL, and configuration bytes reflect last EEPROM save

Any unsaved scratchpad changes since last [rx_ds18b20_save_config\(\)](#) are discarded

Note

Not thread-safe; caller must serialize access to the same handle

The Recall E2 operation is performed automatically by the device on power-up

Example:

```
// Revert any unsaved configuration changes
rx_err_t err = rx_ds18b20_recall_config(&sensor);
if (err != k_rx_ok) {
    rx_log_error("app", "Failed to recall DS18B20 config from EEPROM");
}
```

See also

[rx_ds18b20_save_config\(\)](#) Persist scratchpad configuration to EEPROM

[rx_ds18b20_set_resolution\(\)](#) Modify the scratchpad configuration

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null/initialized, device present), 2 postconditions

Definition at line 999 of file [rx_ds18b20.c](#).

```
01000 {
01001     CHECK_DS18B20_HANDLE(handle, s_tag);
01002
01003     /* Reset and check presence */
01004     bool presence = false;
01005     rx_err_t err = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
01006     if (err != k_rx_ok || !presence) {
01007         rx_log_error(s_tag, "Device not present for EEPROM recall");
01008         return k_rx_err_invalid_state;
01009     }
01010
01011     /* Select device */
01012     err = internal_ds18b20_select_device(handle);
01013     if (err != k_rx_ok) {
01014         return err;
01015     }
01016 }
```

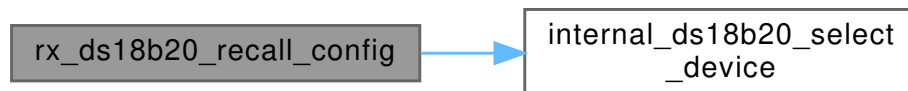
```

01015 }
01016
01017 /* Send Recall E2 command */
01018 err =
01019     rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_recall_eeprom);
01020 if (err != k_rx_ok) {
01021     rx_log_error(s_tag, "Failed to send recall EEPROM command");
01022     return err;
01023 }
01024
01025 return k_rx_ok;
01026 }

```

References [rx_ds18b20_handle_t::bus_manager](#), [rx_ds18b20_handle_t::bus_name](#), [CHECK_DS18B20_HANDLE](#), [internal_ds18b20_select_device\(\)](#), [k_ds18b20_cmd_recall_eeprom](#), and [s_tag](#).

Here is the call graph for this function:



6.1.15.10 rx_ds18b20_save_config()

```

rx_err_t rx_ds18b20_save_config (
    const rx_ds18b20_handle_t * handle) [nodiscard]

```

Save configuration to EEPROM.

Copies scratchpad bytes 2-4 (TH, TL, Config) to non-volatile EEPROM. Values persist across power cycles.

Parameters

in	<i>handle</i>	Pointer to initialized handle. Must not be nullptr.
----	---------------	---

Returns

k_rx_ok on success
 k_rx_err_null_ptr if handle is nullptr
 k_rx_err_invalid_state if not initialized or device not present

Save configuration to EEPROM.

Sends the Copy Scratchpad (0x48) command over the 1-Wire bus, which instructs the DS18B20 to copy the TH alarm, TL alarm, and configuration register bytes from volatile scratchpad RAM to non-volatile EEPROM. The saved values persist across power cycles and are automatically restored on the next power-up via the Recall E2 sequence.

Algorithm steps:

1. Validate handle is initialized and non-null
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Copy Scratchpad command byte (0x48)

Parameters

in	<i>handle</i>	Sensor handle (must be initialized, non-null)
----	---------------	---

Returns

`rx_err_t` Error code

Return values

<code>k_rx_ok</code>	Copy Scratchpad command successfully issued
----------------------	---

<i>k_rx_err_null_ptr</i>	handle is nullptr
<i>k_rx_err_invalid_state</i>	handle not initialized or device not present on bus
<i>k_rx_err_bus_error</i>	1-Wire bus reset or write operation failed

Precondition

handle must be initialized via [rx_ds18b20_init\(\)](#)

DS18B20 device must be physically connected and externally powered (Copy Scratchpad is not guaranteed in parasitic power mode)

Postcondition

DS18B20 EEPROM contains current TH, TL, and configuration register values

Configuration will survive next power cycle

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Not reliable in parasitic power mode; use external power for EEPROM writes

Example:

```
// Configure resolution, then persist to EEPROM
rx_ds18b20_set_resolution(&sensor, k_ds18b20_resolution_bit);
rx_err_t err = rx_ds18b20_save_config(&sensor);
if (err != k_rx_ok) {
    rx_log_error("app", "Failed to save DS18B20 config to EEPROM");
}
```

See also

[rx_ds18b20_recall_config\(\)](#) Restore EEPROM contents to scratchpad

[rx_ds18b20_set_resolution\(\)](#) Configure resolution before saving

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null/initialized, device present), 2 postconditions

Definition at line 919 of file [rx_ds18b20.c](#).

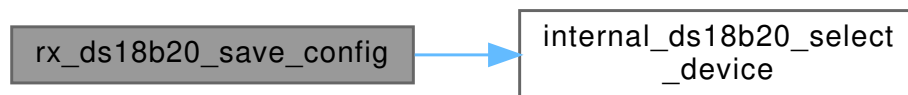
```

00920 {
00921     CHECK_DS18B20_HANDLE(handle, s_tag);
00922
00923     /* Reset and check presence */
00924     bool    presence = false;
00925     rx_err_t err      = rx_bus_owewire_reset(handle->bus_manager, handle->bus_name, &presence);
00926     if (err != k_rx_ok || !presence) {
00927         rx_log_error(s_tag, "Device not present for EEPROM save");
00928         return k_rx_err_invalid_state;
00929     }
00930
00931     /* Select device */
00932     err = internal_ds18b20_select_device(handle);
00933     if (err != k_rx_ok) {
00934         return err;
00935     }
00936
00937     /* Send Copy Scratchpad command */
00938     err =
00939         rx_bus_owewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_copy_scratch);
00940     if (err != k_rx_ok) {
00941         rx_log_error(s_tag, "Failed to send copy scratchpad command");
00942         return err;
00943     }
00944
00945     return k_rx_ok;
00946 }

```

References [rx_ds18b20_handle_t::bus_manager](#), [rx_ds18b20_handle_t::bus_name](#), [CHECK_DS18B20_HANDLE](#), [internal_ds18b20_select_device\(\)](#), [k_ds18b20_cmd_copy_scratch](#), and [s_tag](#).

Here is the call graph for this function:



6.1.15.11 rx_ds18b20_set_resolution()

```

rx_err_t rx_ds18b20_set_resolution (
    rx_ds18b20_handle_t * handle,
    const ds18b20_resolution_t resolution) [nodiscard]

```

Set temperature resolution.

Changes the sensor resolution (9, 10, 11, or 12 bits). Higher resolution provides more precision but takes longer to convert.

Parameters

in	<i>handle</i>	Pointer to initialized handle. Must not be nullptr.
in	<i>resolution</i>	Resolution mode (9-12 bits)

Returns

- `k_rx_ok` on success
- `k_rx_err_null_ptr` if handle is nullptr
- `k_rx_err_invalid_arg` if resolution out of range
- `k_rx_err_invalid_state` if not initialized or device not present

Set temperature resolution.

Updates the configuration register in the DS18B20 scratchpad to change the ADC resolution. The existing TH and TL alarm threshold bytes are preserved by reading the current scratchpad first, then writing back only the updated configuration byte.

Higher resolution provides finer temperature steps but increases conversion time:

- 9-bit: 0.5 degC steps, 94 ms conversion
- 10-bit: 0.25 degC steps, 188 ms conversion
- 11-bit: 0.125 degC steps, 375 ms conversion
- 12-bit: 0.0625 degC steps, 750 ms conversion

Algorithm steps:

1. Validate handle is initialized and resolution is a legal enum value
2. Read current 9-byte scratchpad to preserve TH/TL alarm bytes
3. Convert resolution enum to configuration register byte via [internal_ds18b20_resolution_to_config\(\)](#)
4. Write scratchpad with new config byte (TH/TL unchanged)
5. Update handle->resolution to reflect new setting

Parameters

in,out	<i>handle</i>	Sensor handle (must be initialized); resolution field updated on success
in	<i>resolution</i>	New resolution setting (<code>k_ds18b20_resolutionnobit</code> through <code>k_ds18b20_resolutionbit</code>); all other values are invalid

Returns

`rx_err_t` Error code

Return values

<i>k_rx_ok</i>	Resolution updated in scratchpad and handle
<i>k_rx_err_null_ptr</i>	handle is nullptr
<i>k_rx_err_invalid_state</i>	handle not initialized
<i>k_rx_err_invalid_arg</i>	resolution is not a valid ds18b20_resolution_t value

<code>k_rx_err_crc_failure</code>	Scratchpad read CRC check failed
<code>k_rx_err_bus_error</code>	1-Wire bus communication failure during read or write

Precondition

handle must be initialized via [rx_ds18b20_init\(\)](#)

resolution must be one of `k_ds18b20_resolutionnobit/10bit/11bit/12bit`

Postcondition

handle->resolution reflects the newly configured value on `k_rx_ok`

DS18B20 scratchpad configuration register updated with new resolution bits

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Change is not persistent across power cycles unless [rx_ds18b20_save_config\(\)](#) is called

Example:

```
rx_err_t err = rx_ds18b20_set_resolution(&sensor, k_ds18b20_resolution_bit);
if (err == k_rx_ok) {
    // Save to EEPROM so it persists across power cycles
    rx_ds18b20_save_config(&sensor);
}
```

See also

[rx_ds18b20_get_resolution\(\)](#) Read back the currently configured resolution

[rx_ds18b20_save_config\(\)](#) Persist resolution change to EEPROM

[rx_ds18b20_get_conversion_time_ms\(\)](#) Get required conversion time for new resolution

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null/initialized, valid resolution), 2 postconditions

Definition at line 772 of file [rx_ds18b20.c](#).

```

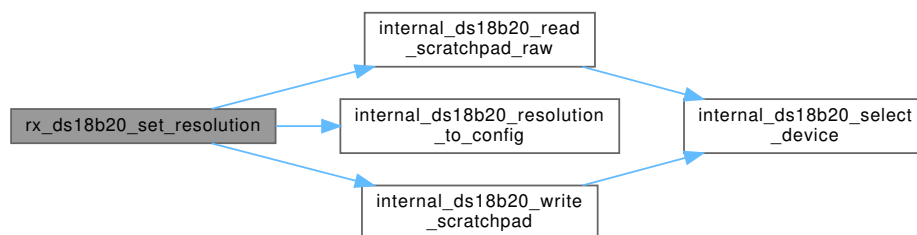
00774 {
00775     CHECK_DS18B20_HANDLE(handle, s_tag);
00776
00777     if (resolution != k_ds18b20_resolution_bit && resolution != k_ds18b20_resolution_bit &&
00778         resolution != k_ds18b20_resolution_bit && resolution != k_ds18b20_resolution_bit) {
00779         rx_log_error(s_tag, "Invalid resolution value");
00780         return k_rx_err_invalid_arg;
00781     }
00782
00783     /* Read current scratchpad */
00784     uint8_t scratchpad[k_ds18b20_scratchpad_bytes];
00785     rx_err_t err = internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
00786     if (err != k_rx_ok) {
00787         return err;
00788     }
00789
00790     /* Generate new config byte */
00791     const uint8_t config = internal_ds18b20_resolution_to_config(resolution);
00792
00793     /* Write scratchpad with new config */
00794     ds18b20_scratchpad_write_t write_cfg;
00795     write_cfg.th = scratchpad[k_ds18b20_scratch_th_reg];
00796     write_cfg.tl = scratchpad[k_ds18b20_scratch_tl_reg];
00797     write_cfg.config = config;
00798     err = internal_ds18b20_write_scratchpad(handle, &write_cfg);
00799     if (err != k_rx_ok) {
00800         return err;
00801     }
00802
00803     /* Update handle */
00804     handle->resolution = resolution;
00805
00806     return k_rx_ok;
00807 }

```

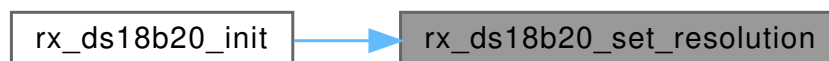
References [CHECK_DS18B20_HANDLE](#), [ds18b20_scratchpad_write_t::config](#), [internal_ds18b20_read_scratchpad_raw\(\)](#), [internal_ds18b20_resolution_to_config\(\)](#), [internal_ds18b20_write_scratchpad\(\)](#), [k_ds18b20_resolution_bit](#), [k_ds18b20_resolution_bit](#), [k_ds18b20_resolution_bit](#), [k_ds18b20_resolutionbit](#), [k_ds18b20_scratch_th_reg](#), [k_ds18b20_scratch_tl_reg](#), [k_ds18b20_scratchpad_bytes](#), [rx_ds18b20_handle_t::resolution](#), [s_tag](#), [ds18b20_scratchpad_write_t::th](#), and [ds18b20_scratchpad_write_t::tl](#).

Referenced by [rx_ds18b20_init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.1.15.12 rx_ds18b20_trigger_conversion()

```
rx_err_t rx_ds18b20_trigger_conversion (  
    const rx\_ds18b20\_handle\_t * handle) [nodiscard]
```

Trigger temperature conversion.

Sends Convert T command to start temperature measurement. Caller must wait for conversion time before reading temperature.

Conversion times:

- 9-bit: 100ms
- 10-bit: 200ms
- 11-bit: 400ms
- 12-bit: 800ms

Parameters

in	<i>handle</i>	Pointer to initialized handle. Must not be nullptr.
----	---------------	---

Returns

- k_rx_ok on success
- k_rx_err_null_ptr if handle is nullptr
- k_rx_err_invalid_state if not initialized or device not present

Trigger temperature conversion.

Issues the Convert T (0x44) command over the 1-Wire bus to start an autonomous temperature measurement. The DS18B20 will perform the ADC conversion internally; the caller must wait the appropriate conversion time before reading results with [rx_ds18b20_read_temperature\(\)](#).

Algorithm steps:

1. Validate handle is initialized and non-null
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Convert T command byte (0x44) to the bus

Conversion time depends on resolution:

- 9-bit: 94 ms
- 10-bit: 188 ms
- 11-bit: 375 ms
- 12-bit: 750 ms

Parameters

in	<i>handle</i>	Sensor handle (must be initialized, non-null)
----	---------------	---

Returns

rx_err_t Error code

Return values

<i>k_rx_ok</i>	Conversion command issued successfully
----------------	--

<i>k_rx_err_null_ptr</i>	handle is nullptr
<i>k_rx_err_invalid_state</i>	handle not initialized or device not present on bus
<i>k_rx_err_bus_error</i>	1-Wire bus reset or write operation failed

Precondition

handle must be initialized via [rx_ds18b20_init\(\)](#)

DS18B20 device must be physically connected and powered on the bus

Postcondition

DS18B20 ADC conversion is in progress

Caller must wait [rx_ds18b20_get_conversion_time_ms\(\)](#) before reading

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Do not call [rx_ds18b20_read_temperature\(\)](#) before the conversion time elapses

Example:

```
rx_err_t err = rx_ds18b20_trigger_conversion(&sensor);
if (err == k_rx_ok) {
    tx_thread_sleep(rx_ds18b20_get_conversion_time_ms(&sensor));
    float temp_celsius = 0.0f;
    err = rx_ds18b20_read_temperature(&sensor, &temp_celsius);
}
```

See also

[rx_ds18b20_read_temperature\(\)](#) Read converted temperature in Celsius

[rx_ds18b20_get_conversion_time_ms\(\)](#) Get required wait time after conversion

[rx_ds18b20_set_resolution\(\)](#) Configure ADC resolution

Since

Version 1.0.0

NASA Power of 10 Compliance:

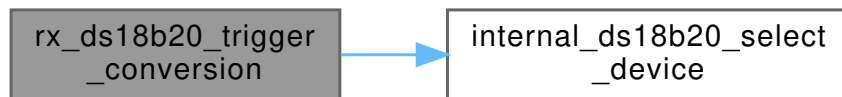
- Rule 5: 2 preconditions (initialized handle, device present), 2 postconditions

Definition at line 513 of file rx_ds18b20.c.

```
00514 {
00515     CHECK_DS18B20_HANDLE(handle, s_tag);
00516
00517     /* Reset and check presence */
00518     bool    presence = false;
00519     rx_err_t err      = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
00520     if (err != k_rx_ok || !presence) {
00521         rx_log_error(s_tag, "Device not present for conversion trigger");
00522         return k_rx_err_invalid_state;
00523     }
00524
00525     /* Select device */
00526     err = internal_ds18b20_select_device(handle);
00527     if (err != k_rx_ok) {
00528         return err;
00529     }
00530
00531     /* Send Convert T command */
00532     err = rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_convert_t);
00533     if (err != k_rx_ok) {
00534         rx_log_error(s_tag, "Failed to send conversion command");
00535         return err;
00536     }
00537
00538     return k_rx_ok;
00539 }
```

References [rx_ds18b20_handle_t::bus_manager](#), [rx_ds18b20_handle_t::bus_name](#), [CHECK_DS18B20_HANDLE](#), [internal_ds18b20_select_device\(\)](#), [k_ds18b20_cmd_convert_t](#), and [s_tag](#).

Here is the call graph for this function:



6.2 rx_ds18b20.h

[Go to the documentation of this file.](#)

```
00001 /* lib/rx_ds18b20/inc/rx_ds18b20.h */
00002
00003 /**
00004  * @file rx_ds18b20.h
00005  * @brief DS18B20 1-Wire Digital Temperature Sensor Driver API
00006  *
00007  * @details
00008  * # Overview
00009  *
00010  * Platform-independent driver for Maxim DS18B20 programmable resolution 1-Wire
00011  * digital thermometer with CRC-8 data integrity checking. Provides high-level
00012  * API for temperature measurement, resolution configuration, and EEPROM
00013  * persistence with automatic error handling and validation.
00014  *
00015  * This driver abstracts the DS18B20's complex 1-Wire protocol into simple init,
00016  * trigger, and read operations while maintaining full access to advanced features
00017  * like ROM matching (multi-drop buses) and alarm thresholds.
00018  *
00019  * # Key Features
00020  *
00021  * ## Measurement Capabilities
00022  * - **Temperature range**: -55degC to +125degC with +/-0.5degC accuracy (-10degC to +85degC)
00023  * - **Programmable resolution**: 9, 10, 11, or 12 bits
00024  * - **Conversion times**: 94ms (9-bit) to 750ms (12-bit)
00025  * - **CRC-8 validation**: Detects transmission errors (polynomial x^8 + x^5 + x^4 + 1)
00026  *
00027  * ## Resolution vs. Precision vs. Conversion Time Trade-off
```

```

00028 *
00029 * | Resolution | Precision | Conversion Time | Use Case |
00030 * |-----|-----|-----|-----|
00031 * | 9-bit      | 0.5degC    | ~94ms       | Fast updates, low precision needed |
00032 * | 10-bit     | 0.25degC   | ~188ms      | Balanced for most applications |
00033 * | 11-bit     | 0.125degC  | ~375ms      | High precision, moderate speed |
00034 * | 12-bit     | 0.0625degC | ~750ms      | Maximum precision, slow updates |
00035 *
00036 * ## Advanced Features
00037 * - **ROM matching**: Supports multi-device 1-Wire buses (up to 255 sensors)
00038 * - **Skip ROM mode**: Optimized for single-sensor configurations
00039 * - **EEPROM storage**: Persist resolution/alarm settings across power cycles
00040 * - **Power mode detection**: Identify parasitic vs. external power operation
00041 * - **Alarm thresholds**: Programmable TH/TL registers (not exposed in this API)
00042 *
00043 * # System Architecture Context
00044 *
00045 * The DS18B20 driver sits at the sensor layer of the STAR firmware stack:
00046 *
00047 * @code{.unparsed}
00048 * +-----+
00049 * | Application Layer (Tasks) |
00050 * | - Temperature monitoring task |
00051 * | - Environmental control |
00052 * +-----+
00053 * |
00054 * +-----v-----+
00055 * | Sensor Driver Layer |
00056 * | +-----+ |
00057 * | | rx_ds18b20 (This Module) | |
00058 * | | - Handle management | |
00059 * | | - CRC validation | |
00060 * | | - Temperature conversion | |
00061 * | +-----+ |
00062 * +-----+
00063 * |
00064 * +-----v-----+
00065 * | Bus Manager Layer (rx_bus_manager) |
00066 * | - Abstraction for I2C, SPI, 1-Wire, etc. |
00067 * | - Dependency injection for testability |
00068 * +-----+
00069 * |
00070 * +-----v-----+
00071 * | 1-Wire HAL (rx_bus_owewire) |
00072 * | - Bit-level 1-Wire protocol |
00073 * | - Reset/presence detection |
00074 * | - ROM commands (Match/Skip/Search) |
00075 * +-----+
00076 * |
00077 * +-----v-----+
00078 * | Hardware Layer (GPIO, Timers) |
00079 * | - Microsecond-precision timing |
00080 * | - Open-drain GPIO control |
00081 * +-----+
00082 * @endcode
00083 *
00084 * # Design Rationale
00085 *
00086 * ## Dependency Injection Pattern
00087 *
00088 * The driver accepts a `rx_bus_manager_t` instead of directly accessing GPIO,
00089 * enabling:
00090 * - **Unit testing** with mock bus implementations (no hardware required)
00091 * - **Hardware independence** - works on any 1-Wire HAL implementation
00092 * - **Multi-bus support** - single driver code for multiple 1-Wire buses
00093 *
00094 * This follows the **Dependency Inversion Principle** (high-level modules depend
00095 * on abstractions, not concrete implementations).
00096 *
00097 * ## Blocking API Design
00098 *
00099 * All operations are blocking (caller must wait for conversion). Rationale:
00100 * - DS18B20 conversions are inherently slow (up to 750ms)
00101 * - Non-blocking would require state machines and polling infrastructure
00102 * - Simplicity favored: caller can use ThreadX semaphores if async needed
00103 * - Typical usage: Dedicated temperature monitoring task with periodic reads
00104 *
00105 * ## Error Handling Strategy
00106 *
00107 * - **All functions return `rx_err_t`** for consistent error propagation
00108 * - **CRC validation on every read** - detect bus noise/corruption
00109 * - **Range checking** - reject impossible temperatures (-880 to +2000 raw)
00110 * - **nullptr checks** - fail fast on API misuse
00111 * - **State validation** - ensure init before use, prevent double-init
00112 *
00113 * # Implementation Approach
00114 *

```

```

00115 * ## Temperature Conversion Process
00116 *
00117 * The DS18B20 uses a two-phase measurement cycle:
00118 *
00119 * 1. **Trigger Phase** ('rx_ds18b20_trigger_conversion()'):
00120 *   - Send Convert T command (0x44)
00121 *   - DS18B20 begins analog-to-digital conversion
00122 *   - Returns immediately (non-blocking)
00123 *
00124 * 2. **Wait Phase** (Caller's responsibility):
00125 *   - Wait for conversion time based on resolution
00126 *   - Use 'rx_ds18b20_get_conversion_time_ms()' to get exact time
00127 *   - ThreadX: 'tx_thread_sleep(time_ms * TX_TIMER_TICKS_PER_SECOND / 1000)'
00128 *
00129 * 3. **Read Phase** ('rx_ds18b20_read_temperature()'):
00130 *   - Read 9-byte scratchpad (includes CRC)
00131 *   - Validate CRC-8 checksum
00132 *   - Extract 16-bit raw temperature
00133 *   - Apply resolution mask (clear undefined bits)
00134 *   - Convert to Celsius (raw / 16.0)
00135 *
00136 * ## Memory Layout and Scratchpad Structure
00137 *
00138 * The DS18B20's scratchpad RAM is 9 bytes:
00139 *
00140 * | Byte | Name           | Description           |
00141 * |-----|-----|-----|
00142 * | 0     | Temp LSB       | Temperature low byte  |
00143 * | 1     | Temp MSB       | Temperature high byte (+ sign) |
00144 * | 2     | TH Register    | High alarm threshold  |
00145 * | 3     | TL Register    | Low alarm threshold   |
00146 * | 4     | Config         | Resolution bits [6:5] |
00147 * | 5     | Reserved       | 0xFF                  |
00148 * | 6     | Reserved       | Factory value         |
00149 * | 7     | Reserved       | Factory value         |
00150 * | 8     | CRC            | CRC-8 of bytes 0-7    |
00151 *
00152 * Configuration register format (byte 4):
00153 * @code
00154 * Bit:  7  6  5  4  3  2  1  0
00155 *       0 R1 R0  1  1  1  1  1
00156 *       +---+ Resolution: 00=9b, 01=10b, 10=11b, 11=12b
00157 * @endcode
00158 *
00159 * # Performance Characteristics
00160 *
00161 * ## Timing Analysis
00162 *
00163 * | Operation           | Duration   | Notes           |
00164 * |-----|-----|-----|
00165 * | Init                 | ~10ms      | Presence detect + read |
00166 * | Trigger conversion   | ~1ms       | Command transmission |
00167 * | Conversion (9-bit)   | 93.75ms (spec) | 100ms recommended |
00168 * | Conversion (10-bit)  | 187.5ms (spec) | 200ms recommended |
00169 * | Conversion (11-bit)  | 375ms (spec)  | 400ms recommended |
00170 * | Conversion (12-bit)  | 750ms (spec)  | 800ms recommended |
00171 * | Read scratchpad      | ~2ms       | 9 bytes + CRC validation |
00172 * | Set resolution       | ~5ms       | Read + modify + write |
00173 *
00174 * **Note**: Recommended times include 10% safety margin above datasheet specs.
00175 *
00176 * ## Memory Usage
00177 *
00178 * | Component           | Size (bytes) | Location       |
00179 * |-----|-----|-----|
00180 * | 'rx_ds18b20_handle_t' | 28           | Caller stack |
00181 * | 'rx_ds18b20_config_t' | 24           | Caller stack |
00182 * | Scratchpad buffer     | 9            | Stack (temp) |
00183 * | Driver code (.text)   | ~2KB         | Flash         |
00184 * | Constant data (.rodata) | ~100 bytes   | Flash         |
00185 *
00186 * **Total RAM per sensor**: 28 bytes (handle only, no dynamic allocation)
00187 *
00188 * # Hardware Requirements
00189 *
00190 * | Resource           | Requirement           |
00191 * |-----|-----|
00192 * | **CPU**           | Any (architecture-independent) |
00193 * | **RAM**           | 28 bytes per sensor + 9-byte temp buffer |
00194 * | **Flash**         | ~2KB code + ~100 bytes constants |
00195 * | **GPIO**          | 1 pin (open-drain or push-pull with resistor) |
00196 * | **Timing**        | Microsecond-precision delays (1-Wire protocol) |
00197 * | **Pull-up**       | 4.7kOhm resistor to VCC (external) |
00198 * | **Power**         | 3.0V-5.5V (parasitic mode: data pin can power) |
00199 *
00200 * ## Typical Wiring (Single Sensor)
00201 *

```

```

00202 * @code{.unparsed}
00203 *   VCC (3.3V or 5V)
00204 *   |
00205 *   +-----+----- 4.7kOhm pull-up resistor
00206 *   |         |
00207 *   +---+---+---+---+
00208 *   | DS18B20 |
00209 *   |         |
00210 *   | DQ  VDD |
00211 *   +---+---+---+---+
00212 *   |         |
00213 *   | +----- VCC (or GND for parasitic mode)
00214 *   |
00215 *   +----- MCU GPIO (data line)
00216 *
00217 *   GND ----- GND
00218 * @endcode
00219 *
00220 * # Example Usage Patterns
00221 *
00222 * ## Example 1: Basic Single-Sensor Configuration
00223 *
00224 * @code
00225 * #include "rx_ds18b20.h"
00226 * #include "rx_bus_manager.h"
00227 * #include "tx_api.h"
00228 *
00229 * void temperature_task_entry(ULONG input) {
00230 *     rx_ds18b20_handle_t sensor;
00231 *     rx_ds18b20_config_t config = {
00232 *         .bus_manager = &g_bus_manager,
00233 *         .bus_name = "onewire0",
00234 *         .resolution = k_ds18b20_resolution bit, // 0.0625degC precision
00235 *         .use_rom_matching = false,              // Skip ROM (single sensor)
00236 *     };
00237 *
00238 *     rx_err_t err = rx_ds18b20_init(&sensor, &config);
00239 *     if (err != k_rx_ok) {
00240 *         // Handle error: check bus wiring, power, pull-up resistor
00241 *         return;
00242 *     }
00243 *
00244 *     while (1) {
00245 *         // Trigger conversion
00246 *         err = rx_ds18b20_trigger_conversion(&sensor);
00247 *         if (err != k_rx_ok) continue;
00248 *
00249 *         // Wait for conversion (800ms for 12-bit)
00250 *         tx_thread_sleep(80); // 800ms at 100 Hz tick rate
00251 *
00252 *         // Read temperature
00253 *         float temp_celsius;
00254 *         err = rx_ds18b20_read_temperature(&sensor, &temp_celsius);
00255 *         if (err == k_rx_ok) {
00256 *             // Process temperature (e.g., log, control, transmit)
00257 *             printf("Temperature: %.2fdegC\n", temp_celsius);
00258 *         } else if (err == k_rx_err_crc_mismatch) {
00259 *             // CRC error: retry or log fault
00260 *         }
00261 *
00262 *         // Repeat every 5 seconds
00263 *         tx_thread_sleep(500);
00264 *     }
00265 * }
00266 * @endcode
00267 *
00268 * ## Example 2: Multi-Sensor Configuration (ROM Matching)
00269 *
00270 * @code
00271 * // Pre-discovered ROM codes (use ROM search to find these)
00272 * uint8_t rom_sensor1[8] = {0x28, 0xFF, 0x12, 0x34, 0x56, 0x78, 0x90, 0xAB};
00273 * uint8_t rom_sensor2[8] = {0x28, 0xFF, 0xAB, 0xCD, 0xEF, 0x01, 0x23, 0x45};
00274 *
00275 * rx_ds18b20_handle_t sensors[2];
00276 * rx_ds18b20_config_t config_template = {
00277 *     .bus_manager = &g_bus_manager,
00278 *     .bus_name = "onewire0",
00279 *     .resolution = k_ds18b20_resolution bit,
00280 *     .use_rom_matching = true, // Enable ROM matching for multi-drop
00281 * };
00282 *
00283 * // Init sensor 1
00284 * memcpy(config_template.rom, rom_sensor1, 8);
00285 * rx_ds18b20_init(&sensors[0], &config_template);
00286 *
00287 * // Init sensor 2
00288 * memcpy(config_template.rom, rom_sensor2, 8);

```

```

00289 * rx_ds18b20_init(&sensors[1], &config_template);
00290 *
00291 * // Trigger both simultaneously (broadcast not shown, or trigger individually)
00292 * rx_ds18b20_trigger_conversion(&sensors[0]);
00293 * rx_ds18b20_trigger_conversion(&sensors[1]);
00294 *
00295 * tx_thread_sleep(40); // Wait 400ms for 11-bit
00296 *
00297 * // Read both
00298 * float temps[2];
00299 * rx_ds18b20_read_temperature(&sensors[0], &temps[0]);
00300 * rx_ds18b20_read_temperature(&sensors[1], &temps[1]);
00301 * @endcode
00302 *
00303 * ## Example 3: Error Handling and Diagnostics
00304 *
00305 * @code
00306 * rx_err_t err = rx_ds18b20_read_temperature(&sensor, &temp_c);
00307 *
00308 * switch (err) {
00309 *     case k_rx_ok:
00310 *         // Success: use temp_c
00311 *         break;
00312 *
00313 *     case k_rx_err_crc_mismatch:
00314 *         // CRC error: possible bus noise, retry
00315 *         retry_count++;
00316 *         break;
00317 *
00318 *     case k_rx_err_invalid_state:
00319 *         // Sensor disconnected or not responding
00320 *         // Check: wiring, power, pull-up resistor
00321 *         break;
00322 *
00323 *     case k_rx_err_out_of_range:
00324 *         // Temperature outside -55degC to +125degC
00325 *         // Sensor fault or extreme environment
00326 *         break;
00327 *
00328 *     default:
00329 *         // Unexpected error
00330 *         break;
00331 * }
00332 * @endcode
00333 *
00334 * ## Example 4: Runtime Resolution Change
00335 *
00336 * @code
00337 * // Switch to 9-bit for faster updates (0.5degC precision)
00338 * rx_ds18b20_set_resolution(&sensor, k_ds18b20_resolution_bit);
00339 *
00340 * // Persist to EEPROM (survives power cycle)
00341 * rx_ds18b20_save_config(&sensor);
00342 *
00343 * // Now trigger with shorter wait time
00344 * rx_ds18b20_trigger_conversion(&sensor);
00345 * tx_thread_sleep(10); // 100ms for 9-bit
00346 * rx_ds18b20_read_temperature(&sensor, &temp_c);
00347 * @endcode
00348 *
00349 * # Thread Safety
00350 *
00351 * **Not thread-safe**. Caller must serialize access if multiple threads use the
00352 * same sensor handle. Use ThreadX mutexes:
00353 *
00354 * @code
00355 * TX_MUTEX sensor_mutex;
00356 * tx_mutex_create(&sensor_mutex, "DS18B20", TX_NO_INHERIT);
00357 *
00358 * // Thread 1:
00359 * tx_mutex_get(&sensor_mutex, TX_WAIT_FOREVER);
00360 * rx_ds18b20_trigger_conversion(&sensor);
00361 * tx_mutex_put(&sensor_mutex);
00362 *
00363 * // Thread 2:
00364 * tx_mutex_get(&sensor_mutex, TX_WAIT_FOREVER);
00365 * rx_ds18b20_read_temperature(&sensor, &temp_c);
00366 * tx_mutex_put(&sensor_mutex);
00367 * @endcode
00368 *
00369 * **Note**: Different sensor handles (different physical sensors) CAN be accessed
00370 * concurrently without synchronization, even on the same 1-Wire bus (ROM matching
00371 * addresses each sensor independently).
00372 *
00373 * # Known Limitations
00374 *
00375 * 1. **No alarm threshold API**: TH/TL registers accessible via scratchpad but

```

```

00376 *   not exposed. Users can read/write scratchpad directly if needed.
00377
00378 * 2. **No broadcast/simultaneous conversion**: Must trigger each sensor individually
00379 *   when using ROM matching. Skip ROM mode allows broadcast but only for single-sensor.
00380
00381 * 3. **No ROM search**: Must provide ROM codes manually for multi-sensor. Consider
00382 *   adding `rx_ds18b20_discover()` helper in future.
00383
00384 * 4. **Blocking API only**: No asynchronous/callback interface. Caller must use
00385 *   RTOS primitives (tasks, timers) for non-blocking behavior.
00386
00387 * # References and Standards
00388
00389 * - **DS18B20 Datasheet**: Maxim Integrated (now Analog Devices)
00390 *   https://www.analog.com/media/en/technical-documentation/data-sheets/DS18B20.pdf
00391 * - **1-Wire Protocol**: Book of iButton Standards, Maxim Integrated
00392 * - **CRC-8 Polynomial**:  $x^8 + x^5 + x^4 + 1$  (standard for 1-Wire devices)
00393 * - **ROM Structure**: 8-byte format (1-byte family + 6-byte serial + 1-byte CRC)
00394 * - **IEEE 802.3 Ethernet CRC**: Different from DS18B20's CRC-8, not applicable here
00395
00396 * @see rx_bus_manager.h - Bus manager abstraction layer
00397 * @see rx_bus_1wire.h - 1-Wire protocol implementation
00398 * @see rx_crc.h - CRC calculation utilities (CRC-8 for DS18B20)
00399 * @see docs/sections/03_hardware_pinout.tex - GPIO pin assignments
00400
00401 * @par NASA Power of 10 Compliance:
00402 * - **Rule 1** (No goto): [OK] All control flow uses structured statements
00403 * - **Rule 2** (Bounded loops): [OK] All loops have compile-time or runtime bounds
00404 * - **Rule 3** (No heap): [OK] Zero dynamic allocation (static handles only)
00405 * - **Rule 4** (Functions  $\leq 60$  lines): [OK] All public functions  $< 60$  lines
00406 * - **Rule 5** (Assertions): [OK] 2+ preconditions per function (NULL checks, state)
00407 * - **Rule 6** (Data scope): [OK] Variables declared at smallest scope
00408 * - **Rule 7** (Return checking): [OK] All HAL returns validated
00409 * - **Rule 8** (Preprocessor limits): [OK] C23 typed enums only, minimal macros
00410 * - **Rule 9** (Pointer restrictions): [WARN] Intentional deviation for DIP (bus_manager)
00411 * - **Rule 10** (Compiler warnings): [OK] -Wall -Wextra -Werror, zero warnings
00412
00413 * @par SOLID Principles:
00414 * - **S (Single Responsibility)**: Module handles ONLY DS18B20 sensor operations
00415 * - **O (Open/Closed)**: Extensible via bus_manager (add new buses w/o changes)
00416 * - **L (Liskov Substitution)**: Any rx_bus_manager_t* is interchangeable
00417 * - **I (Interface Segregation)**: Focused API (temp read, no unrelated operations)
00418 * - **D (Dependency Inversion)**: Depends on rx_bus_manager_t abstraction, not GPIO
00419
00420 * @par Module Dependencies:
00421 * @code{,unparsed}
00422 * rx_ds18b20
00423 * +-- rx_bus_manager (bus abstraction, injected)
00424 * +-- rx_bus_1wire (1-Wire protocol layer)
00425 * +-- rx_crc (CRC-8 validation)
00426 * +-- rx_err (error codes)
00427 * +-- rx_log (diagnostic logging)
00428 * +-- rx_check (nullptr validation macros)
00429 * @endcode
00430
00431 * @author STAR Team
00432 * @date 2026-01-30
00433 * @version 1.0.0
00434 * @copyright MIT License
00435 */
00436
00437 #pragma once
00438
00439 #ifndef __cplusplus
00440 extern "C" {
00441 #endif
00442
00443 #include <stdbool.h>
00444 #include <stdint.h>
00445
00446 #include "rx_bus_manager.h"
00447 #include "rx_bus_1wire.h"
00448 #include "rx_err.h"
00449
00450 /*
=====
00451 * DS18B20 Constants
00452 *
=====
00453 */
00454
00455 /**
00456 * @enum ds18b20_command_t
00457 * @brief DS18B20 function commands per datasheet specification
00458 *
00459 * @details
00460 * Command codes sent after ROM selection (Match ROM or Skip ROM) to initiate

```

```

00461 * DS18B20 operations. Each command triggers a specific sensor function.
00462 *
00463 **Protocol Sequence:**
00464 * 1. Reset pulse + presence detection
00465 * 2. ROM command (Match ROM 0x55 or Skip ROM 0xCC)
00466 * 3. Function command (one of these values)
00467 * 4. Additional data bytes (if required by command)
00468 *
00469 **Command Details:**
00470 *
00471 * | Command | Code | Data Bytes | Duration | Description |
00472 * |-----|-----|-----|-----|-----|
00473 * | Convert T | 0x44 | 0 | 94-750ms | Start temperature conversion |
00474 * | Write Scratchpad | 0x4E | 3 | ~2ms | Write TH, TL, Config registers |
00475 * | Read Scratchpad | 0xBE | 0 | ~2ms | Read 9-byte scratchpad + CRC |
00476 * | Copy Scratchpad | 0x48 | 0 | ~10ms | Save scratchpad to EEPROM |
00477 * | Recall E2 | 0xB8 | 0 | ~1ms | Restore EEPROM to scratchpad |
00478 * | Read Power Supply | 0xB4 | 0 | <1ms | Check parasitic vs. external |
00479 *
00480 * @par Usage Example:
00481 * @code
00482 * // Trigger conversion
00483 * rx_bus_onewire_reset(bus_mgr, "onewire0", &presence);
00484 * rx_bus_onewire_skip_rom(bus_mgr, "onewire0");
00485 * rx_bus_onewire_write_byte(bus_mgr, "onewire0", k_ds18b20_cmd_convert_t);
00486 * // Wait conversion time...
00487 * @endcode
00488 *
00489 * @see DS18B20 datasheet section "Function Commands" for timing specifications
00490 * @see rx_bus_onewire.h for ROM command functions
00491 *
00492 * @since Version 1.0.0
00493 */
00494 typedef enum : uint8_t {
00495     k_ds18b20_cmd_convert_t = 0x44, /**< Trigger temperature conversion (ADC operation) */
00496     k_ds18b20_cmd_write_scratch = 0x4E, /**< Write to scratchpad memory (TH, TL, Config bytes) */
00497     k_ds18b20_cmd_read_scratch = 0xBE, /**< Read from scratchpad memory (9 bytes: data + CRC) */
00498     k_ds18b20_cmd_copy_scratch = 0x48, /**< Copy scratchpad to EEPROM (non-volatile storage) */
00499     k_ds18b20_cmd_recall_eeprom = 0xB8, /**< Recall EEPROM to scratchpad (restore saved values) */
00500     k_ds18b20_cmd_read_power = 0xB4, /**< Read power supply mode (0=parasitic, 1=external) */
00501 } ds18b20_command_t;
00502
00503 /**
00504 * @enum ds18b20_scratchpad_index_t
00505 * @brief DS18B20 scratchpad memory layout indices
00506 *
00507 * @details
00508 * The DS18B20's 9-byte scratchpad RAM contains temperature data, alarm thresholds,
00509 * configuration, and a CRC-8 checksum. Use these indices to access specific bytes
00510 * when reading via Read Scratchpad command (0xBE).
00511 *
00512 **Memory Layout:**
00513 *
00514 * | Index | Byte | Name | Description | Access |
00515 * |-----|-----|-----|-----|-----|
00516 * | 0 | 0 | Temp LSB | Temperature low byte (bits 0-7) | Read-only |
00517 * | 1 | 1 | Temp MSB | Temperature high byte (sign + bits 8-11) | Read-only |
00518 * | 2 | 2 | TH Register | High alarm threshold (-55 to +125degC) | R/W |
00519 * | 3 | 3 | TL Register | Low alarm threshold (-55 to +125degC) | R/W |
00520 * | 4 | 4 | Config | Resolution bits [6:5]: 00=9b, 01=10b, 10=11b, 11=12b | R/W |
00521 * | 5 | 5 | Reserved | Always 0xFF (ignore) | Read-only |
00522 * | 6 | 6 | Reserved | Factory-programmed (ignore) | Read-only |
00523 * | 7 | 7 | Reserved | Factory-programmed (ignore) | Read-only |
00524 * | 8 | 8 | CRC | CRC-8 checksum of bytes 0-7 | Read-only |
00525 *
00526 **Temperature Format (Bytes 0-1):**
00527 *
00528 * 16-bit signed integer in 1/16degC units (two's complement):
00529 * @code
00530 * MSB (byte 1): S S S S S T10 T9 T8
00531 * LSB (byte 0): T7 T6 T5 T4 T3 T2 T1 T0
00532 * +-----+-----+ Temperature bits (12-bit for 12-bit resolution)
00533 * @endcode
00534 *
00535 * - **S**: Sign bits (all 0 for positive, all 1 for negative)
00536 * - **T10-T0**: Temperature bits (11 bits used, T0-T2 undefined for lower resolutions)
00537 * - **Value range**: -880 to +2000 (represents -55.0degC to +125.0degC)
00538 * - **Conversion**: `temp_celsius = raw_value / 16.0`
00539 *
00540 **Configuration Register (Byte 4):**
00541 *
00542 * Format: `0 R1 R0 1 1 1 1` where R1:R0 = resolution bits
00543 *
00544 * | R1 | R0 | Resolution | Precision | Conversion Time |
00545 * |----|----|-----|-----|-----|
00546 * | 0 | 0 | 9-bit | 0.5degC | 93.75ms |
00547 * | 0 | 1 | 10-bit | 0.25degC | 187.5ms |

```



```

00548 * | 1 | 0 | 11-bit | 0.125degC | 375ms |
00549 * | 1 | 1 | 12-bit | 0.0625degC | 750ms |
00550 *
00551 * **CRC-8 Calculation (Byte 8):**
00552 *
00553 * - **Polynomial**:  $x^8 + x^5 + x^4 + 1$  (0x8C)
00554 * - **Initial value**: 0x00
00555 * - **Data**: Bytes 0-7 (inclusive)
00556 * - **Purpose**: Detect transmission errors on 1-Wire bus
00557 *
00558 * @par Usage Example:
00559 * @code
00560 * uint8_t scratchpad[k_ds18b20_scratchpad_bytes];
00561 * rx_ds18b20_read_scratchpad(&sensor, scratchpad);
00562 *
00563 * // Extract temperature
00564 * uint16_t temp_raw = scratchpad[k_ds18b20_scratch_temp_lsb] |
00565 * (scratchpad[k_ds18b20_scratch_temp_msb] « 8);
00566 *
00567 * // Validate CRC
00568 * uint8_t crc_calc = rx_crc8_maxim(scratchpad, k_ds18b20_crc_bytes);
00569 * if (crc_calc != scratchpad[k_ds18b20_scratch_crc]) {
00570 * // CRC mismatch: retry read
00571 * }
00572 *
00573 * // Read alarm thresholds
00574 * int8_t th = (int8_t)scratchpad[k_ds18b20_scratch_th_reg];
00575 * int8_t tl = (int8_t)scratchpad[k_ds18b20_scratch_tl_reg];
00576 * @endcode
00577 *
00578 * @invariant Byte 5 (reserved1) is always 0xFF
00579 * @invariant CRC (byte 8) is valid for bytes 0-7
00580 * @invariant Temperature bytes (0-1) represent value in range [-880, +2000]
00581 *
00582 * @see rx_ds18b20_read_scratchpad() Read entire scratchpad with CRC validation
00583 * @see rx_crc8_maxim() CRC-8 calculation function
00584 * @see DS18B20 datasheet section "Memory" for detailed scratchpad specification
00585 *
00586 * @since Version 1.0.0
00587 */
00588 typedef enum : uint8_t {
00589 k_ds18b20_scratch_temp_lsb = 0, /**< Temperature LSB (bits 0-7 of 16-bit value) */
00590 k_ds18b20_scratch_temp_msb = 1, /**< Temperature MSB (sign bits + bits 8-11) */
00591 k_ds18b20_scratch_th_reg = 2, /**< TH (high alarm) register (signed byte, -55 to +125) */
00592 k_ds18b20_scratch_tl_reg = 3, /**< TL (low alarm) register (signed byte, -55 to +125) */
00593 k_ds18b20_scratch_config = 4, /**< Configuration register (resolution bits at [6:5]) */
00594 k_ds18b20_scratch_reserved1 = 5, /**< Reserved byte (always 0xFF, do not write) */
00595 k_ds18b20_scratch_reserved2 = 6, /**< Reserved byte (factory-programmed, do not write) */
00596 k_ds18b20_scratch_reserved3 = 7, /**< Reserved byte (factory-programmed, do not write) */
00597 k_ds18b20_scratch_crc = 8, /**< CRC-8 of bytes 0-7 (polynomial 0x8C) */
00598 k_ds18b20_scratchpad_bytes = 9, /**< Total scratchpad size in bytes */
00599 } ds18b20_scratchpad_index_t;
00600
00601 /**
00602 * @brief DS18B20 configuration register bit positions
00603 */
00604 typedef enum : uint8_t {
00605 k_ds18b20_config_r0_bit = 5, /**< Resolution bit 0 */
00606 k_ds18b20_config_r1_bit = 6, /**< Resolution bit 1 */
00607 } ds18b20_config_bits_t;
00608
00609 /**
00610 * @brief DS18B20 temperature resolution modes
00611 */
00612 typedef enum : uint8_t {
00613 k_ds18b20_resolution_bit = 0, /**< 9-bit: 0.5degC, 93.75ms */
00614 k_ds18b20_resolution_bit = 1, /**< 10-bit: 0.25degC, 187.5ms */
00615 k_ds18b20_resolution_bit = 2, /**< 11-bit: 0.125degC, 375ms */
00616 k_ds18b20_resolution_bit = 3, /**< 12-bit: 0.0625degC, 750ms */
00617 } ds18b20_resolution_t;
00618
00619 /**
00620 * @brief DS18B20 conversion times (milliseconds)
00621 *
00622 * Maximum conversion time for each resolution.
00623 * Add margin for safety (spec values + 10%).
00624 */
00625 typedef enum : uint16_t {
00626 k_ds18b20_conv_time_bit_ms = 100, /**< 9-bit conversion: 100ms */
00627 k_ds18b20_conv_time_bit_ms = 200, /**< 10-bit conversion: 200ms */
00628 k_ds18b20_conv_time_bit_ms = 400, /**< 11-bit conversion: 400ms */
00629 k_ds18b20_conv_time_bit_ms = 800, /**< 12-bit conversion: 800ms */
00630 } ds18b20_conversion_time_ms_t;
00631
00632 /**
00633 * @brief DS18B20 temperature masks for resolution
00634 */

```



```

00635 * Lower bits are undefined for resolutions < 12-bit and must be masked.
00636 * This prevents garbage bits from corrupting the temperature reading.
00637 */
00638 typedef enum : uint16_t {
00639     k_ds18b20_temp_mask_bit = 0xFFFF8, /**< Mask for 9-bit (discard bits 0-2) */
00640     k_ds18b20_temp_mask_bit = 0xFFFC, /**< Mask for 10-bit (discard bits 0-1) */
00641     k_ds18b20_temp_mask_bit = 0xFFFE, /**< Mask for 11-bit (discard bit 0) */
00642     k_ds18b20_temp_mask_bit = 0xFFFF, /**< Mask for 12-bit (all bits valid) */
00643 } ds18b20_temp_mask_t;
00644
00645 /**
00646  * @brief DS18B20 temperature conversion constants
00647  */
00648 typedef enum : int32_t {
00649     k_ds18b20_temp_shift = 4, /**< Shift to get integer temperature */
00650     k_ds18b20_sign_bit = 0x8000, /**< Sign bit in 16-bit temperature */
00651     k_ds18b20_crc_bytes = 8, /**< Number of bytes for CRC calculation */
00652     k_ds18b20_shift_byte = 8, /**< Shift for byte positioning */
00653     k_ds18b20_scratchpad_write_bytes = 3, /**< Scratchpad write size (TH, TL, Config) */
00654     k_ds18b20_temp_min_raw = -880, /**< Minimum temperature (-55degC raw value) */
00655     k_ds18b20_temp_max_raw = 2000, /**< Maximum temperature (+125degC raw value) */
00656 } ds18b20_conversion_constants_t;
00657
00658 /**
00659  * @brief DS18B20 scratchpad write buffer indices
00660  */
00661 typedef enum : uint8_t {
00662     k_ds18b20_write_idx_th = 0, /**< TH (high alarm) register index */
00663     k_ds18b20_write_idx_tl = 1, /**< TL (low alarm) register index */
00664     k_ds18b20_write_idx_config = 2, /**< Configuration register index */
00665 } ds18b20_write_idx_t;
00666
00667 /**
00668  * @brief DS18B20 initialization values
00669  */
00670 typedef enum : uint8_t {
00671     k_ds18b20_config_register_cleared = 0, /**< Cleared config register value */
00672 } ds18b20_init_values_t;
00673
00674 /**
00675  * @brief DS18B20 power supply modes
00676  */
00677 typedef enum : uint8_t {
00678     k_ds18b20_power_parasitic = 0, /**< Parasitic power mode */
00679     k_ds18b20_power_external = 1, /**< External power mode */
00680 } ds18b20_power_mode_t;
00681
00682 /*
=====
00683  * Type Definitions
00684  *
=====
00685  */
00686
00687 /**
00688  * @brief DS18B20 configuration structure
00689  */
00690 typedef struct {
00691     rx_bus_manager_t* bus_manager; /**< Bus manager instance (required) */
00692     const char* bus_name; /**< OneWire bus name (required) */
00693     ds18b20_resolution_t resolution; /**< Temperature resolution */
00694     bool use_rom_matching; /**< Use ROM matching (true) or skip ROM (false) */
00695     uint8_t rom[k_onewire_rom_bytes]; /**< Device ROM (if use_rom_matching=true) */
00696 } rx_ds18b20_config_t;
00697
00698 /**
00699  * @brief DS18B20 driver handle structure
00700  */
00701 typedef struct {
00702     rx_bus_manager_t* bus_manager; /**< Bus manager reference (not owned) */
00703     const char* bus_name; /**< OneWire bus name (not owned) */
00704     ds18b20_resolution_t resolution; /**< Current temperature resolution */
00705     bool use_rom_matching; /**< ROM matching enabled */
00706     uint8_t rom[k_onewire_rom_bytes]; /**< Device ROM code */
00707     bool initialized; /**< True after successful init */
00708 } rx_ds18b20_handle_t;
00709
00710 /*
=====
00711  * Public API
00712  *
=====
00713  */
00714
00715 /**
00716  * @brief Initialize DS18B20 sensor
00717  */

```

```

00718 * Initializes the DS18B20 sensor and configures resolution.
00719 * If ROM matching is disabled, uses Skip ROM (single device mode).
00720 *
00721 * @param[out] handle Pointer to DS18B20 handle. Must not be nullptr.
00722 * @param[in] config Pointer to configuration. Must not be nullptr.
00723 *
00724 * @return k_rx_ok on success
00725 * @return k_rx_err_null_ptr if handle or config is nullptr
00726 * @return k_rx_err_invalid_arg if bus_manager or bus_name is nullptr
00727 * @return k_rx_err_invalid_state if device not responding
00728 * @return k_rx_err_crc if scratchpad CRC check fails
00729 */
00730 [[nodiscard]] rx_err_t rx_ds18b20_init(rx_ds18b20_handle_t* handle,
00731                                         const rx_ds18b20_config_t* config);
00732
00733 /**
00734 * @brief Deinitialize DS18B20 sensor
00735 *
00736 * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00737 *
00738 * @return k_rx_ok on success
00739 * @return k_rx_err_null_ptr if handle is nullptr
00740 * @return k_rx_err_invalid_state if not initialized
00741 */
00742 [[nodiscard]] rx_err_t rx_ds18b20_deinit(rx_ds18b20_handle_t* handle);
00743
00744 /**
00745 * @brief Trigger temperature conversion
00746 *
00747 * Sends Convert T command to start temperature measurement.
00748 * Caller must wait for conversion time before reading temperature.
00749 *
00750 * Conversion times:
00751 * - 9-bit: 100ms
00752 * - 10-bit: 200ms
00753 * - 11-bit: 400ms
00754 * - 12-bit: 800ms
00755 *
00756 * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00757 *
00758 * @return k_rx_ok on success
00759 * @return k_rx_err_null_ptr if handle is nullptr
00760 * @return k_rx_err_invalid_state if not initialized or device not present
00761 */
00762 [[nodiscard]] rx_err_t rx_ds18b20_trigger_conversion(const rx_ds18b20_handle_t* handle);
00763
00764 /**
00765 * @brief Read temperature in Celsius
00766 *
00767 * Reads the scratchpad and converts raw temperature to Celsius.
00768 * Temperature conversion must complete before calling (see conversion times).
00769 *
00770 * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00771 * @param[out] temperature_celsius Pointer to store temperature in degC. Must not be nullptr.
00772 *
00773 * @return k_rx_ok on success
00774 * @return k_rx_err_null_ptr if any pointer is nullptr
00775 * @return k_rx_err_invalid_state if not initialized or device not present
00776 * @return k_rx_err_crc if scratchpad CRC check fails
00777 */
00778 [[nodiscard]] rx_err_t rx_ds18b20_read_temperature(rx_ds18b20_handle_t* handle,
00779                                                    float* temperature_celsius);
00780
00781 /**
00782 * @brief Read raw temperature value
00783 *
00784 * Reads the raw 16-bit temperature from scratchpad without conversion.
00785 * Useful for debugging or custom temperature processing.
00786 *
00787 * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00788 * @param[out] raw_temp Pointer to store raw temperature. Must not be nullptr.
00789 *
00790 * @return k_rx_ok on success
00791 * @return k_rx_err_null_ptr if any pointer is nullptr
00792 * @return k_rx_err_invalid_state if not initialized or device not present
00793 * @return k_rx_err_crc if scratchpad CRC check fails
00794 */
00795 [[nodiscard]] rx_err_t rx_ds18b20_read_temperature_raw(rx_ds18b20_handle_t* handle,
00796                                                         int16_t* raw_temp);
00797
00798 /**
00799 * @brief Set temperature resolution
00800 *
00801 * Changes the sensor resolution (9, 10, 11, or 12 bits).
00802 * Higher resolution provides more precision but takes longer to convert.
00803 *
00804 * @param[in] handle Pointer to initialized handle. Must not be nullptr.

```

```

00805 * @param[in] resolution Resolution mode (9-12 bits)
00806 *
00807 * @return k_rx_ok on success
00808 * @return k_rx_err_null_ptr if handle is nullptr
00809 * @return k_rx_err_invalid_arg if resolution out of range
00810 * @return k_rx_err_invalid_state if not initialized or device not present
00811 */
00812 [[nodiscard]] rx_err_t rx_ds18b20_set_resolution(rx_ds18b20_handle_t* handle,
00813                                                  ds18b20_resolution_t resolution);
00814
00815 /**
00816 * @brief Get current temperature resolution
00817 *
00818 * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00819 * @param[out] resolution Pointer to store resolution. Must not be nullptr.
00820 *
00821 * @return k_rx_ok on success
00822 * @return k_rx_err_null_ptr if any pointer is nullptr
00823 * @return k_rx_err_invalid_state if not initialized
00824 */
00825 [[nodiscard]] rx_err_t rx_ds18b20_get_resolution(const rx_ds18b20_handle_t* handle,
00826                                                  ds18b20_resolution_t* resolution);
00827
00828 /**
00829 * @brief Save configuration to EEPROM
00830 *
00831 * Copies scratchpad bytes 2-4 (TH, TL, Config) to non-volatile EEPROM.
00832 * Values persist across power cycles.
00833 *
00834 * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00835 *
00836 * @return k_rx_ok on success
00837 * @return k_rx_err_null_ptr if handle is nullptr
00838 * @return k_rx_err_invalid_state if not initialized or device not present
00839 */
00840 [[nodiscard]] rx_err_t rx_ds18b20_save_config(const rx_ds18b20_handle_t* handle);
00841
00842 /**
00843 * @brief Recall configuration from EEPROM
00844 *
00845 * Restores scratchpad bytes 2-4 (TH, TL, Config) from EEPROM.
00846 * Useful after power-up to restore saved settings.
00847 *
00848 * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00849 *
00850 * @return k_rx_ok on success
00851 * @return k_rx_err_null_ptr if handle is nullptr
00852 * @return k_rx_err_invalid_state if not initialized or device not present
00853 */
00854 [[nodiscard]] rx_err_t rx_ds18b20_recall_config(const rx_ds18b20_handle_t* handle);
00855
00856 /**
00857 * @brief Read power supply mode
00858 *
00859 * Checks if sensor is using parasitic power or external power.
00860 *
00861 * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00862 * @param[out] external_power True if external power, false if parasitic. Must not be nullptr.
00863 *
00864 * @return k_rx_ok on success
00865 * @return k_rx_err_null_ptr if any pointer is nullptr
00866 * @return k_rx_err_invalid_state if not initialized or device not present
00867 */
00868 [[nodiscard]] rx_err_t rx_ds18b20_read_power_mode(const rx_ds18b20_handle_t* handle,
00869                                                  bool* external_power);
00870
00871 /**
00872 * @brief Read scratchpad memory
00873 *
00874 * Reads the entire 9-byte scratchpad (8 data bytes + 1 CRC byte).
00875 * Performs CRC validation.
00876 *
00877 * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00878 * @param[out] scratchpad Pointer to 9-byte buffer. Must not be nullptr.
00879 *
00880 * @return k_rx_ok on success
00881 * @return k_rx_err_null_ptr if any pointer is nullptr
00882 * @return k_rx_err_invalid_state if not initialized or device not present
00883 * @return k_rx_err_crc if CRC check fails
00884 */
00885 [[nodiscard]] rx_err_t rx_ds18b20_read_scratchpad(rx_ds18b20_handle_t* handle,
00886                                                  uint8_t scratchpad[k_ds18b20_scratchpad_bytes]);
00887
00888 /**
00889 * @brief Get conversion time for current resolution
00890 *
00891 * Returns the maximum conversion time in milliseconds for the configured resolution.

```

```

00892 * Caller should wait this long after triggering conversion before reading temperature.
00893 *
00894 * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00895 *
00896 * @return Conversion time in milliseconds, or 0 if handle is nullptr or not initialized
00897 */
00898 uint32_t rx_ds18b20_get_conversion_time_ms(const rx_ds18b20_handle_t* handle);
00899
00900 #ifdef __cplusplus
00901 }
00902 #endif

```

6.3 libs/rx_ds18b20/src/rx_ds18b20.c File Reference

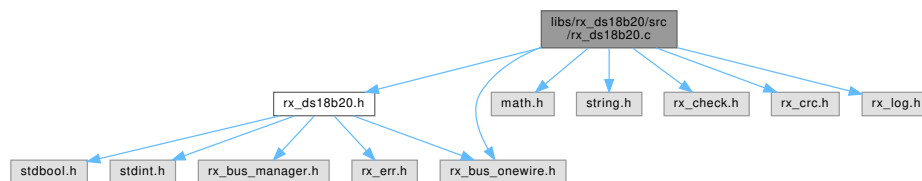
DS18B20 1-Wire Digital Temperature Sensor Driver Implementation.

```

#include "rx_ds18b20.h"
#include <math.h>
#include <string.h>
#include "rx_bus_onewire.h"
#include "rx_check.h"
#include "rx_crc.h"
#include "rx_log.h"

```

Include dependency graph for rx_ds18b20.c:



Data Structures

- struct [ds18b20_scratchpad_write_t](#)
Scratchpad write parameters for TH, TL, and config registers.

Macros

- #define [CHECK_DS18B20_HANDLE](#)(handle, tag)
Validate DS18B20 handle pointer and initialization state.

Functions

- static rx_err_t [internal_ds18b20_validate_config](#) (const [rx_ds18b20_config_t](#)* config, const [rx_ds18b20_handle_t](#)* handle)
Validate DS18B20 configuration parameters before initialization.
- static rx_err_t [internal_ds18b20_verify_device_presence](#) ([rx_ds18b20_handle_t](#)* handle)
Verify DS18B20 device presence and establish communication.
- static rx_err_t [internal_ds18b20_select_device](#) (const [rx_ds18b20_handle_t](#)* handle)
Select DS18B20 device using ROM matching or Skip ROM command.
- static rx_err_t [internal_ds18b20_read_scratchpad_raw](#) (const [rx_ds18b20_handle_t](#)* handle, uint8_t scratchpad[k_ds18b20_scratchpad_bytes])

- Read DS18B20 scratchpad memory with CRC-8 validation.*
- static rx_err_t [internal_ds18b20_write_scratchpad](#) (const rx_ds18b20_handle_t *handle, const ds18b20_scratchpad_write_t *scratchpad)
- Write DS18B20 scratchpad registers (TH, TL, Config).*
- static uint8_t [internal_ds18b20_resolution_to_config](#) (const ds18b20_resolution_t resolution)
- Convert resolution enum to config register value.*
- static uint16_t [internal_ds18b20_get_temp_mask](#) (const ds18b20_resolution_t resolution)
- Get temperature mask for resolution.*
- static float [internal_ds18b20_raw_to_celsius](#) (const int16_t raw_temp)
- Convert raw temperature to Celsius.*
- rx_err_t [rx_ds18b20_init](#) (rx_ds18b20_handle_t *handle, const rx_ds18b20_config_t *config)
- Initialize DS18B20 sensor.*
- rx_err_t [rx_ds18b20_deinit](#) (rx_ds18b20_handle_t *handle)
- Deinitialize DS18B20 sensor handle and release all resources.*
- rx_err_t [rx_ds18b20_trigger_conversion](#) (const rx_ds18b20_handle_t *handle)
- Trigger temperature conversion on DS18B20 sensor.*
- rx_err_t [rx_ds18b20_read_temperature](#) (rx_ds18b20_handle_t *handle, float *temperature_celsius)
- Read temperature from DS18B20 sensor in degrees Celsius.*
- rx_err_t [rx_ds18b20_read_temperature_raw](#) (rx_ds18b20_handle_t *handle, int16_t *raw_temp)
- Read raw temperature value from DS18B20 sensor in 1/16 degree Celsius units.*
- rx_err_t [rx_ds18b20_set_resolution](#) (rx_ds18b20_handle_t *handle, const ds18b20_resolution_t resolution)
- Set ADC measurement resolution on DS18B20 sensor.*
- rx_err_t [rx_ds18b20_get_resolution](#) (const rx_ds18b20_handle_t *handle, ds18b20_resolution_t *resolution)
- Get the currently configured ADC resolution from DS18B20 handle.*
- rx_err_t [rx_ds18b20_save_config](#) (const rx_ds18b20_handle_t *handle)
- Save current scratchpad configuration to DS18B20 EEPROM.*
- rx_err_t [rx_ds18b20_recall_config](#) (const rx_ds18b20_handle_t *handle)
- Recall configuration from DS18B20 EEPROM into scratchpad RAM.*
- rx_err_t [rx_ds18b20_read_power_mode](#) (const rx_ds18b20_handle_t *handle, bool *external_power)
- Read DS18B20 power supply mode (external vs. parasitic).*
- rx_err_t [rx_ds18b20_read_scratchpad](#) (rx_ds18b20_handle_t *handle, uint8_t scratchpad[k_ds18b20_scratchpad_bytes])
- Read the full 9-byte scratchpad register from DS18B20 sensor.*
- uint32_t [rx_ds18b20_get_conversion_time_ms](#) (const rx_ds18b20_handle_t *handle)
- Get temperature conversion time for current resolution.*

Variables

- static const char [s_tag](#) [] = "DS18B20"
- Logging tag for DS18B20 driver.*
- static const uint8_t [s_ds18b20_family_code](#) = 0x28U
- DS18B20 family code (byte 0 of ROM).*
- static const float [s_temp_conversion_divisor](#) = 16.0f
- Temperature conversion divisor (raw to Celsius).*
- static const uint8_t [s_ds18b20_reserved_byte_value](#) = 0xFFU
- Reserved byte expected value in scratchpad.*
- static const uint32_t [s_ds18b20_conversion_time_invalid_u32](#) = 0U
- Invalid conversion time sentinel.*

6.3.1 Detailed Description

DS18B20 1-Wire Digital Temperature Sensor Driver Implementation.

Complete DS18B20 driver implementation with hardware abstraction via bus manager. Supports all DS18B20 features: temperature measurement, resolution configuration, ROM matching for multi-drop buses, and non-volatile EEPROM storage.

6.3.1.1 Architecture

Dependency Injection Pattern:

- Uses `rx_bus_manager_t` interface for 1-Wire communication
- Enables unit testing with mock bus implementations
- Zero direct hardware dependencies

Communication Flow:

`[rx_ds18b20] -> [rx_bus_manager] -> [rx_bus_onewire] -> [GPIO HAL]`

Thread Safety:

- NOT thread-safe - caller must provide synchronization
- Multiple handles can coexist (different sensors)
- Concurrent access to same handle causes race conditions

6.3.1.2 NASA Power of 10 Compliance

Rule	Status	Implementation
1. Simple control flow	[OK]	No goto, no recursion
2. Fixed loop bounds	[OK]	All loops bounded by enum constants
3. No dynamic allocation	[OK]	Zero malloc/free
4. Short functions	[OK]	Max 60 lines per function
5. Assertions	[OK]	Min 2 checks per function (nullptr, state)
6. Data scope	[OK]	Variables declared at smallest scope
7. Check returns	[OK]	All function returns validated
8. Limit preprocessor	[OK]	Typed enums only, no magic numbers
9. Pointer restrictions	[OK]	Single-level dereferencing
10. Compile warnings	[OK]	-Wall -Wextra -Werror

6.3.1.3 SOLID Principles

Single Responsibility:

- One purpose: DS18B20 sensor communication
- Bus operations delegated to `rx_bus_onewire`
- CRC validation delegated to `rx_crc`

Open/Closed:

- Extensible via bus manager interface
- New bus types added without modifying this driver

Liskov Substitution:

- Any `rx_bus_manager_t` implementation works
- Mock bus substitutes for testing

Interface Segregation:

- Clean API with focused functions
- Separate read/write/config operations

Dependency Inversion:

- Depends on `rx_bus_manager_t` abstraction
- Bus manager injected via config structure

6.3.1.4 Performance

Typical Operation Times:

- Initialization: ~100ms (bus init + scratchpad read)
- Trigger conversion: ~2ms (command send)
- Read temperature: ~5ms (scratchpad read + CRC check)
- Resolution change: ~10ms (read + modify + write scratchpad)

Conversion Times (sensor internal):

- 9-bit: 94ms
- 10-bit: 188ms
- 11-bit: 375ms
- 12-bit: 750ms (default)

6.3.1.5 Memory Layout

DS18B20 Scratchpad Structure (9 bytes):

Byte	Field	Description
0	Temp LSB	Temperature low byte (1/16degC units)
1	Temp MSB	Temperature high byte (sign-extended)
2	TH Register	High alarm threshold (user-defined)
3	TL Register	Low alarm threshold (user-defined)
4	Config	Resolution bits R1:R0 (bits 6:5)
5	Reserved	0xFF (always)
6	Reserved	Factory programmed (implementation-specific)
7	Reserved	0x10 (count remaining, legacy)
8	CRC	CRC-8/MAXIM checksum

Temperature Data Format:

```

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
| S | S | S | S | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 2-1 | 2-2 | 2-3 | 2-4 |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
Sign extend   Integer part   Fractional (1/16degC)

```

Resolution vs Temperature Bits:

- 9-bit: Bits 15:3 valid, bits 2:0 undefined (mask 0xFFF8)
- 10-bit: Bits 15:2 valid, bits 1:0 undefined (mask 0xFFFC)
- 11-bit: Bits 15:1 valid, bit 0 undefined (mask 0xFFFE)
- 12-bit: All bits valid (mask 0xFFFF)

6.3.1.6 Error Handling

Error Propagation Strategy:

```

// Check bus errors -> return immediately
err = rx_bus_onewire_reset(...);
if (err != k_rx_ok) return err;

// Check CRC -> return k_rx_err_crc_mismatch
if (crc_calc != crc_device) return k_rx_err_crc_mismatch;

// Check range -> return k_rx_err_out_of_range
if (temp < MIN || temp > MAX) return k_rx_err_out_of_range;

```

Common Error Codes:

- k_rx_err_null_ptr - nullptr handle/config/output pointer
- k_rx_err_invalid_state - Sensor not initialized or not present
- k_rx_err_invalid_arg - Invalid resolution or ROM family code
- k_rx_err_crc_mismatch - Scratchpad CRC validation failed
- k_rx_err_out_of_range - Temperature outside -55degC to +125degC

Example: Basic Temperature Reading

```

rx_ds18b20_handle_t sensor;
rx_ds18b20_config_t config = {
    .bus_manager = &g_bus_manager,
    .bus_name = "onewire0",
    .resolution = k_ds18b20_resolution_bit,
    .use_rom_matching = false // Single sensor
};

// 1. Initialize
rx_err_t err = rx_ds18b20_init(&sensor, &config);
if (err != k_rx_ok) handle_error(err);

// 2. Trigger conversion
err = rx_ds18b20_trigger_conversion(&sensor);
if (err != k_rx_ok) handle_error(err);

// 3. Wait for conversion (12-bit = 750ms)
delay_ms(rx_ds18b20_get_conversion_time_ms(&sensor));

// 4. Read temperature
float temp_c = 0.0f;
err = rx_ds18b20_read_temperature(&sensor, &temp_c);
if (err == k_rx_ok) {
    printf("Temperature: %.2fdegC\n", temp_c);
}

// 5. Cleanup
rx_ds18b20_deinit(&sensor);

```

Example: Multi-Drop Bus with ROM Matching

```

// Read ROM codes first (use search algorithm or known values)
uint8_t sensor1_rom[8] = {0x28, 0x12, 0x34, 0x56, 0x78, 0x9A, 0xBC, 0xDE};
uint8_t sensor2_rom[8] = {0x28, 0xFE, 0xDC, 0xBA, 0x98, 0x76, 0x54, 0x32};

rx_ds18b20_handle_t sensor1, sensor2;
rx_ds18b20_config_t config1 = {
    .bus_manager = &g_bus_manager,
    .bus_name = "onewire0",
    .resolution = k_ds18b20_resolution_bit,
    .use_rom_matching = true,
    .rom = {0x28, 0x12, 0x34, 0x56, 0x78, 0x9A, 0xBC, 0xDE}
};

// Initialize both sensors on same bus
rx_ds18b20_init(&sensor1, &config1);
config1.rom = sensor2_rom; // Reuse config, change ROM
rx_ds18b20_init(&sensor2, &config1);

// Read from specific sensor
float temp1 = 0.0f;
rx_ds18b20_trigger_conversion(&sensor1);
delay_ms(750);
rx_ds18b20_read_temperature(&sensor1, &temp1);

```

See also

[rx_ds18b20.h](#)
[rx_bus_onewire.h](#)
[rx_crc.h](#)

Date

2026-01-05

Copyright

Copyright (c) 2026 STAR Project

Definition in file [rx_ds18b20.c](#).

6.3.2 Data Structure Documentation

6.3.2.1 struct ds18b20_scratchpad_write_t

Scratchpad write parameters for TH, TL, and config registers.

Internal structure for writing DS18B20 scratchpad registers. DS18B20 Write Scratchpad command (0x4E) writes exactly 3 bytes: TH alarm, TL alarm, and configuration register.

Memory Layout (3 bytes total):

Offset	Field	Description
0	th	High temperature alarm threshold (-55 to +125degC)
1	tl	Low temperature alarm threshold (-55 to +125degC)
2	config	Resolution bits R1:R0 at bits 6:5

Configuration Register Format:

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0	R1	R0	1	1	1	1	1
+-----+							
Resolution bits							

Resolution Encoding:

- R1=0, R0=0: 9-bit (0.5degC, 93.75ms)
- R1=0, R0=1: 10-bit (0.25degC, 187.5ms)
- R1=1, R0=0: 11-bit (0.125degC, 375ms)
- R1=1, R0=1: 12-bit (0.0625degC, 750ms)

Note

This structure is for internal use only - not exposed in public API.

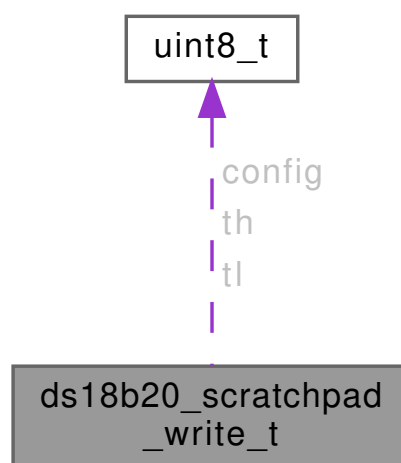
See also

[internal_ds18b20_write_scratchpad\(\)](#)

[internal_ds18b20_resolution_to_config\(\)](#)

Definition at line 332 of file [rx_ds18b20.c](#).

Collaboration diagram for ds18b20_scratchpad_write_t:



Data Fields

uint8_t	config	Configuration register (bits 6:5 = resolution)
---------	--------	--

uint8_t	th	High alarm threshold register (signed, stored as uint8_t)
uint8_t	tl	Low alarm threshold register (signed, stored as uint8_t)

6.3.3 Macro Definition Documentation

6.3.3.1 CHECK_DS18B20_HANDLE

```
#define CHECK_DS18B20_HANDLE(  
    handle,  
    tag)
```

Value:

```
do {  
    RX_CHECK_NULL_PTR(handle, tag, "handle is nullptr");  
    if (!(handle->initialized) {  
        rx_log_error(tag, "DS18B20 not initialized");  
        return k_rx_err_invalid_state;  
    }  
} while (0)
```



Validate DS18B20 handle pointer and initialization state.

Composite validation macro that checks both nullptr and initialization. Used at the start of all public API functions to enforce preconditions.

Checks Performed:

1. Handle is not nullptr (via RX_CHECK_NULL_PTR)
2. Handle is initialized (handle->initialized == true)

Error Returns:

- k_rx_err_null_ptr if handle is nullptr
- k_rx_err_invalid_state if not initialized

Parameters

in	<i>handle</i>	Handle to validate (type: rx_ds18b20_handle_t*)
in	<i>tag</i>	Logging tag for error messages (type: const char*)

Example:

```
rx_err_t rx_ds18b20_read_temperature(rx_ds18b20_handle_t* handle, float* temp)  
{  
    CHECK_DS18B20_HANDLE(handle, s_tag); // Early return on error  
    // ... function body ...  
}
```

Note

This is a statement macro (uses do-while(0) pattern) - requires semicolon.

Warning

Do not use in expressions - this macro contains early returns.

See also

`RX_CHECK_NULL_PTR`

Definition at line 281 of file `rx_ds18b20.c`.

```
00281 #define CHECK_DS18B20_HANDLE(handle, tag)
00282 do {
00283     RX_CHECK_NULL_PTR(handle, tag, "handle is nullptr");
00284     if (!(handle)->initialized) {
00285         rx_log_error(tag, "DS18B20 not initialized");
00286         return k_rx_err_invalid_state;
00287     }
00288 } while (0)
```

Referenced by `rx_ds18b20_get_resolution()`, `rx_ds18b20_read_power_mode()`, `rx_ds18b20_read_scratchpad()`, `rx_ds18b20_read_temperature()`, `rx_ds18b20_read_temperature_raw()`, `rx_ds18b20_recall_config()`, `rx_ds18b20_save_config()`, `rx_ds18b20_set_resolution()`, and `rx_ds18b20_trigger_conversion()`.

6.3.4 Function Documentation

6.3.4.1 `internal_ds18b20_get_temp_mask()`

```
uint16_t internal_ds18b20_get_temp_mask (
    const ds18b20_resolution_t resolution) [static]
```

Get temperature mask for resolution.

Lower bits are undefined for resolutions < 12-bit and must be masked.

Parameters

in	<i>resolution</i>	Resolution mode
----	-------------------	-----------------

Returns

Temperature mask

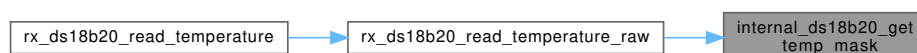
Definition at line 1817 of file `rx_ds18b20.c`.

```
01818 {
01819     switch (resolution) {
01820         case k_ds18b20_resolution_bit:
01821             return k_ds18b20_temp_mask_bit;
01822         case k_ds18b20_resolution_bit:
01823             return k_ds18b20_temp_mask_bit;
01824         case k_ds18b20_resolution_bit:
01825             return k_ds18b20_temp_mask_bit;
01826         case k_ds18b20_resolution_bit:
01827             return k_ds18b20_temp_mask_bit;
01828         default:
01829             return k_ds18b20_temp_mask_bit;
01830     }
01831 }
```

References `k_ds18b20_resolution_bit`, `k_ds18b20_resolution_bit`, `k_ds18b20_resolution_bit`, `k_ds18b20_resolutionbit`, `k_ds18b20_temp_mask_bit`, `k_ds18b20_temp_mask_bit`, `k_ds18b20_temp_mask_bit`, and `k_ds18b20_temp_maskbit`.

Referenced by `rx_ds18b20_read_temperature_raw()`.

Here is the caller graph for this function:



6.3.4.2 internal_ds18b20_raw_to_celsius()

```
float internal_ds18b20_raw_to_celsius (
    const int16_t raw_temp) [static]
```

Convert raw temperature to Celsius.

DS18B20 stores temperature as 16-bit signed value in 1/16degC units. Divide by 16 to get Celsius.

Parameters

in	<i>raw_temp</i>	Raw 16-bit temperature value
----	-----------------	------------------------------

Returns

Temperature in degrees Celsius

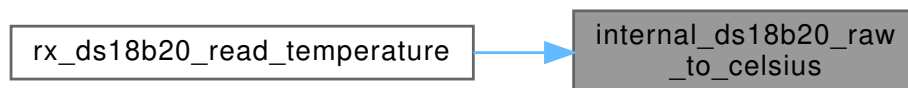
Definition at line 1843 of file rx_ds18b20.c.

```
01844 {
01845     if (raw_temp < k_ds18b20_temp_min_raw || raw_temp > k_ds18b20_temp_max_raw) {
01846         rx_log_error(s_tag, "Raw temperature out of range - returning NaN sentinel");
01847         return NAN;
01848     }
01849
01850     /* Convert from 1/16degC to degC */
01851     float result = (float)raw_temp / s_temp_conversion_divisor;
01852
01853     /* Post-condition: Validate result is finite (not NaN or Inf) */
01854     if (!isfinite(result)) {
01855         rx_log_error(s_tag, "Computed Celsius not finite - returning NaN sentinel");
01856         return NAN;
01857     }
01858
01859     return result;
01860 }
```

References [k_ds18b20_temp_max_raw](#), [s_tag](#), and [s_temp_conversion_divisor](#).

Referenced by [rx_ds18b20_read_temperature\(\)](#).

Here is the caller graph for this function:



6.3.4.3 internal_ds18b20_read_scratchpad_raw()

```
rx_err_t internal_ds18b20_read_scratchpad_raw (
    const rx_ds18b20_handle_t * handle,
    uint8_t scratchpad[k_ds18b20_scratchpad_bytes]) [static]
```

Read DS18B20 scratchpad memory with CRC-8 validation.

Reads all 9 bytes of scratchpad memory and validates data integrity using CRC-8/MAXIM checksum. This is the primary method for reading temperature and configuration data from the sensor.

Algorithm:

```

@startuml
start
:Send reset pulse;
if (Presence detected?) then (yes)
:Select device (Match/Skip ROM);
:Send Read Scratchpad (0xBE);
:Read 9 bytes (8 data + 1 CRC);
:Calculate CRC-8 of first 8 bytes;
if (CRC matches byte 8?) then (yes)
if (Reserved byte == 0xFF?) then (yes)
:Return k_rx_ok;
stop
else (reserved invalid)
:Log warning;
:Return k_rx_ok anyway;
note right: Non-fatal, continue
endif
else (CRC mismatch)
:Log error;
:Return k_rx_err_crc_mismatch;
stop
endif
else (no presence)
:Return k_rx_err_invalid_state;
endif
@enduml

```

Scratchpad Memory Map:

Byte	Field	Description
0	Temp LSB	Temperature low byte
1	Temp MSB	Temperature high byte
2	TH Register	High alarm threshold
3	TL Register	Low alarm threshold
4	Config	Resolution bits (6:5)
5	Reserved	Always 0xFF
6	Reserved	Factory-programmed
7	Reserved	Count remain (legacy)
8	CRC	CRC-8/MAXIM checksum

CRC-8/MAXIM Calculation:

- Polynomial: $0x31 (x^8 + x^5 + x^4 + 1)$
- Initial value: 0x00
- Input: First 8 bytes of scratchpad
- Result: Must match byte 8

Post-Conditions:

- Scratchpad buffer contains valid sensor data
- CRC verified (data integrity guaranteed)
- Reserved byte checked (warning if unexpected)

Parameters

in	<i>handle</i>	DS18B20 sensor handle (must be initialized)
out	<i>scratchpad</i>	9-byte buffer to receive scratchpad data Array must be at least k_ds18b20_scratchpad_bytes (9) bytes

Returns

k_rx_ok Scratchpad read successful, CRC valid

Return values

<i>k_rx_err_null_ptr</i>	handle or scratchpad is nullptr
--------------------------	---------------------------------

<code>k_rx_err_invalid_state</code>	Device not present on bus
<code>k_rx_err_crc_mismatch</code>	CRC validation failed (corrupted data)
<code>k_rx_err_*</code>	Bus communication error

Precondition

`handle != nullptr`
`handle->initialized == true`
`scratchpad != nullptr`
`scratchpad size >= 9 bytes`

Postcondition

If `k_rx_ok`: scratchpad contains validated sensor data
 If error: scratchpad contents undefined

Note

Reserved byte validation is non-fatal (log warning only)

Warning

Do not use scratchpad data if CRC check fails

Performance:

Execution time: ~5ms (reset 1ms + select 1ms + read 3ms)

Example: Manual Scratchpad Access

```

uint8_t scratchpad[9];
rx_err_t err = internal_ds18b20_read_scratchpad_raw(&sensor, scratchpad);
if (err == k_rx_ok) {
    int16_t temp_raw = scratchpad[0] | (scratchpad[1] « 8);
    uint8_t config = scratchpad[4];
    uint8_t resolution = (config » 5) & 0x03;
}

```

See also

[rx_crc8_maxim\(\)](#)
[internal_ds18b20_select_device\(\)](#)

Definition at line 1590 of file rx_ds18b20.c.

```

01592 {
01593     bool presence = false;
01594     rx_err_t err = k_rx_ok;
01595     uint8_t crc_calc = 0;
01596     uint8_t crc_device = 0;
01597
01598     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
01599     RX_CHECK_NULL_PTR(scratchpad, s_tag, "scratchpad is nullptr");
01600

```

```

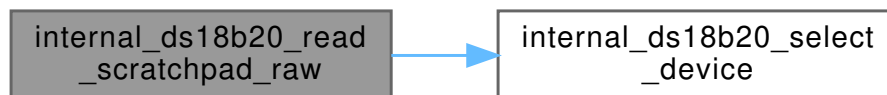
01601  /* Reset and check presence */
01602  err = rx_bus_owewire_reset(handle->bus_manager, handle->bus_name, &presence);
01603  if (err != k_rx_ok || !presence) {
01604      rx_log_error(s_tag, "Device not present for scratchpad read");
01605      return k_rx_err_invalid_state;
01606  }
01607
01608  /* Select device */
01609  err = internal_ds18b20_select_device(handle);
01610  if (err != k_rx_ok) {
01611      return err;
01612  }
01613
01614  /* Send Read Scratchpad command */
01615  err =
01616      rx_bus_owewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_read_scratch);
01617  if (err != k_rx_ok) {
01618      rx_log_error(s_tag, "Failed to send read scratchpad command");
01619      return err;
01620  }
01621
01622  /* Read 9 bytes (8 data + 1 CRC) */
01623  err = rx_bus_owewire_read(handle->bus_manager,
01624                          handle->bus_name,
01625                          scratchpad,
01626                          k_ds18b20_scratchpad_bytes);
01627  if (err != k_rx_ok) {
01628      rx_log_error(s_tag, "Failed to read scratchpad data");
01629      return err;
01630  }
01631
01632  /* Validate CRC */
01633  crc_calc = rx_crc8_maxim(scratchpad, k_ds18b20_crc_bytes);
01634  crc_device = scratchpad[k_ds18b20_scratch_crc];
01635
01636  if (crc_calc != crc_device) {
01637      rx_log_error(s_tag, "Scratchpad CRC mismatch");
01638      return k_rx_err_crc_mismatch;
01639  }
01640
01641  /* Post-condition: Validate scratchpad structure (reserved byte should be 0xFF) */
01642  if (scratchpad[k_ds18b20_scratch_reserved1] != s_ds18b20_reserved_byte_value) {
01643      rx_log_warn(s_tag, "Scratchpad reserved byte unexpected value");
01644  }
01645
01646  return k_rx_ok;
01647 }

```

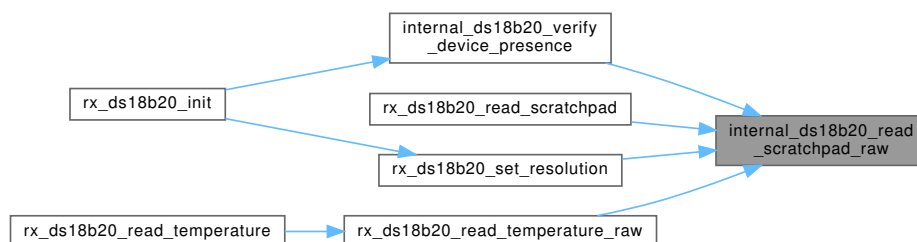
References [rx_ds18b20_handle_t::bus_manager](#), [rx_ds18b20_handle_t::bus_name](#), [internal_ds18b20_select_device\(\)](#), [k_ds18b20_cmd_read_scratch](#), [k_ds18b20_crc_bytes](#), [k_ds18b20_scratch_crc](#), [k_ds18b20_scratch_reserved1](#), [k_ds18b20_scratchpad_bytes](#), [s_ds18b20_reserved_byte_value](#), and [s_tag](#).

Referenced by [internal_ds18b20_verify_device_presence\(\)](#), [rx_ds18b20_read_scratchpad\(\)](#), [rx_ds18b20_read_temperature_raw\(\)](#), and [rx_ds18b20_set_resolution\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.3.4.4 internal_ds18b20_resolution_to_config()

```
uint8_t internal_ds18b20_resolution_to_config (
    const ds18b20_resolution_t resolution) [static]
```

Convert resolution enum to config register value.

Parameters

in	<i>resolution</i>	Resolution mode
----	-------------------	-----------------

Returns

Configuration register value

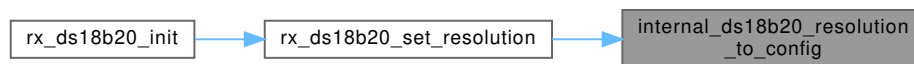
Definition at line 1800 of file [rx_ds18b20.c](#).

```
01801 {
01802     /* Resolution bits are R1:R0 at bits 6:5 */
01803     const uint8_t config = (uint8_t)resolution « k_ds18b20_config_r0_bit;
01804
01805     return config;
01806 }
```

References [k_ds18b20_config_r0_bit](#).

Referenced by [rx_ds18b20_set_resolution\(\)](#).

Here is the caller graph for this function:



6.3.4.5 internal_ds18b20_select_device()

```
rx_err_t internal_ds18b20_select_device (
    const rx_ds18b20_handle_t * handle) [static]
```

Select DS18B20 device using ROM matching or Skip ROM command.

Sends appropriate ROM command based on bus configuration:

- **Match ROM (0x55)** - Address specific device on multi-drop bus
- **Skip ROM (0xCC)** - Broadcast to single device or all devices

Decision Flow:

```

@startuml
start
if (use_rom_matching?) then (yes)
:Send Match ROM (0x55);
:Send 64-bit ROM code;
note right
    Addresses specific device
    by unique ROM ID
end note
else (no)
:Send Skip ROM (0xCC);
note right
    Single device: addresses only device
    Multiple devices: broadcasts to all
end note
endif
:Device selected;
:Ready for function command;
stop
@enduml

```

ROM Matching (Multi-Drop Bus):

- Used when multiple DS18B20 sensors on one bus
- Requires 64-bit ROM code (obtained via search or label)
- Ensures commands go to intended sensor only

Skip ROM (Single Device):

- Faster - no need to send 64-bit ROM
- Safe when only one device on bus
- Caution: broadcasts if multiple devices present

Timing:

- Match ROM: ~3ms (1 byte command + 8 bytes ROM)
- Skip ROM: ~1ms (1 byte command)

Parameters

in	<i>handle</i>	DS18B20 handle with ROM configuration
----	---------------	---------------------------------------

Returns

`k_rx_ok` Device selected successfully

Return values

<code>k_rx_err_*</code>	Bus communication error
-------------------------	-------------------------

Precondition

handle->use_rom_matching is configured

If ROM matching: handle->rom contains valid 64-bit ID

Postcondition

Selected device ready for function commands

Note

Called before every DS18B20 function command

Warning

Skip ROM on multi-drop bus affects ALL devices

Example: ROM Code Format

Byte 0: Family Code (0x28 for DS18B20)
 Byte 1-6: Serial Number (48-bit unique ID)
 Byte 7: CRC-8 checksum

See also

rx_bus_onewire_match_rom()

rx_bus_onewire_skip_rom()

Definition at line 1466 of file rx_ds18b20.c.

```

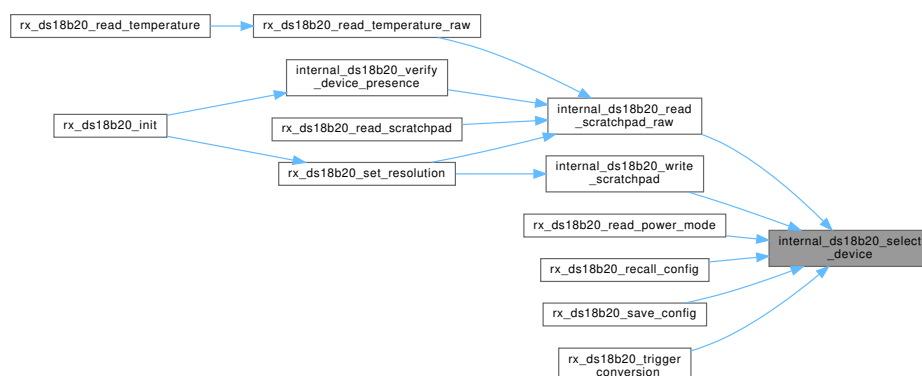
01467 {
01468     if (handle->use_rom_matching) {
01469         /* Use Match ROM to address specific device */
01470         rx_err_t err = rx_bus_onewire_match_rom(handle->bus_manager, handle->bus_name, handle->rom);
01471         if (err != k_rx_ok) {
01472             rx_log_error(s_tag, "Failed to send Match ROM");
01473             return err;
01474         }
01475     } else {
01476         /* Use Skip ROM for single device or broadcast */
01477         rx_err_t err = rx_bus_onewire_skip_rom(handle->bus_manager, handle->bus_name);
01478         if (err != k_rx_ok) {
01479             rx_log_error(s_tag, "Failed to send Skip ROM");
01480             return err;
01481         }
01482     }
01483     return k_rx_ok;
01484 }
01485

```

References [rx_ds18b20_handle_t::bus_manager](#), [rx_ds18b20_handle_t::bus_name](#), [rx_ds18b20_handle_t::rom](#), [s_tag](#), and [rx_ds18b20_handle_t::use_rom_matching](#).

Referenced by [internal_ds18b20_read_scratchpad_raw\(\)](#), [internal_ds18b20_write_scratchpad\(\)](#), [rx_ds18b20_read_power_mode\(\)](#), [rx_ds18b20_recall_config\(\)](#), [rx_ds18b20_save_config\(\)](#), and [rx_ds18b20_trigger_conversion\(\)](#).

Here is the caller graph for this function:



6.3.4.6 internal_ds18b20_validate_config()

```
rx_err_t internal_ds18b20_validate_config (
    const rx_ds18b20_config_t * config,
    const rx_ds18b20_handle_t * handle) [static]
```

Validate DS18B20 configuration parameters before initialization.

Comprehensive validation of configuration structure and handle state. Called by `rx_ds18b20_init()` to enforce pre-conditions and prevent misconfiguration.

Algorithm:

1. Check config pointer is not nullptr
2. Check bus_manager pointer is not nullptr
3. Check bus_name string is not nullptr
4. Verify handle is not already initialized (prevent re-initialization)
5. Validate resolution is in valid range (0-3)
6. If ROM matching enabled, verify family code is 0x28 (DS18B20)

Validation Rules:

Field	Check	Error Code
config	!= nullptr	k_rx_err_null_ptr
bus_manager	!= nullptr	k_rx_err_null_ptr
bus_name	!= nullptr	k_rx_err_null_ptr
initialized	== false	k_rx_err_invalid_state
resolution	<= 3	k_rx_err_invalid_arg
rom[0] (if ROM)	== 0x28	k_rx_err_invalid_arg

Parameters

in	<i>config</i>	Configuration structure to validate
in	<i>handle</i>	Handle to check for existing initialization

Returns

k_rx_ok Configuration is valid

Return values

<i>k_rx_err_null_ptr</i>	config, bus_manager, or bus_name is nullptr
<i>k_rx_err_invalid_state</i>	handle already initialized
<i>k_rx_err_invalid_arg</i>	Invalid resolution or wrong family code

Precondition

config != nullptr
handle != nullptr (checked by caller)

Postcondition

If `k_rx_ok` returned, all fields are valid for initialization

Note

This is an internal helper - not exposed in public API

Warning

Do not call after initialization - will return error

See also

[rx_ds18b20_init\(\)](#)

[ds18b20_resolution_t](#)

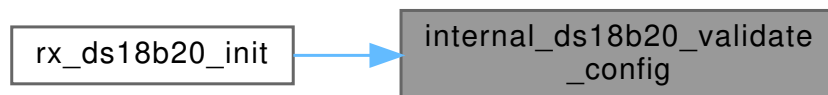
Definition at line 1277 of file [rx_ds18b20.c](#).

```
01279 {
01280     RX_CHECK_NULL_PTR(config, s_tag, "config is nullptr");
01281     RX_CHECK_NULL_PTR(config->bus_manager, s_tag, "bus_manager is nullptr");
01282     RX_CHECK_NULL_PTR(config->bus_name, s_tag, "bus_name is nullptr");
01283
01284     if (handle->initialized) {
01285         rx_log_warn(s_tag, "DS18B20 already initialized");
01286         return k_rx_err_invalid_state;
01287     }
01288
01289     if (config->resolution > k_ds18b20_resolution_bit) {
01290         rx_log_error(s_tag, "Invalid resolution setting");
01291         return k_rx_err_invalid_arg;
01292     }
01293
01294     if (config->use_rom_matching && config->rom[0] != s_ds18b20_family_code) {
01295         rx_log_error(s_tag, "Invalid DS18B20 family code");
01296         return k_rx_err_invalid_arg;
01297     }
01298
01299     return k_rx_ok;
01300 }
```

References [rx_ds18b20_config_t::bus_manager](#), [rx_ds18b20_config_t::bus_name](#), [rx_ds18b20_handle_t::initialized](#), [k_ds18b20_resolution_bit](#), [rx_ds18b20_config_t::resolution](#), [rx_ds18b20_config_t::rom](#), [s_ds18b20_family_code](#), [s_tag](#), and [rx_ds18b20_config_t::use_rom_matching](#).

Referenced by [rx_ds18b20_init\(\)](#).

Here is the caller graph for this function:



6.3.4.7 internal_ds18b20_verify_device_presence()

```
rx_err_t internal_ds18b20_verify_device_presence (
    rx_ds18b20_handle_t * handle)  [static]
```

Verify DS18B20 device presence and establish communication.

Three-step verification process to ensure sensor is physically connected and responding correctly before completing initialization.

Algorithm:

```
@startuml
start
:Initialize OneWire bus;
if (Bus init successful?) then (yes)
:Send reset pulse;
if (Presence pulse detected?) then (yes)
:Read scratchpad (with CRC);
if (Scratchpad valid?) then (yes)
:Return k_rx_ok;
stop
else (CRC fail)
:Return error;
endif
else (no presence)
:Log "device not present";
:Return k_rx_err_invalid_state;
stop
endif
endif
else (bus init fail)
:Return error;
endif
@enduml
```

Verification Steps:

1. **Bus Initialization** - Configure GPIO for 1-Wire timing
2. **Presence Detection** - Send reset, check for presence pulse
3. **Communication Test** - Read scratchpad with CRC validation

Timing:

- Reset pulse: 480-960 us LOW
- Presence detect: 60-240 us after reset
- Scratchpad read: ~5ms (9 bytes + CRC)

Parameters

in,out	<i>handle</i>	DS18B20 handle with bus configuration
--------	---------------	---------------------------------------

Returns

k_rx_ok Device present and communicating

Return values

<i>k_rx_err_invalid_state</i>	No presence pulse detected
-------------------------------	----------------------------

<code>k_rx_err_*</code>	Bus initialization or scratchpad read error
-------------------------	---

Precondition

handle->bus_manager is valid

handle->bus_name is valid

Postcondition

If `k_rx_ok`, device is ready for configuration

Note

This function is called during [rx_ds18b20_init\(\)](#)

Warning

Blocks for ~100ms total (bus init + scratchpad read)

Performance:

Execution time: ~100ms (bus init 50ms + reset 1ms + scratchpad 5ms + margin)

See also

[rx_bus_owewire_init\(\)](#)

[rx_bus_owewire_reset\(\)](#)

[internal_ds18b20_read_scratchpad_raw\(\)](#)

Definition at line 1365 of file [rx_ds18b20.c](#).

```

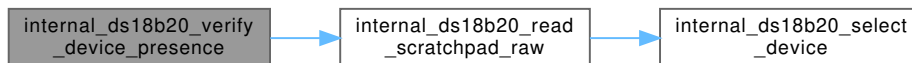
01366 {
01367     /* Initialize OneWire bus */
01368     rx_err_t err = rx_bus_owewire_init(handle->bus_manager, handle->bus_name);
01369     if (err != k_rx_ok) {
01370         rx_log_error(s_tag, "Failed to initialize OneWire bus");
01371         return err;
01372     }
01373
01374     /* Check device presence */
01375     bool presence = false;
01376     err = rx_bus_owewire_reset(handle->bus_manager, handle->bus_name, &presence);
01377     if (err != k_rx_ok) {
01378         rx_log_error(s_tag, "OneWire reset failed");
01379         return err;
01380     }
01381
01382     if (!presence) {
01383         rx_log_error(s_tag, "DS18B20 device not present on bus");
01384         return k_rx_err_invalid_state;
01385     }
01386
01387     /* Read scratchpad to verify communication */
01388     uint8_t scratchpad[k_ds18b20_scratchpad_bytes];
01389     err = internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
01390     if (err != k_rx_ok) {
01391         rx_log_error(s_tag, "Failed to read scratchpad during init");
01392         return err;
01393     }
01394
01395     return k_rx_ok;
01396 }

```

References [rx_ds18b20_handle_t::bus_manager](#), [rx_ds18b20_handle_t::bus_name](#), [internal_ds18b20_read_scratchpad_raw\(\)](#), [k_ds18b20_scratchpad_bytes](#), and [s_tag](#).

Referenced by [rx_ds18b20_init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.3.4.8 internal_ds18b20_write_scratchpad()

```

rx_err_t internal_ds18b20_write_scratchpad (
    const rx_ds18b20_handle_t * handle,
    const ds18b20_scratchpad_write_t * scratchpad) [static]
  
```

Write DS18B20 scratchpad registers (TH, TL, Config).

Writes the three user-modifiable scratchpad registers: high alarm (TH), low alarm (TL), and configuration (resolution). This is the only way to change sensor resolution and alarm thresholds.

Algorithm:

```

@startuml
start
:Send reset pulse;
if (Presence detected?) then (yes)
:Select device (Match/Skip ROM);
:Send Write Scratchpad (0x4E);
:Write TH byte;
:Write TL byte;
:Write Config byte;
:Return k_rx_ok;
note right
    Changes take effect immediately
    but are volatile (lost on power cycle)
end note
stop
else (no presence)
:Return k_rx_err_invalid_state;
endif
@enduml
  
```

Write Sequence:

1. Reset bus and check presence
2. Select device (Match ROM or Skip ROM)
3. Send Write Scratchpad command (0x4E)
4. Write 3 bytes in order: TH, TL, Config

Configuration Register Encoding:

```

Bit: 7 6 5 4 3 2 1 0
+---+---+---+---+---+---+---+
| 0 | R1 | R0 | 1 | 1 | 1 | 1 | 1 |
+---+---+---+---+---+---+---+
      +---+---+
      Resolution
  
```

R1	R0	Resolution	Conversion Time
0	0	9-bit	94 ms
0	1	10-bit	188 ms
1	0	11-bit	375 ms
1	1	12-bit	750 ms

Persistence:

- Scratchpad writes are **volatile** (RAM only)
- Lost on power cycle or reset
- Use Copy Scratchpad (0x48) to save to EEPROM

Parameters

in	<i>handle</i>	DS18B20 sensor handle (must be initialized)
----	---------------	---

in	<i>scratchpad</i>	Write parameters (TH, TL, config)
----	-------------------	-----------------------------------

Returns

k_rx_ok Scratchpad written successfully

Return values

<i>k_rx_err_null_ptr</i>	handle or scratchpad is nullptr
<i>k_rx_err_invalid_state</i>	Device not present on bus
<i>k_rx_err_*</i>	Bus communication error

Precondition

handle != nullptr
handle->initialized == true
scratchpad != nullptr

Postcondition

Sensor resolution/alarms updated (volatile, RAM only)

Note

Changes are immediate but volatile (lost on power cycle)

Warning

No CRC verification - ensure bus integrity

Performance:

Execution time: ~3ms (reset 1ms + select 1ms + write 1ms)

Example: Change Resolution to 10-bit

```
uint8_t scratchpad_data[9];
internal_ds18b20_read_scratchpad_raw(&sensor, scratchpad_data);

ds18b20_scratchpad_write_t write_cfg = {
    .th = scratchpad_data[2], // Preserve TH
    .tl = scratchpad_data[3], // Preserve TL
    .config = (0x01 « 5) | 0x1F // 10-bit: R1=0, R0=1
};
internal_ds18b20_write_scratchpad(&sensor, &write_cfg);

// Save to EEPROM to make permanent
rx_ds18b20_save_config(&sensor);
```

See also

[rx_ds18b20_save_config\(\)](#) Make changes permanent
[internal_ds18b20_select_device\(\)](#)
[ds18b20_scratchpad_write_t](#)

Definition at line 1747 of file [rx_ds18b20.c](#).

```

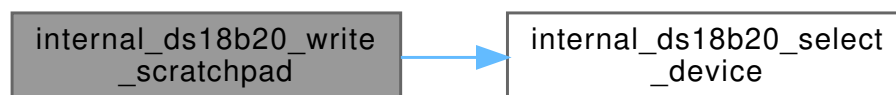
01749 {
01750     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
01751     RX_CHECK_NULL_PTR(scratchpad, s_tag, "scratchpad is nullptr");
01752
01753     /* Reset and check presence */
01754     bool presence = false;
01755     rx_err_t err = rx_bus_owewire_reset(handle->bus_manager, handle->bus_name, &presence);
01756     if (err != k_rx_ok || !presence) {
01757         rx_log_error(s_tag, "Device not present for scratchpad write");
01758         return k_rx_err_invalid_state;
01759     }
01760
01761     /* Select device */
01762     err = internal_ds18b20_select_device(handle);
01763     if (err != k_rx_ok) {
01764         return err;
01765     }
01766
01767     /* Send Write Scratchpad command */
01768     err =
01769         rx_bus_owewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_write_scratch);
01770     if (err != k_rx_ok) {
01771         rx_log_error(s_tag, "Failed to send write scratchpad command");
01772         return err;
01773     }
01774
01775     /* Write 3 bytes (TH, TL, Config) */
01776     uint8_t write_buf[k_ds18b20_scratchpad_write_bytes];
01777     write_buf[k_ds18b20_write_idx_th] = scratchpad->th;
01778     write_buf[k_ds18b20_write_idx_tl] = scratchpad->tl;
01779     write_buf[k_ds18b20_write_idx_config] = scratchpad->config;
01780
01781     err = rx_bus_owewire_write(handle->bus_manager,
01782                               handle->bus_name,
01783                               write_buf,
01784                               k_ds18b20_scratchpad_write_bytes);
01785     if (err != k_rx_ok) {
01786         rx_log_error(s_tag, "Failed to write scratchpad data");
01787         return err;
01788     }
01789
01790     return k_rx_ok;
01791 }

```

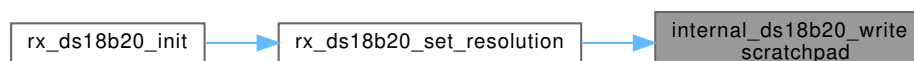
References [rx_ds18b20_handle_t::bus_manager](#), [rx_ds18b20_handle_t::bus_name](#), [ds18b20_scratchpad_write_t::config](#), [internal_ds18b20_select_device\(\)](#), [k_ds18b20_cmd_write_scratch](#), [k_ds18b20_scratchpad_write_bytes](#), [k_ds18b20_write_idx_config](#), [k_ds18b20_write_idx_th](#), [k_ds18b20_write_idx_tl](#), [s_tag](#), [ds18b20_scratchpad_write_t::th](#), and [ds18b20_scratchpad_write_t::tl](#).

Referenced by [rx_ds18b20_set_resolution\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.3.4.9 rx_ds18b20_deinit()

```
rx_err_t rx_ds18b20_deinit (
    rx_ds18b20_handle_t * handle) [nodiscard]
```

Deinitialize DS18B20 sensor handle and release all resources.

Deinitialize DS18B20 sensor.

Tears down a previously initialized DS18B20 handle by zeroing all internal state via memset. After this call the handle is safe to reinitialize with [rx_ds18b20_init\(\)](#).

Algorithm steps:

1. Validate handle is not nullptr
2. Verify handle is currently initialized
3. Clear the entire handle structure with memset (zeros all fields)
4. Log deinitialization success

Parameters

in,out	<i>handle</i>	Sensor handle to deinitialize (must be initialized, non-null)
--------	---------------	---

Returns

rx_err_t Error code

Return values

<i>k_rx_ok</i>	Handle successfully cleared
<i>k_rx_err_null_ptr</i>	handle pointer is nullptr
<i>k_rx_err_invalid_state</i>	handle was not previously initialized

Precondition

handle must be a valid non-null pointer

handle must have been successfully initialized via [rx_ds18b20_init\(\)](#)

Postcondition

All handle fields are zeroed (initialized flag is false)

handle may be safely reused with [rx_ds18b20_init\(\)](#)

Note

Not thread-safe; caller must ensure no concurrent access to handle

Example:

```
rx_ds18b20_handle_t sensor;
rx_ds18b20_init(&sensor, &config);
// ... use sensor ...
rx_err_t err = rx_ds18b20_deinit(&sensor);
if (err != k_rx_ok) {
    rx_log_error("app", "Failed to deinit DS18B20");
}
```

See also

[rx_ds18b20_init\(\)](#) Initialize the handle before use

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

Definition at line 441 of file [rx_ds18b20.c](#).

```
00442 {
00443     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00444
00445     if (!handle->initialized) {
00446         rx_log_warn(s_tag, "DS18B20 not initialized");
00447         return k_rx_err_invalid_state;
00448     }
00449
00450     /* Clear handle */
00451     *handle = (rx_ds18b20_handle_t){0};
00452
00453     rx_log_info(s_tag, "DS18B20 deinitialized");
00454     return k_rx_ok;
00455 }
```

References [rx_ds18b20_handle_t::initialized](#), and [s_tag](#).

6.3.4.10 rx_ds18b20_get_conversion_time_ms()

```
uint32_t rx_ds18b20_get_conversion_time_ms (
    const rx\_ds18b20\_handle\_t * handle)
```

Get temperature conversion time for current resolution.

Get conversion time for current resolution.

Returns the required wait time after triggering conversion based on the configured resolution (9-bit: 94ms, 10-bit: 188ms, 11-bit: 375ms, 12-bit: 750ms).

Parameters

in	<i>handle</i>	Sensor handle
----	---------------	---------------

Returns

Conversion time in milliseconds, or invalid value if handle is nullptr/uninitialized

Definition at line 1204 of file [rx_ds18b20.c](#).

```

01205 {
01206     if (handle == nullptr || !handle->initialized) {
01207         return s_ds18b20_conversion_time_invalid_u32;
01208     }
01209
01210     switch (handle->resolution) {
01211         case k_ds18b20_resolution_bit:
01212             return k_ds18b20_conv_time_bit_ms;
01213         case k_ds18b20_resolution_bit:
01214             return k_ds18b20_conv_time_bit_ms;
01215         case k_ds18b20_resolution_bit:
01216             return k_ds18b20_conv_time_bit_ms;
01217         case k_ds18b20_resolution_bit:
01218             return k_ds18b20_conv_time_bit_ms;
01219         default:
01220             return s_ds18b20_conversion_time_invalid_u32;
01221     }
01222 }
```

References [rx_ds18b20_handle_t::initialized](#), [k_ds18b20_conv_time_bit_ms](#), [k_ds18b20_conv_time_bit_ms](#), [k_ds18b20_conv_time_bit_ms](#), [k_ds18b20_conv_timebit_ms](#), [k_ds18b20_resolution_bit](#), [k_ds18b20_resolution_bit](#), [k_ds18b20_resolution_bit](#), [k_ds18b20_resolutionbit](#), [rx_ds18b20_handle_t::resolution](#), and [s_ds18b20_conversion_time_invalid_u32](#)

6.3.4.11 rx_ds18b20_get_resolution()

```

rx_err_t rx_ds18b20_get_resolution (
    const rx_ds18b20_handle_t * handle,
    ds18b20_resolution_t * resolution) [nodiscard]
```

Get the currently configured ADC resolution from DS18B20 handle.

Get current temperature resolution.

Returns the resolution value stored in the handle, which reflects the last successfully applied resolution (set during initialization or via [rx_ds18b20_set_resolution\(\)](#)). This is a cached read from the handle state; it does not issue a 1-Wire bus transaction.

Algorithm steps:

1. Validate handle and output pointer are non-null
2. Verify handle is initialized
3. Copy handle->resolution to *resolution

Parameters

in	<i>handle</i>	Sensor handle (must be initialized, non-null)
out	<i>resolution</i>	Receives the currently configured ds18b20_resolution_t value; undefined on error

Returns

[rx_err_t](#) Error code

Return values

<i>k_rx_ok</i>	Resolution successfully returned
----------------	----------------------------------

<code>k_rx_err_null_ptr</code>	handle or resolution pointer is nullptr
<code>k_rx_err_invalid_state</code>	handle not initialized (CHECK_DS18B20_HANDLE fails)

Precondition

handle must be initialized via [rx_ds18b20_init\(\)](#)
 resolution must be a valid non-null pointer

Postcondition

*resolution contains the current resolution value on `k_rx_ok`
 handle state is unchanged

Note

Not thread-safe; caller must ensure handle is not concurrently modified
 Returns cached value from handle – does not issue 1-Wire bus transaction

Example:

```
ds18b20_resolution_t res;
rx_err_t err = rx_ds18b20_get_resolution(&sensor, &res);
if (err == k_rx_ok) {
    uint32_t conv_ms = rx_ds18b20_get_conversion_time_ms(&sensor);
    rx_log_info("app", "Resolution %d, conv time %u ms", (int)res, (unsigned)conv_ms);
}
```

See also

[rx_ds18b20_set_resolution\(\)](#) Change the ADC resolution
[rx_ds18b20_get_conversion_time_ms\(\)](#) Get conversion time for current resolution

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

Definition at line 858 of file [rx_ds18b20.c](#).

```
00860 {
00861     CHECK_DS18B20_HANDLE(handle, s_tag);
00862     RX_CHECK_NULL_PTR(resolution, s_tag, "resolution is nullptr");
00863     *resolution = handle->resolution;
00865     return k_rx_ok;
00866 }
```

References [CHECK_DS18B20_HANDLE](#), [rx_ds18b20_handle_t::resolution](#), and [s_tag](#).

6.3.4.12 rx_ds18b20_init()

```
rx_err_t rx_ds18b20_init (  
    rx_ds18b20_handle_t * handle,  
    const rx_ds18b20_config_t * config)  [nodiscard]
```

Initialize DS18B20 sensor.

Initializes the DS18B20 sensor and configures resolution. If ROM matching is disabled, uses Skip ROM (single device mode).

Parameters

out	<i>handle</i>	Pointer to DS18B20 handle. Must not be nullptr.
-----	---------------	---

in	config	Pointer to configuration. Must not be nullptr.
----	--------	--

Returns

k_rx_ok on success
 k_rx_err_null_ptr if handle or config is nullptr
 k_rx_err_invalid_arg if bus_manager or bus_name is nullptr
 k_rx_err_invalid_state if device not responding
 k_rx_err_crc if scratchpad CRC check fails

Definition at line 356 of file rx_ds18b20.c.

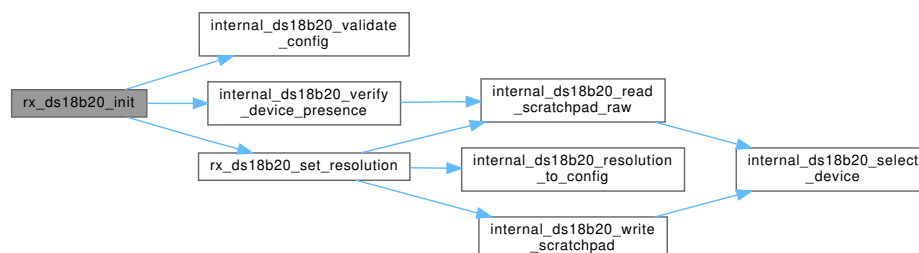
```

00357 {
00358     /* Validate inputs */
00359     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00360
00361     rx_err_t err = internal_ds18b20_validate_config(config, handle);
00362     if (err != k_rx_ok) {
00363         return err;
00364     }
00365
00366     /* Initialize handle */
00367     *handle = (rx_ds18b20_handle_t){0};
00368     handle->bus_manager = config->bus_manager;
00369     handle->bus_name = config->bus_name;
00370     handle->resolution = config->resolution;
00371     handle->use_rom_matching = config->use_rom_matching;
00372
00373     if (config->use_rom_matching) {
00374         memcpy(handle->rom, config->rom, k_onewire_rom_bytes);
00375     }
00376
00377     /* Verify device presence */
00378     err = internal_ds18b20_verify_device_presence(handle);
00379     if (err != k_rx_ok) {
00380         return err;
00381     }
00382
00383     /* Configure resolution */
00384     handle->initialized = true;
00385     err = rx_ds18b20_set_resolution(handle, config->resolution);
00386     if (err != k_rx_ok) {
00387         handle->initialized = false;
00388         return err;
00389     }
00390
00391     rx_log_info(s_tag, "DS18B20 initialized successfully");
00392     return k_rx_ok;
00393 }

```

References [rx_ds18b20_config_t::bus_manager](#), [rx_ds18b20_handle_t::bus_manager](#), [rx_ds18b20_config_t::bus_name](#), [rx_ds18b20_handle_t::bus_name](#), [rx_ds18b20_handle_t::initialized](#), [internal_ds18b20_validate_config\(\)](#), [internal_ds18b20_verify_device_presence\(\)](#), [rx_ds18b20_config_t::resolution](#), [rx_ds18b20_handle_t::resolution](#), [rx_ds18b20_config_t::rom](#), [rx_ds18b20_handle_t::rom](#), [rx_ds18b20_set_resolution\(\)](#), [s_tag](#), [rx_ds18b20_config_t::use_rom_matching](#), and [rx_ds18b20_handle_t::use_rom_matching](#).

Here is the call graph for this function:



6.3.4.13 rx_ds18b20_read_power_mode()

```
rx_err_t rx_ds18b20_read_power_mode (
    const rx_ds18b20_handle_t * handle,
    bool * external_power) [nodiscard]
```

Read DS18B20 power supply mode (external vs. parasitic).

Read power supply mode.

Issues the Read Power Supply (0xB4) command over the 1-Wire bus and reads back a single bit to determine whether the DS18B20 is operating on an external voltage supply or in parasitic (bus-powered) mode.

Per the DS18B20 datasheet: the sensor pulls the bus low for one read time slot to indicate parasitic power mode (0), or releases the bus (1) to indicate external power mode.

Algorithm steps:

1. Validate handle and output pointer are non-null; handle is initialized
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Read Power Supply command byte (0xB4)
5. Read one bit from the bus (0 = parasitic, 1 = external)
6. Write result to *external_power

Parameters

in	<i>handle</i>	Sensor handle (must be initialized, non-null)
out	<i>external_power</i>	Set to true if device uses external power supply, false if device uses parasitic (bus-powered) mode; undefined on error

Returns

rx_err_t Error code

Return values

<i>k_rx_ok</i>	Power mode successfully read
<i>k_rx_err_null_ptr</i>	handle or external_power pointer is nullptr
<i>k_rx_err_invalid_state</i>	handle not initialized or device not present on bus
<i>k_rx_err_bus_error</i>	1-Wire bus reset, write, or read operation failed

Precondition

handle must be initialized via [rx_ds18b20_init\(\)](#)

DS18B20 device must be physically connected to the bus

Postcondition

*external_power reflects the device power supply mode on k_rx_ok
handle state is unchanged

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Some operations (Copy Scratchpad) are unreliable in parasitic power mode

Example:

```
bool ext_power = false;
rx_err_t err = rx_ds18b20_read_power_mode(&sensor, &ext_power);
if (err == k_rx_ok && !ext_power) {
    rx_log_warn("app", "DS18B20 in parasitic mode: EEPROM writes may fail");
}
```

See also

[rx_ds18b20_save_config\(\)](#) EEPROM write (requires external power)

[rx_ds18b20_init\(\)](#) Initialize sensor handle

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null checks/initialized, device present), 2 postconditions

Definition at line 1083 of file rx_ds18b20.c.

```
01084 {
01085     CHECK_DS18B20_HANDLE(handle, s_tag);
01086     RX_CHECK_NULL_PTR(external_power, s_tag, "external_power is nullptr");
01087
01088     /* Reset and check presence */
01089     bool presence = false;
01090     rx_err_t err = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
01091     if (err != k_rx_ok || !presence) {
01092         rx_log_error(s_tag, "Device not present for power mode read");
01093         return k_rx_err_invalid_state;
01094     }
01095
01096     /* Select device */
01097     err = internal_ds18b20_select_device(handle);
01098     if (err != k_rx_ok) {
01099         return err;
01100     }
01101
01102     /* Send Read Power Supply command */
01103     err = rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_read_power);
01104     if (err != k_rx_ok) {
01105         rx_log_error(s_tag, "Failed to send read power command");
01106         return err;
01107     }
01108
01109     /* Read power bit (1 = external, 0 = parasitic) */
01110     bool power_bit = false;
01111     err = rx_bus_onewire_read_bit(handle->bus_manager, handle->bus_name, &power_bit);
```

```

01112 if (err != k_rx_ok) {
01113     rx_log_error(s_tag, "Failed to read power bit");
01114     return err;
01115 }
01116
01117 *external_power = power_bit;
01118 return k_rx_ok;
01119 }

```

References [rx_ds18b20_handle_t::bus_manager](#), [rx_ds18b20_handle_t::bus_name](#), [CHECK_DS18B20_HANDLE](#), [internal_ds18b20_select_device\(\)](#), [k_ds18b20_cmd_read_power](#), and [s_tag](#).

Here is the call graph for this function:



6.3.4.14 rx_ds18b20_read_scratchpad()

```

rx_err_t rx_ds18b20_read_scratchpad (
    rx_ds18b20_handle_t * handle,
    uint8_t scratchpad[k_ds18b20_scratchpad_bytes]) [nodiscard]

```

Read the full 9-byte scratchpad register from DS18B20 sensor.

Read scratchpad memory.

Reads all nine bytes of the DS18B20 scratchpad memory and returns the raw buffer to the caller. The scratchpad layout is:

Byte	Field	Description
0	Temperature LSB	Low byte of temperature register
1	Temperature MSB	High byte of temperature register
2	TH Register	High alarm threshold
3	TL Register	Low alarm threshold
4	Config Register	Resolution configuration byte
5-7	Reserved	Reserved (0xFF, 0x10, 0x10)
8	CRC	CRC-8 of bytes 0-7

CRC is verified internally by [internal_ds18b20_read_scratchpad_raw\(\)](#) before the buffer is returned. This function is a thin public wrapper around the internal helper.

Algorithm steps:

1. Validate handle and scratchpad buffer pointer are non-null
2. Verify handle is initialized
3. Delegate to [internal_ds18b20_read_scratchpad_raw\(\)](#) which issues the 1-Wire reset, Match/Skip ROM selection, Read Scratchpad (0xBE) command, reads 9 bytes, and verifies CRC-8

Parameters

in,out	<i>handle</i>	Sensor handle (must be initialized, non-null)
--------	---------------	---

out	<i>scratchpad</i>	Buffer of exactly k_ds18b20_scratchpad_bytes (9) bytes to receive the raw scratchpad contents; undefined on error
-----	-------------------	---

Returns

rx_err_t Error code

Return values

<i>k_rx_ok</i>	Scratchpad successfully read and CRC verified
<i>k_rx_err_null_ptr</i>	handle or scratchpad buffer is nullptr
<i>k_rx_err_invalid_state</i>	handle not initialized
<i>k_rx_err_crc_failure</i>	Scratchpad CRC-8 check failed (data corrupted)
<i>k_rx_err_bus_error</i>	1-Wire bus communication failure

Precondition

handle must be initialized via [rx_ds18b20_init\(\)](#)

scratchpad must point to a buffer of at least k_ds18b20_scratchpad_bytes bytes

Postcondition

scratchpad[0..8] contains valid data on k_rx_ok

CRC of scratchpad bytes 0-7 matches byte 8 on k_rx_ok

Note

Not thread-safe; caller must serialize access to the same handle

Use [rx_ds18b20_read_temperature\(\)](#) for converted Celsius value

Example:

```
uint8_t raw[k_ds18b20_scratchpad_bytes];
rx_err_t err = rx_ds18b20_read_scratchpad(&sensor, raw);
if (err == k_rx_ok) {
    uint8_t config_reg = raw[4];
}
```

See also

[rx_ds18b20_read_temperature\(\)](#) High-level temperature read returning float Celsius

[rx_ds18b20_read_temperature_raw\(\)](#) Read raw 16-bit temperature with masking

Since

Version 1.0.0

NASA Power of 10 Compliance:

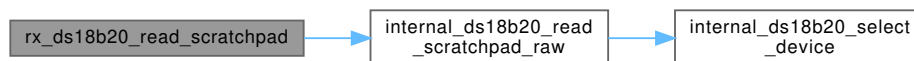
- Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

Definition at line 1185 of file [rx_ds18b20.c](#).

```
01187 {
01188     CHECK_DS18B20_HANDLE(handle, s_tag);
01189     RX_CHECK_NULL_PTR(scratchpad, s_tag, "scratchpad is nullptr");
01190
01191     return internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
01192 }
```

References [CHECK_DS18B20_HANDLE](#), [internal_ds18b20_read_scratchpad_raw\(\)](#), [k_ds18b20_scratchpad_bytes](#), and [s_tag](#).

Here is the call graph for this function:



6.3.4.15 rx_ds18b20_read_temperature()

```
rx_err_t rx_ds18b20_read_temperature (
    rx_ds18b20_handle_t * handle,
    float * temperature_celsius) [nodiscard]
```

Read temperature from DS18B20 sensor in degrees Celsius.

Read temperature in Celsius.

Reads the full 9-byte scratchpad from the DS18B20, extracts the 16-bit raw temperature value, applies the resolution mask to clear undefined low-order bits, validates the result is within the sensor's physical range, and converts to a floating-point Celsius value using the DS18B20 fixed-point encoding (1 LSB = 1/16 degC).

Algorithm steps:

1. Validate handle and output pointer are non-null
2. Call [rx_ds18b20_read_temperature_raw\(\)](#) to obtain masked raw value
3. Convert raw signed 16-bit value to float: $celsius = raw / 16.0f$
4. Write result to `*temperature_celsius`

Parameters

in,out	<i>handle</i>	Sensor handle (must be initialized)
out	<i>temperature_celsius</i>	Converted temperature in degrees Celsius, valid range [-55.0, +125.0]; undefined on error

Returns

`rx_err_t` Error code

Return values

<i>k_rx_ok</i>	Temperature read and converted successfully
----------------	---

<code>k_rx_err_null_ptr</code>	handle or temperature_celsius is nullptr
<code>k_rx_err_invalid_state</code>	handle not initialized
<code>k_rx_err_out_of_range</code>	raw temperature outside [-55degC, +125degC]
<code>k_rx_err_crc_failure</code>	Scratchpad CRC mismatch
<code>k_rx_err_bus_error</code>	1-Wire bus communication failure

Precondition

handle must be initialized via [rx_ds18b20_init\(\)](#)
[rx_ds18b20_trigger_conversion\(\)](#) must have been called and conversion time elapsed

Postcondition

*temperature_celsius contains valid temperature in [-55.0, +125.0] on k_rx_ok
 handle->stats.read_count incremented (via internal_ds18b20_read_scratchpad_raw)

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Returns stale data if conversion time has not elapsed after trigger

Example:

```
float temp_celsius = 0.0f;
rx_err_t err = rx_ds18b20_read_temperature(&sensor, &temp_celsius);
if (err == k_rx_ok) {
    rx_log_info("app", "Temperature: %.4f C", (double)temp_celsius);
}
```

See also

[rx_ds18b20_trigger_conversion\(\)](#) Must be called before reading
[rx_ds18b20_read_temperature_raw\(\)](#) Read the raw 16-bit value
[rx_ds18b20_get_conversion_time_ms\(\)](#) Determine required wait time

Since

Version 1.0.0

NASA Power of 10 Compliance:

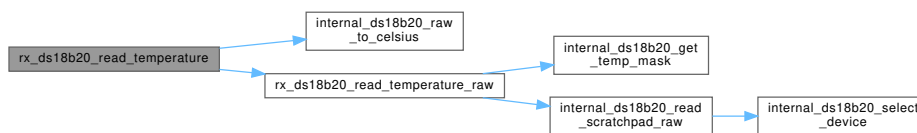
- Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

Definition at line 595 of file `rx_ds18b20.c`.

```
00596 {
00597     CHECK_DS18B20_HANDLE(handle, s_tag);
00598     RX_CHECK_NULL_PTR(temperature_celsius, s_tag, "temperature_celsius is nullptr");
00599
00600     /* Read raw temperature */
00601     int16_t raw_temp = 0;
00602     rx_err_t err = rx_ds18b20_read_temperature_raw(handle, &raw_temp);
00603     if (err != k_rx_ok) {
00604         return err;
00605     }
00606
00607     /* Convert to Celsius */
00608     *temperature_celsius = internal_ds18b20_raw_to_celsius(raw_temp);
00609
00610     return k_rx_ok;
00611 }
```

References `CHECK_DS18B20_HANDLE`, `internal_ds18b20_raw_to_celsius()`, `rx_ds18b20_read_temperature_raw()`, and `s_tag`.

Here is the call graph for this function:



6.3.4.16 rx_ds18b20_read_temperature_raw()

```
rx_err_t rx_ds18b20_read_temperature_raw (
    rx_ds18b20_handle_t * handle,
    int16_t * raw_temp) [nodiscard]
```

Read raw temperature value from DS18B20 sensor in 1/16 degree Celsius units.

Read raw temperature value.

Reads the 9-byte DS18B20 scratchpad register, extracts the two temperature bytes (LSB at offset 0, MSB at offset 1), combines them into a 16-bit unsigned value, applies the resolution-dependent mask to clear undefined low-order bits, casts to signed `int16_t`, and range-validates the result.

The DS18B20 temperature encoding uses a signed fixed-point format where 1 LSB = 1/16 degC (12-bit mode). The valid raw range is:

- Minimum: -55 degC = 0xFC90 (int16: -880)
- Maximum: +125 degC = 0x07D0 (int16: +2000)

Resolution masks (clears undefined bits in lower-resolution modes):

- 9-bit: 0xFFF8 (3 undefined bits)
- 10-bit: 0xFFFC (2 undefined bits)

- 11-bit: 0xFFFE (1 undefined bit)
- 12-bit: 0xFFFF (no undefined bits)

Algorithm steps:

1. Validate handle and output pointer are non-null and handle is initialized
2. Read raw scratchpad bytes via [internal_ds18b20_read_scratchpad_raw\(\)](#)
3. Combine temperature LSB and MSB bytes into uint16_t
4. Apply resolution mask from [internal_ds18b20_get_temp_mask\(\)](#)
5. Cast to int16_t and validate range [k_ds18b20_temp_min_raw, k_ds18b20_temp_max_raw]
6. Write validated result to *raw_temp

Parameters

in,out	<i>handle</i>	Sensor handle (must be initialized)
out	<i>raw_temp</i>	Signed 16-bit raw temperature in 1/16 degC units, valid range [k_ds18b20_temp_min_raw, k_ds18b20_temp_max_raw]; undefined on error

Returns

rx_err_t Error code

Return values

<i>k_rx_ok</i>	Raw temperature read, masked, and validated successfully
<i>k_rx_err_null_ptr</i>	handle or raw_temp is nullptr
<i>k_rx_err_invalid_state</i>	handle not initialized
<i>k_rx_err_out_of_range</i>	raw value outside physical sensor range
<i>k_rx_err_crc_failure</i>	Scratchpad CRC check failed
<i>k_rx_err_bus_error</i>	1-Wire bus communication failure

Precondition

handle must be initialized via [rx_ds18b20_init\(\)](#)

[rx_ds18b20_trigger_conversion\(\)](#) must have been called and conversion time elapsed

Postcondition

*raw_temp is in [k_ds18b20_temp_min_raw, k_ds18b20_temp_max_raw] on k_rx_ok

Resolution mask has been applied; undefined bits are cleared

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Returns stale data if conversion time has not elapsed after trigger

Example:

```
int16_t raw = 0;
rx_err_t err = rx_ds18b20_read_temperature_raw(&sensor, &raw);
if (err == k_rx_ok) {
    float celsius = (float)raw / 16.0f;
}
```

See also

[rx_ds18b20_read_temperature\(\)](#) Convenience wrapper returning float Celsius

[rx_ds18b20_trigger_conversion\(\)](#) Must be called before reading

[rx_ds18b20_set_resolution\(\)](#) Configure ADC resolution and mask

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions (range, mask)

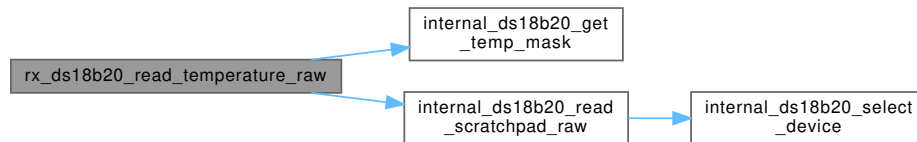
Definition at line 680 of file [rx_ds18b20.c](#).

```
00681 {
00682     CHECK_DS18B20_HANDLE(handle, s_tag);
00683     RX_CHECK_NULL_PTR(raw_temp, s_tag, "raw_temp is nullptr");
00684
00685     /* Read scratchpad */
00686     uint8_t scratchpad[k_ds18b20_scratchpad_bytes];
00687     rx_err_t err = internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
00688     if (err != k_rx_ok) {
00689         return err;
00690     }
00691
00692     /* Extract temperature (LSB + MSB) */
00693     uint16_t temp = (uint16_t)scratchpad[k_ds18b20_scratch_temp_lsb];
00694     temp |= ((uint16_t)scratchpad[k_ds18b20_scratch_temp_msb]) « k_ds18b20_shift_byte;
00695
00696     /* Apply resolution mask to clear undefined bits */
00697     const uint16_t mask = internal_ds18b20_get_temp_mask(handle->resolution);
00698     temp &= mask;
00699
00700     /* Post-condition: Validate temperature is within sensor range */
00701     *raw_temp = (int16_t)temp;
00702     if (*raw_temp < k_ds18b20_temp_min_raw || *raw_temp > k_ds18b20_temp_max_raw) {
00703         rx_log_error(s_tag, "Temperature out of sensor range (-55degC to +125degC)");
00704         return k_rx_err_out_of_range;
00705     }
00706
00707     return k_rx_ok;
00708 }
```

References [CHECK_DS18B20_HANDLE](#), [internal_ds18b20_get_temp_mask\(\)](#), [internal_ds18b20_read_scratchpad_raw\(\)](#), [k_ds18b20_scratch_temp_lsb](#), [k_ds18b20_scratch_temp_msb](#), [k_ds18b20_scratchpad_bytes](#), [k_ds18b20_shift_byte](#), [k_ds18b20_temp_max_raw](#), [rx_ds18b20_handle_t::resolution](#), and [s_tag](#).

Referenced by [rx_ds18b20_read_temperature\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.3.4.17 rx_ds18b20_recall_config()

```
rx_err_t rx_ds18b20_recall_config (
    const rx_ds18b20_handle_t * handle) [nodiscard]
```

Recall configuration from DS18B20 EEPROM into scratchpad RAM.

Recall configuration from EEPROM.

Sends the Recall E2 (0xB8) command over the 1-Wire bus, which instructs the DS18B20 to copy the TH alarm, TL alarm, and configuration register bytes from non-volatile EEPROM back into volatile scratchpad RAM. This is the same operation the device performs automatically on power-up.

Use this function to discard unsaved scratchpad changes and revert to the last saved EEPROM state without cycling power.

Algorithm steps:

1. Validate handle is initialized and non-null
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Recall E2 command byte (0xB8)

Parameters

in	<i>handle</i>	Sensor handle (must be initialized, non-null)
----	---------------	---

Returns

rx_err_t Error code

Return values

<i>k_rx_ok</i>	Recall E2 command successfully issued
<i>k_rx_err_null_ptr</i>	handle is nullptr

<code>k_rx_err_invalid_state</code>	handle not initialized or device not present on bus
<code>k_rx_err_bus_error</code>	1-Wire bus reset or write operation failed

Precondition

handle must be initialized via [rx_ds18b20_init\(\)](#)

DS18B20 device must be physically connected to the bus

Postcondition

DS18B20 scratchpad TH, TL, and configuration bytes reflect last EEPROM save

Any unsaved scratchpad changes since last [rx_ds18b20_save_config\(\)](#) are discarded

Note

Not thread-safe; caller must serialize access to the same handle

The Recall E2 operation is performed automatically by the device on power-up

Example:

```
// Revert any unsaved configuration changes
rx_err_t err = rx_ds18b20_recall_config(&sensor);
if (err != k_rx_ok) {
    rx_log_error("app", "Failed to recall DS18B20 config from EEPROM");
}
```

See also

[rx_ds18b20_save_config\(\)](#) Persist scratchpad configuration to EEPROM

[rx_ds18b20_set_resolution\(\)](#) Modify the scratchpad configuration

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null/initialized, device present), 2 postconditions

Definition at line 999 of file [rx_ds18b20.c](#).

```
01000 {
01001     CHECK_DS18B20_HANDLE(handle, s_tag);
01002
01003     /* Reset and check presence */
01004     bool presence = false;
01005     rx_err_t err = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
01006     if (err != k_rx_ok || !presence) {
01007         rx_log_error(s_tag, "Device not present for EEPROM recall");
01008         return k_rx_err_invalid_state;
01009     }
01010
01011     /* Select device */
01012     err = internal_ds18b20_select_device(handle);
01013     if (err != k_rx_ok) {
01014         return err;
01015     }
01016 }
```

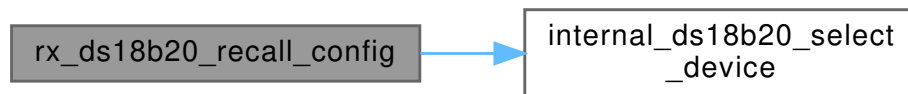
```

01015 }
01016
01017 /* Send Recall E2 command */
01018 err =
01019     rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_recall_eeprom);
01020 if (err != k_rx_ok) {
01021     rx_log_error(s_tag, "Failed to send recall EEPROM command");
01022     return err;
01023 }
01024
01025 return k_rx_ok;
01026 }

```

References [rx_ds18b20_handle_t::bus_manager](#), [rx_ds18b20_handle_t::bus_name](#), [CHECK_DS18B20_HANDLE](#), [internal_ds18b20_select_device\(\)](#), [k_ds18b20_cmd_recall_eeprom](#), and [s_tag](#).

Here is the call graph for this function:



6.3.4.18 rx_ds18b20_save_config()

```

rx_err_t rx_ds18b20_save_config (
    const rx_ds18b20_handle_t * handle) [nodiscard]

```

Save current scratchpad configuration to DS18B20 EEPROM.

Save configuration to EEPROM.

Sends the Copy Scratchpad (0x48) command over the 1-Wire bus, which instructs the DS18B20 to copy the TH alarm, TL alarm, and configuration register bytes from volatile scratchpad RAM to non-volatile EEPROM. The saved values persist across power cycles and are automatically restored on the next power-up via the Recall E2 sequence.

Algorithm steps:

1. Validate handle is initialized and non-null
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Copy Scratchpad command byte (0x48)

Parameters

in	<i>handle</i>	Sensor handle (must be initialized, non-null)
----	---------------	---

Returns

`rx_err_t` Error code

Return values

<i>k_rx_ok</i>	Copy Scratchpad command successfully issued
----------------	---

<code>k_rx_err_null_ptr</code>	handle is nullptr
<code>k_rx_err_invalid_state</code>	handle not initialized or device not present on bus
<code>k_rx_err_bus_error</code>	1-Wire bus reset or write operation failed

Precondition

handle must be initialized via [rx_ds18b20_init\(\)](#)

DS18B20 device must be physically connected and externally powered (Copy Scratchpad is not guaranteed in parasitic power mode)

Postcondition

DS18B20 EEPROM contains current TH, TL, and configuration register values

Configuration will survive next power cycle

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Not reliable in parasitic power mode; use external power for EEPROM writes

Example:

```
// Configure resolution, then persist to EEPROM
rx_ds18b20_set_resolution(&sensor, k_ds18b20_resolution_bit);
rx_err_t err = rx_ds18b20_save_config(&sensor);
if (err != k_rx_ok) {
    rx_log_error("app", "Failed to save DS18B20 config to EEPROM");
}
```

See also

[rx_ds18b20_recall_config\(\)](#) Restore EEPROM contents to scratchpad

[rx_ds18b20_set_resolution\(\)](#) Configure resolution before saving

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null/initialized, device present), 2 postconditions

Definition at line 919 of file [rx_ds18b20.c](#).

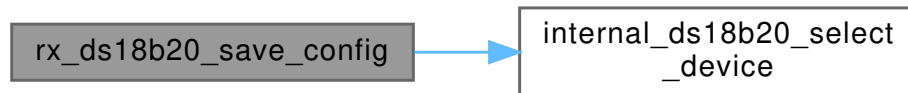
```

00920 {
00921     CHECK_DS18B20_HANDLE(handle, s_tag);
00922
00923     /* Reset and check presence */
00924     bool presence = false;
00925     rx_err_t err = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
00926     if (err != k_rx_ok || !presence) {
00927         rx_log_error(s_tag, "Device not present for EEPROM save");
00928         return k_rx_err_invalid_state;
00929     }
00930
00931     /* Select device */
00932     err = internal_ds18b20_select_device(handle);
00933     if (err != k_rx_ok) {
00934         return err;
00935     }
00936
00937     /* Send Copy Scratchpad command */
00938     err =
00939         rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_copy_scratch);
00940     if (err != k_rx_ok) {
00941         rx_log_error(s_tag, "Failed to send copy scratchpad command");
00942         return err;
00943     }
00944
00945     return k_rx_ok;
00946 }

```

References [rx_ds18b20_handle_t::bus_manager](#), [rx_ds18b20_handle_t::bus_name](#), [CHECK_DS18B20_HANDLE](#), [internal_ds18b20_select_device\(\)](#), [k_ds18b20_cmd_copy_scratch](#), and [s_tag](#).

Here is the call graph for this function:



6.3.4.19 rx_ds18b20_set_resolution()

```

rx_err_t rx_ds18b20_set_resolution (
    rx_ds18b20_handle_t * handle,
    const ds18b20_resolution_t resolution) [nodiscard]

```

Set ADC measurement resolution on DS18B20 sensor.

Set temperature resolution.

Updates the configuration register in the DS18B20 scratchpad to change the ADC resolution. The existing TH and TL alarm threshold bytes are preserved by reading the current scratchpad first, then writing back only the updated configuration byte.

Higher resolution provides finer temperature steps but increases conversion time:

- 9-bit: 0.5 degC steps, 94 ms conversion
- 10-bit: 0.25 degC steps, 188 ms conversion
- 11-bit: 0.125 degC steps, 375 ms conversion

- 12-bit: 0.0625 degC steps, 750 ms conversion

Algorithm steps:

1. Validate handle is initialized and resolution is a legal enum value
2. Read current 9-byte scratchpad to preserve TH/TL alarm bytes
3. Convert resolution enum to configuration register byte via [internal_ds18b20_resolution_to_config\(\)](#)
4. Write scratchpad with new config byte (TH/TL unchanged)
5. Update handle->resolution to reflect new setting

Parameters

in,out	<i>handle</i>	Sensor handle (must be initialized); resolution field updated on success
in	<i>resolution</i>	New resolution setting (k_ds18b20_resolutionnobit through k_ds18b20_resolution bit); all other values are invalid

Returns

rx_err_t Error code

Return values

<i>k_rx_ok</i>	Resolution updated in scratchpad and handle
<i>k_rx_err_null_ptr</i>	handle is nullptr
<i>k_rx_err_invalid_state</i>	handle not initialized
<i>k_rx_err_invalid_arg</i>	resolution is not a valid ds18b20_resolution_t value
<i>k_rx_err_crc_failure</i>	Scratchpad read CRC check failed
<i>k_rx_err_bus_error</i>	1-Wire bus communication failure during read or write

Precondition

handle must be initialized via [rx_ds18b20_init\(\)](#)

resolution must be one of k_ds18b20_resolutionnobit/10bit/11bit/12bit

Postcondition

handle->resolution reflects the newly configured value on k_rx_ok

DS18B20 scratchpad configuration register updated with new resolution bits

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Change is not persistent across power cycles unless [rx_ds18b20_save_config\(\)](#) is called

Example:

```
rx_err_t err = rx_ds18b20_set_resolution(&sensor, k_ds18b20_resolution_bit);
if (err == k_rx_ok) {
    // Save to EEPROM so it persists across power cycles
    rx_ds18b20_save_config(&sensor);
}
```

See also

[rx_ds18b20_get_resolution\(\)](#) Read back the currently configured resolution

[rx_ds18b20_save_config\(\)](#) Persist resolution change to EEPROM

[rx_ds18b20_get_conversion_time_ms\(\)](#) Get required conversion time for new resolution

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null/initialized, valid resolution), 2 postconditions

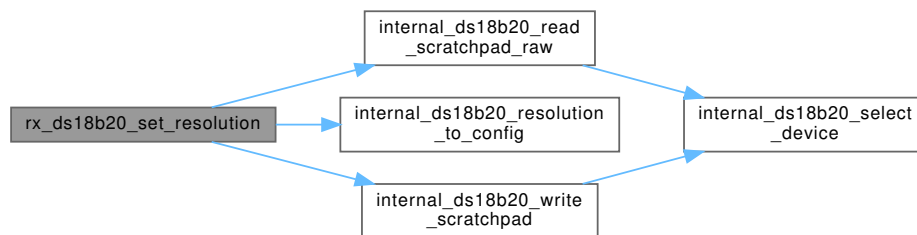
Definition at line 772 of file [rx_ds18b20.c](#).

```
00774 {
00775     CHECK_DS18B20_HANDLE(handle, s_tag);
00776
00777     if (resolution != k_ds18b20_resolution_bit && resolution != k_ds18b20_resolution_bit &&
00778         resolution != k_ds18b20_resolution_bit && resolution != k_ds18b20_resolution_bit) {
00779         rx_log_error(s_tag, "Invalid resolution value");
00780         return k_rx_err_invalid_arg;
00781     }
00782
00783     /* Read current scratchpad */
00784     uint8_t scratchpad[k_ds18b20_scratchpad_bytes];
00785     rx_err_t err = internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
00786     if (err != k_rx_ok) {
00787         return err;
00788     }
00789
00790     /* Generate new config byte */
00791     const uint8_t config = internal_ds18b20_resolution_to_config(resolution);
00792
00793     /* Write scratchpad with new config */
00794     ds18b20_scratchpad_write_t write_cfg;
00795     write_cfg.th = scratchpad[k_ds18b20_scratch_th_reg];
00796     write_cfg.tl = scratchpad[k_ds18b20_scratch_tl_reg];
00797     write_cfg.config = config;
00798     err = internal_ds18b20_write_scratchpad(handle, &write_cfg);
00799     if (err != k_rx_ok) {
00800         return err;
00801     }
00802
00803     /* Update handle */
00804     handle->resolution = resolution;
00805
00806     return k_rx_ok;
00807 }
```

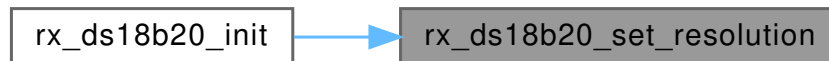
References [CHECK_DS18B20_HANDLE](#), [ds18b20_scratchpad_write_t::config](#), [internal_ds18b20_read_scratchpad_raw\(\)](#), [internal_ds18b20_resolution_to_config\(\)](#), [internal_ds18b20_write_scratchpad\(\)](#), [k_ds18b20_resolution_bit](#), [k_ds18b20_resolution_bit](#), [k_ds18b20_resolutionnobit](#), [k_ds18b20_scratch_th_reg](#), [k_ds18b20_scratch_tl_reg](#), [k_ds18b20_scratchpad_bytes](#), [rx_ds18b20_handle_t::resolution](#), [s_tag](#), [ds18b20_scratchpad_write_t::th](#), and [ds18b20_scratchpad_write_t::tl](#).

Referenced by [rx_ds18b20_init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.3.4.20 rx_ds18b20_trigger_conversion()

```
rx_err_t rx_ds18b20_trigger_conversion (
    const rx_ds18b20_handle_t * handle) [nodiscard]
```

Trigger temperature conversion on DS18B20 sensor.

Trigger temperature conversion.

Issues the Convert T (0x44) command over the 1-Wire bus to start an autonomous temperature measurement. The DS18B20 will perform the ADC conversion internally; the caller must wait the appropriate conversion time before reading results with [rx_ds18b20_read_temperature\(\)](#).

Algorithm steps:

1. Validate handle is initialized and non-null
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Convert T command byte (0x44) to the bus

Conversion time depends on resolution:

- 9-bit: 94 ms
- 10-bit: 188 ms
- 11-bit: 375 ms
- 12-bit: 750 ms

Parameters

in	<i>handle</i>	Sensor handle (must be initialized, non-null)
----	---------------	---

Returns

rx_err_t Error code

Return values

<i>k_rx_ok</i>	Conversion command issued successfully
----------------	--

<i>k_rx_err_null_ptr</i>	handle is nullptr
<i>k_rx_err_invalid_state</i>	handle not initialized or device not present on bus
<i>k_rx_err_bus_error</i>	1-Wire bus reset or write operation failed

Precondition

handle must be initialized via [rx_ds18b20_init\(\)](#)

DS18B20 device must be physically connected and powered on the bus

Postcondition

DS18B20 ADC conversion is in progress

Caller must wait [rx_ds18b20_get_conversion_time_ms\(\)](#) before reading

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Do not call [rx_ds18b20_read_temperature\(\)](#) before the conversion time elapses

Example:

```
rx_err_t err = rx_ds18b20_trigger_conversion(&sensor);
if (err == k_rx_ok) {
    tx_thread_sleep(rx_ds18b20_get_conversion_time_ms(&sensor));
    float temp_celsius = 0.0f;
    err = rx_ds18b20_read_temperature(&sensor, &temp_celsius);
}
```

See also

[rx_ds18b20_read_temperature\(\)](#) Read converted temperature in Celsius

[rx_ds18b20_get_conversion_time_ms\(\)](#) Get required wait time after conversion

[rx_ds18b20_set_resolution\(\)](#) Configure ADC resolution

Since

Version 1.0.0

NASA Power of 10 Compliance:

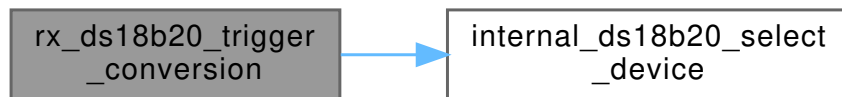
- Rule 5: 2 preconditions (initialized handle, device present), 2 postconditions

Definition at line 513 of file `rx_ds18b20.c`.

```
00514 {
00515     CHECK_DS18B20_HANDLE(handle, s_tag);
00516
00517     /* Reset and check presence */
00518     bool    presence = false;
00519     rx_err_t err      = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
00520     if (err != k_rx_ok || !presence) {
00521         rx_log_error(s_tag, "Device not present for conversion trigger");
00522         return k_rx_err_invalid_state;
00523     }
00524
00525     /* Select device */
00526     err = internal_ds18b20_select_device(handle);
00527     if (err != k_rx_ok) {
00528         return err;
00529     }
00530
00531     /* Send Convert T command */
00532     err = rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_convert_t);
00533     if (err != k_rx_ok) {
00534         rx_log_error(s_tag, "Failed to send conversion command");
00535         return err;
00536     }
00537
00538     return k_rx_ok;
00539 }
```

References `rx_ds18b20_handle_t::bus_manager`, `rx_ds18b20_handle_t::bus_name`, `CHECK_DS18B20_HANDLE`, `internal_ds18b20_select_device()`, `k_ds18b20_cmd_convert_t`, and `s_tag`.

Here is the call graph for this function:



6.4 rx_ds18b20.c

[Go to the documentation of this file.](#)

```
00001 /* lib/rx_ds18b20/src/rx_ds18b20.c */
00002
00003 /**
00004  * @file rx_ds18b20.c
00005  * @brief DS18B20 1-Wire Digital Temperature Sensor Driver Implementation
00006  *
00007  * @details
00008  * Complete DS18B20 driver implementation with hardware abstraction via bus manager.
00009  * Supports all DS18B20 features: temperature measurement, resolution configuration,
00010  * ROM matching for multi-drop buses, and non-volatile EEPROM storage.
00011  *
00012  * ## Architecture
00013  *
00014  * **Dependency Injection Pattern:**
00015  * - Uses `rx_bus_manager_t` interface for 1-Wire communication
00016  * - Enables unit testing with mock bus implementations
00017  * - Zero direct hardware dependencies
00018  *
00019  * **Communication Flow:**
00020  * ```
00021  * [rx_ds18b20] -> [rx_bus_manager] -> [rx_bus_onewire] -> [GPIO HAL]
00022  * ```
00023  *
00024  * **Thread Safety:**
00025  * - NOT thread-safe - caller must provide synchronization
00026  * - Multiple handles can coexist (different sensors)
00027  * - Concurrent access to same handle causes race conditions

```

```

00028 *
00029 * ## NASA Power of 10 Compliance
00030 *
00031 * | Rule | Status | Implementation |
00032 * |-----|-----|-----|
00033 * | 1. Simple control flow | [OK] | No goto, no recursion |
00034 * | 2. Fixed loop bounds | [OK] | All loops bounded by enum constants |
00035 * | 3. No dynamic allocation | [OK] | Zero malloc/free |
00036 * | 4. Short functions | [OK] | Max 60 lines per function |
00037 * | 5. Assertions | [OK] | Min 2 checks per function (nullptr, state) |
00038 * | 6. Data scope | [OK] | Variables declared at smallest scope |
00039 * | 7. Check returns | [OK] | All function returns validated |
00040 * | 8. Limit preprocessor | [OK] | Typed enums only, no magic numbers |
00041 * | 9. Pointer restrictions | [OK] | Single-level dereferencing |
00042 * | 10. Compile warnings | [OK] | -Wall -Wextra -Werror |
00043 *
00044 * ## SOLID Principles
00045 *
00046 * **Single Responsibility:**
00047 * - One purpose: DS18B20 sensor communication
00048 * - Bus operations delegated to `rx_bus_onewire`
00049 * - CRC validation delegated to `rx_crc`
00050 *
00051 * **Open/Closed:**
00052 * - Extensible via bus manager interface
00053 * - New bus types added without modifying this driver
00054 *
00055 * **Liskov Substitution:**
00056 * - Any `rx_bus_manager_t` implementation works
00057 * - Mock bus substitutes for testing
00058 *
00059 * **Interface Segregation:**
00060 * - Clean API with focused functions
00061 * - Separate read/write/config operations
00062 *
00063 * **Dependency Inversion:**
00064 * - Depends on `rx_bus_manager_t` abstraction
00065 * - Bus manager injected via config structure
00066 *
00067 * ## Performance
00068 *
00069 * **Typical Operation Times:**
00070 * - Initialization: ~100ms (bus init + scratchpad read)
00071 * - Trigger conversion: ~2ms (command send)
00072 * - Read temperature: ~5ms (scratchpad read + CRC check)
00073 * - Resolution change: ~10ms (read + modify + write scratchpad)
00074 *
00075 * **Conversion Times (sensor internal):**
00076 * - 9-bit: 94ms
00077 * - 10-bit: 188ms
00078 * - 11-bit: 375ms
00079 * - 12-bit: 750ms (default)
00080 *
00081 * ## Memory Layout
00082 *
00083 * **DS18B20 Scratchpad Structure (9 bytes):**
00084 * ``
00085 * Byte | Field | Description
00086 * -----|-----|-----
00087 * 0 | Temp LSB | Temperature low byte (1/16degC units)
00088 * 1 | Temp MSB | Temperature high byte (sign-extended)
00089 * 2 | TH Register | High alarm threshold (user-defined)
00090 * 3 | TL Register | Low alarm threshold (user-defined)
00091 * 4 | Config | Resolution bits R1:R0 (bits 6:5)
00092 * 5 | Reserved | 0xFF (always)
00093 * 6 | Reserved | Factory programmed (implementation-specific)
00094 * 7 | Reserved | 0x10 (count remaining, legacy)
00095 * 8 | CRC | CRC-8/MAXIM checksum
00096 * ``
00097 *
00098 * **Temperature Data Format:**
00099 * ``
00100 * 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
00101 * +---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
00102 * | S| S| S| S| S| 2^6| 2^5| 2^4| 2^3| 2^2| 2^1| 2^0|2^-1|2^-2|2^-3|2^-4|
00103 * +---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
00104 * Sign extend Integer part Fractional (1/16degC)
00105 * ``
00106 *
00107 * **Resolution vs Temperature Bits:**
00108 * - 9-bit: Bits 15:3 valid, bits 2:0 undefined (mask 0xFFF8)
00109 * - 10-bit: Bits 15:2 valid, bits 1:0 undefined (mask 0xFFFC)
00110 * - 11-bit: Bits 15:1 valid, bit 0 undefined (mask 0xFFFE)
00111 * - 12-bit: All bits valid (mask 0xFFFF)
00112 *
00113 * ## Error Handling
00114 *

```

```

00115 * **Error Propagation Strategy:**
00116 * ``c
00117 * // Check bus errors -> return immediately
00118 * err = rx_bus_owewire_reset(...);
00119 * if (err != k_rx_ok) return err;
00120 *
00121 * // Check CRC -> return k_rx_err_crc_mismatch
00122 * if (crc_calc != crc_device) return k_rx_err_crc_mismatch;
00123 *
00124 * // Check range -> return k_rx_err_out_of_range
00125 * if (temp < MIN || temp > MAX) return k_rx_err_out_of_range;
00126 * ``
00127 *
00128 * **Common Error Codes:**
00129 * - `k_rx_err_null_ptr` - nullptr handle/config/output pointer
00130 * - `k_rx_err_invalid_state` - Sensor not initialized or not present
00131 * - `k_rx_err_invalid_arg` - Invalid resolution or ROM family code
00132 * - `k_rx_err_crc_mismatch` - Scratchpad CRC validation failed
00133 * - `k_rx_err_out_of_range` - Temperature outside -55degC to +125degC
00134 *
00135 * @par Example: Basic Temperature Reading
00136 * @code
00137 * rx_ds18b20_handle_t sensor;
00138 * rx_ds18b20_config_t config = {
00139 *     .bus_manager = &g_bus_manager,
00140 *     .bus_name = "onewire0",
00141 *     .resolution = k_ds18b20_resolution_bit,
00142 *     .use_rom_matching = false // Single sensor
00143 * };
00144 *
00145 * // 1. Initialize
00146 * rx_err_t err = rx_ds18b20_init(&sensor, &config);
00147 * if (err != k_rx_ok) handle_error(err);
00148 *
00149 * // 2. Trigger conversion
00150 * err = rx_ds18b20_trigger_conversion(&sensor);
00151 * if (err != k_rx_ok) handle_error(err);
00152 *
00153 * // 3. Wait for conversion (12-bit = 750ms)
00154 * delay_ms(rx_ds18b20_get_conversion_time_ms(&sensor));
00155 *
00156 * // 4. Read temperature
00157 * float temp_c = 0.0f;
00158 * err = rx_ds18b20_read_temperature(&sensor, &temp_c);
00159 * if (err == k_rx_ok) {
00160 *     printf("Temperature: %.2fdegC\n", temp_c);
00161 * }
00162 *
00163 * // 5. Cleanup
00164 * rx_ds18b20_deinit(&sensor);
00165 * @endcode
00166 *
00167 * @par Example: Multi-Drop Bus with ROM Matching
00168 * @code
00169 * // Read ROM codes first (use search algorithm or known values)
00170 * uint8_t sensor1_rom[8] = {0x28, 0x12, 0x34, 0x56, 0x78, 0x9A, 0xBC, 0xDE};
00171 * uint8_t sensor2_rom[8] = {0x28, 0xFE, 0xDC, 0xBA, 0x98, 0x76, 0x54, 0x32};
00172 *
00173 * rx_ds18b20_handle_t sensor1, sensor2;
00174 * rx_ds18b20_config_t config1 = {
00175 *     .bus_manager = &g_bus_manager,
00176 *     .bus_name = "onewire0",
00177 *     .resolution = k_ds18b20_resolution_bit,
00178 *     .use_rom_matching = true,
00179 *     .rom = {0x28, 0x12, 0x34, 0x56, 0x78, 0x9A, 0xBC, 0xDE}
00180 * };
00181 *
00182 * // Initialize both sensors on same bus
00183 * rx_ds18b20_init(&sensor1, &config1);
00184 * config1.rom = sensor2_rom; // Reuse config, change ROM
00185 * rx_ds18b20_init(&sensor2, &config1);
00186 *
00187 * // Read from specific sensor
00188 * float temp1 = 0.0f;
00189 * rx_ds18b20_trigger_conversion(&sensor1);
00190 * delay_ms(750);
00191 * rx_ds18b20_read_temperature(&sensor1, &temp1);
00192 * @endcode
00193 *
00194 * @see rx_ds18b20.h
00195 * @see rx_bus_owewire.h
00196 * @see rx_crc.h
00197 *
00198 * @date 2026-01-05
00199 * @copyright Copyright (c) 2026 STAR Project
00200 */
00201

```

```

00202 #include "rx_ds18b20.h"
00203
00204 #include <math.h>
00205 #include <string.h>
00206
00207 #include "rx_bus_onewire.h"
00208 #include "rx_check.h"
00209 #include "rx_crc.h"
00210 #include "rx_log.h"
00211
00212 /**
00213  * @defgroup ds18b20_internal_constants Internal Constants
00214  * @{
00215  */
00216
00217 /** @brief Logging tag for DS18B20 driver */
00218 static const char s_tag[] = "DS18B20";
00219
00220 /** @brief DS18B20 family code (byte 0 of ROM) */
00221 static const uint8_t s_ds18b20_family_code = 0x28U;
00222
00223 /**
00224  * @brief Temperature conversion divisor (raw to Celsius)
00225  * @details DS18B20 stores temperature in 1/16degC units. Divide by 16 to get Celsius.
00226  */
00227 static const float s_temp_conversion_divisor = 16.0f;
00228
00229 /**
00230  * @brief Reserved byte expected value in scratchpad
00231  * @details Byte 5 of scratchpad should always be 0xFF per datasheet.
00232  */
00233 static const uint8_t s_ds18b20_reserved_byte_value = 0xFFU;
00234
00235 /**
00236  * @brief Invalid conversion time sentinel
00237  * @details Returned by rx_ds18b20_get_conversion_time_ms() on error.
00238  */
00239 static const uint32_t s_ds18b20_conversion_time_invalid_u32 = 0U;
00240
00241 /** @} */
00242
00243 /*
=====
00244  * Internal Validation Macros
00245  *
=====
00246  */
00247
00248 /**
00249  * @def CHECK_DS18B20_HANDLE
00250  * @brief Validate DS18B20 handle pointer and initialization state
00251  *
00252  * @details
00253  * Composite validation macro that checks both nullptr and initialization.
00254  * Used at the start of all public API functions to enforce preconditions.
00255  *
00256  * **Checks Performed:**
00257  * 1. Handle is not nullptr (via RX_CHECK_NULL_PTR)
00258  * 2. Handle is initialized (handle->initialized == true)
00259  *
00260  * **Error Returns:**
00261  * - `k_rx_err_null_ptr` if handle is nullptr
00262  * - `k_rx_err_invalid_state` if not initialized
00263  *
00264  * @param[in] handle Handle to validate (type: rx_ds18b20_handle_t*)
00265  * @param[in] tag Logging tag for error messages (type: const char*)
00266  *
00267  * @par Example:
00268  * @code
00269  * rx_err_t rx_ds18b20_read_temperature(rx_ds18b20_handle_t* handle, float* temp)
00270  * {
00271  *     CHECK_DS18B20_HANDLE(handle, s_tag); // Early return on error
00272  *     // ... function body ...
00273  * }
00274  * @endcode
00275  *
00276  * @note This is a statement macro (uses do-while(0) pattern) - requires semicolon.
00277  * @warning Do not use in expressions - this macro contains early returns.
00278  *
00279  * @see RX_CHECK_NULL_PTR
00280  */
00281 #define CHECK_DS18B20_HANDLE(handle, tag)
00282 do {
00283     RX_CHECK_NULL_PTR(handle, tag, "handle is nullptr");
00284     if (!(handle->initialized)) {
00285         rx_log_error(tag, "DS18B20 not initialized");
00286         return k_rx_err_invalid_state;
00287     }
00288 } while(0)

```

```

00287     }
00288 } while (0)
00289
00290 /*
=====
00291 * Internal Helper Functions
00292 *
=====
00293 */
00294
00295 /**
00296  * @struct ds18b20_scratchpad_write_t
00297  * @brief Scratchpad write parameters for TH, TL, and config registers
00298  *
00299  * @details
00300  * Internal structure for writing DS18B20 scratchpad registers.
00301  * DS18B20 Write Scratchpad command (0x4E) writes exactly 3 bytes:
00302  * TH alarm, TL alarm, and configuration register.
00303  *
00304  * **Memory Layout (3 bytes total):**
00305  * ```
00306  * Offset | Field | Description
00307  * -----|-----|-----
00308  * 0      | th    | High temperature alarm threshold (-55 to +125degC)
00309  * 1      | tl    | Low temperature alarm threshold (-55 to +125degC)
00310  * 2      | config| Resolution bits R1:R0 at bits 6:5
00311  * ```
00312  *
00313  * **Configuration Register Format:**
00314  * ```
00315  * Bit 7 | Bit 6 | Bit 5 | Bit 4 | Bit 3 | Bit 2 | Bit 1 | Bit 0
00316  * -----|-----|-----|-----|-----|-----|-----|-----
00317  * 0      | R1    | R0    | 1      | 1      | 1      | 1      | 1
00318  * +-----+-----+
00319  *          Resolution bits
00320  * ```
00321  *
00322  * **Resolution Encoding:**
00323  * - R1=0, R0=0: 9-bit (0.5degC, 93.75ms)
00324  * - R1=0, R0=1: 10-bit (0.25degC, 187.5ms)
00325  * - R1=1, R0=0: 11-bit (0.125degC, 375ms)
00326  * - R1=1, R0=1: 12-bit (0.0625degC, 750ms)
00327  *
00328  * @note This structure is for internal use only - not exposed in public API.
00329  * @see internal_ds18b20_write_scratchpad()
00330  * @see internal_ds18b20_resolution_to_config()
00331  */
00332 typedef struct {
00333     uint8_t th; /*< High alarm threshold register (signed, stored as uint8_t) */
00334     uint8_t tl; /*< Low alarm threshold register (signed, stored as uint8_t) */
00335     uint8_t config; /*< Configuration register (bits 6:5 = resolution) */
00336 } ds18b20_scratchpad_write_t;
00337
00338 static rx_err_t internal_ds18b20_validate_config(const rx_ds18b20_config_t* config,
00339                                                  const rx_ds18b20_handle_t* handle);
00340 static rx_err_t internal_ds18b20_verify_device_presence(rx_ds18b20_handle_t* handle);
00341 static rx_err_t internal_ds18b20_select_device(const rx_ds18b20_handle_t* handle);
00342 static rx_err_t
00343     internal_ds18b20_read_scratchpad_raw(const rx_ds18b20_handle_t* handle,
00344                                          uint8_t scratchpad[k_ds18b20_scratchpad_bytes]);
00345 static rx_err_t internal_ds18b20_write_scratchpad(const rx_ds18b20_handle_t* handle,
00346                                                  const ds18b20_scratchpad_write_t* scratchpad);
00347 static uint8_t internal_ds18b20_resolution_to_config(const ds18b20_resolution_t resolution);
00348 static uint16_t internal_ds18b20_get_temp_mask(ds18b20_resolution_t resolution);
00349 static float internal_ds18b20_raw_to_celsius(const int16_t raw_temp);
00350
00351 /*
=====
00352 * Public API Implementation
00353 *
=====
00354 */
00355
00356 rx_err_t rx_ds18b20_init(rx_ds18b20_handle_t* handle, const rx_ds18b20_config_t* config)
00357 {
00358     /* Validate inputs */
00359     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00360
00361     rx_err_t err = internal_ds18b20_validate_config(config, handle);
00362     if (err != k_rx_ok) {
00363         return err;
00364     }
00365
00366     /* Initialize handle */
00367     *handle = (rx_ds18b20_handle_t){0};
00368     handle->bus_manager = config->bus_manager;
00369     handle->bus_name = config->bus_name;

```



```

00370 handle->resolution      = config->resolution;
00371 handle->use_rom_matching = config->use_rom_matching;
00372
00373 if (config->use_rom_matching) {
00374     memcpy(handle->rom, config->rom, k_onewire_rom_bytes);
00375 }
00376
00377 /* Verify device presence */
00378 err = internal_ds18b20_verify_device_presence(handle);
00379 if (err != k_rx_ok) {
00380     return err;
00381 }
00382
00383 /* Configure resolution */
00384 handle->initialized = true;
00385 err = rx_ds18b20_set_resolution(handle, config->resolution);
00386 if (err != k_rx_ok) {
00387     handle->initialized = false;
00388     return err;
00389 }
00390
00391 rx_log_info(s_tag, "DS18B20 initialized successfully");
00392 return k_rx_ok;
00393 }
00394
00395 /**
00396  * @brief Deinitialize DS18B20 sensor handle and release all resources
00397  *
00398  * @details
00399  * Tears down a previously initialized DS18B20 handle by zeroing all internal
00400  * state via memset. After this call the handle is safe to reinitialize with
00401  * rx_ds18b20_init().
00402  *
00403  * Algorithm steps:
00404  * 1. Validate handle is not nullptr
00405  * 2. Verify handle is currently initialized
00406  * 3. Clear the entire handle structure with memset (zeros all fields)
00407  * 4. Log deinitialization success
00408  *
00409  * @param[in,out] handle Sensor handle to deinitialize (must be initialized, non-null)
00410  *
00411  * @return rx_err_t Error code
00412  * @retval k_rx_ok Handle successfully cleared
00413  * @retval k_rx_err_null_ptr handle pointer is nullptr
00414  * @retval k_rx_err_invalid_state handle was not previously initialized
00415  *
00416  * @pre handle must be a valid non-null pointer
00417  * @pre handle must have been successfully initialized via rx_ds18b20_init()
00418  * @post All handle fields are zeroed (initialized flag is false)
00419  * @post handle may be safely reused with rx_ds18b20_init()
00420  *
00421  * @note Not thread-safe; caller must ensure no concurrent access to handle
00422  *
00423  * @par Example:
00424  * @code
00425  * rx_ds18b20_handle_t sensor;
00426  * rx_ds18b20_init(&sensor, &config);
00427  * // ... use sensor ...
00428  * rx_err_t err = rx_ds18b20_deinit(&sensor);
00429  * if (err != k_rx_ok) {
00430  *     rx_log_error("app", "Failed to deinit DS18B20");
00431  * }
00432  * @endcode
00433  *
00434  * @see rx_ds18b20_init() Initialize the handle before use
00435  *
00436  * @since Version 1.0.0
00437  *
00438  * @par NASA Power of 10 Compliance:
00439  * - Rule 5: 2 preconditions (null check, initialized check), 2 postconditions
00440  */
00441 rx_err_t rx_ds18b20_deinit(rx_ds18b20_handle_t* handle)
00442 {
00443     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00444
00445     if (!handle->initialized) {
00446         rx_log_warn(s_tag, "DS18B20 not initialized");
00447         return k_rx_err_invalid_state;
00448     }
00449
00450     /* Clear handle */
00451     *handle = (rx_ds18b20_handle_t){0};
00452
00453     rx_log_info(s_tag, "DS18B20 deinitialized");
00454     return k_rx_ok;
00455 }
00456

```

```

00457 /**
00458  * @brief Trigger temperature conversion on DS18B20 sensor
00459  *
00460  * @details
00461  * Issues the Convert T (0x44) command over the 1-Wire bus to start an
00462  * autonomous temperature measurement. The DS18B20 will perform the ADC
00463  * conversion internally; the caller must wait the appropriate conversion
00464  * time before reading results with rx_ds18b20_read_temperature().
00465  *
00466  * Algorithm steps:
00467  * 1. Validate handle is initialized and non-null
00468  * 2. Issue 1-Wire bus reset and verify device presence pulse
00469  * 3. Select the target device (Match ROM or Skip ROM depending on config)
00470  * 4. Write the Convert T command byte (0x44) to the bus
00471  *
00472  * Conversion time depends on resolution:
00473  * - 9-bit: 94 ms
00474  * - 10-bit: 188 ms
00475  * - 11-bit: 375 ms
00476  * - 12-bit: 750 ms
00477  *
00478  * @param[in] handle Sensor handle (must be initialized, non-null)
00479  *
00480  * @return rx_err_t Error code
00481  * @retval k_rx_ok Conversion command issued successfully
00482  * @retval k_rx_err_null_ptr handle is nullptr
00483  * @retval k_rx_err_invalid_state handle not initialized or device not present on bus
00484  * @retval k_rx_err_bus_error 1-Wire bus reset or write operation failed
00485  *
00486  * @pre handle must be initialized via rx_ds18b20_init()
00487  * @pre DS18B20 device must be physically connected and powered on the bus
00488  * @post DS18B20 ADC conversion is in progress
00489  * @post Caller must wait rx_ds18b20_get_conversion_time_ms() before reading
00490  *
00491  * @note Not thread-safe; caller must serialize access to the same handle
00492  * @warning Do not call rx_ds18b20_read_temperature() before the conversion time elapses
00493  *
00494  * @par Example:
00495  * @code
00496  * rx_err_t err = rx_ds18b20_trigger_conversion(&sensor);
00497  * if (err == k_rx_ok) {
00498  *     tx_thread_sleep(rx_ds18b20_get_conversion_time_ms(&sensor));
00499  *     float temp_celsius = 0.0f;
00500  *     err = rx_ds18b20_read_temperature(&sensor, &temp_celsius);
00501  * }
00502  * @endcode
00503  *
00504  * @see rx_ds18b20_read_temperature() Read converted temperature in Celsius
00505  * @see rx_ds18b20_get_conversion_time_ms() Get required wait time after conversion
00506  * @see rx_ds18b20_set_resolution() Configure ADC resolution
00507  *
00508  * @since Version 1.0.0
00509  *
00510  * @par NASA Power of 10 Compliance:
00511  * - Rule 5: 2 preconditions (initialized handle, device present), 2 postconditions
00512  */
00513 rx_err_t rx_ds18b20_trigger_conversion(const rx_ds18b20_handle_t* handle)
00514 {
00515     CHECK_DS18B20_HANDLE(handle, s_tag);
00516
00517     /* Reset and check presence */
00518     bool presence = false;
00519     rx_err_t err = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
00520     if (err != k_rx_ok || !presence) {
00521         rx_log_error(s_tag, "Device not present for conversion trigger");
00522         return k_rx_err_invalid_state;
00523     }
00524
00525     /* Select device */
00526     err = internal_ds18b20_select_device(handle);
00527     if (err != k_rx_ok) {
00528         return err;
00529     }
00530
00531     /* Send Convert T command */
00532     err = rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_convert_t);
00533     if (err != k_rx_ok) {
00534         rx_log_error(s_tag, "Failed to send conversion command");
00535         return err;
00536     }
00537
00538     return k_rx_ok;
00539 }
00540
00541 /**
00542  * @brief Read temperature from DS18B20 sensor in degrees Celsius
00543  *

```

```

00544 * @details
00545 * Reads the full 9-byte scratchpad from the DS18B20, extracts the 16-bit raw
00546 * temperature value, applies the resolution mask to clear undefined low-order
00547 * bits, validates the result is within the sensor's physical range, and
00548 * converts to a floating-point Celsius value using the DS18B20 fixed-point
00549 * encoding (1 LSB = 1/16 degC).
00550 *
00551 * Algorithm steps:
00552 * 1. Validate handle and output pointer are non-null
00553 * 2. Call rx_ds18b20_read_temperature_raw() to obtain masked raw value
00554 * 3. Convert raw signed 16-bit value to float: celsius = raw / 16.0f
00555 * 4. Write result to *temperature_celsius
00556 *
00557 * @param[in,out] handle Sensor handle (must be initialized)
00558 * @param[out] temperature_celsius Converted temperature in degrees Celsius,
00559 *         valid range [-55.0, +125.0]; undefined on error
00560 *
00561 * @return rx_err_t Error code
00562 * @retval k_rx_ok Temperature read and converted successfully
00563 * @retval k_rx_err_null_ptr handle or temperature_celsius is nullptr
00564 * @retval k_rx_err_invalid_state handle not initialized
00565 * @retval k_rx_err_out_of_range raw temperature outside [-55degC, +125degC]
00566 * @retval k_rx_err_crc_failure Scratchpad CRC mismatch
00567 * @retval k_rx_err_bus_error 1-Wire bus communication failure
00568 *
00569 * @pre handle must be initialized via rx_ds18b20_init()
00570 * @pre rx_ds18b20_trigger_conversion() must have been called and conversion time elapsed
00571 * @post *temperature_celsius contains valid temperature in [-55.0, +125.0] on k_rx_ok
00572 * @post handle->stats.read_count incremented (via internal_ds18b20_read_scratchpad_raw)
00573 *
00574 * @note Not thread-safe; caller must serialize access to the same handle
00575 * @warning Returns stale data if conversion time has not elapsed after trigger
00576 *
00577 * @par Example:
00578 * @code
00579 * float temp_celsius = 0.0f;
00580 * rx_err_t err = rx_ds18b20_read_temperature(&sensor, &temp_celsius);
00581 * if (err == k_rx_ok) {
00582 *     rx_log_info("app", "Temperature: %.4f C", (double)temp_celsius);
00583 * }
00584 * @endcode
00585 *
00586 * @see rx_ds18b20_trigger_conversion() Must be called before reading
00587 * @see rx_ds18b20_read_temperature_raw() Read the raw 16-bit value
00588 * @see rx_ds18b20_get_conversion_time_ms() Determine required wait time
00589 *
00590 * @since Version 1.0.0
00591 *
00592 * @par NASA Power of 10 Compliance:
00593 * - Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions
00594 */
00595 rx_err_t rx_ds18b20_read_temperature(rx_ds18b20_handle_t* handle, float* temperature_celsius)
00596 {
00597     CHECK_DS18B20_HANDLE(handle, s_tag);
00598     RX_CHECK_NULL_PTR(temperature_celsius, s_tag, "temperature_celsius is nullptr");
00599
00600     /* Read raw temperature */
00601     int16_t raw_temp = 0;
00602     rx_err_t err = rx_ds18b20_read_temperature_raw(handle, &raw_temp);
00603     if (err != k_rx_ok) {
00604         return err;
00605     }
00606
00607     /* Convert to Celsius */
00608     *temperature_celsius = internal_ds18b20_raw_to_celsius(raw_temp);
00609
00610     return k_rx_ok;
00611 }
00612 /**
00613 * @brief Read raw temperature value from DS18B20 sensor in 1/16 degree Celsius units
00614 *
00615 * @details
00616 * Reads the 9-byte DS18B20 scratchpad register, extracts the two temperature
00617 * bytes (LSB at offset 0, MSB at offset 1), combines them into a 16-bit
00618 * unsigned value, applies the resolution-dependent mask to clear undefined
00619 * low-order bits, casts to signed int16_t, and range-validates the result.
00620 *
00621 * The DS18B20 temperature encoding uses a signed fixed-point format where
00622 * 1 LSB = 1/16 degC (12-bit mode). The valid raw range is:
00623 * - Minimum: -55 degC = 0xFC90 (int16: -880)
00624 * - Maximum: +125 degC = 0x07D0 (int16: +2000)
00625 *
00626 * Resolution masks (clears undefined bits in lower-resolution modes):
00627 * - 9-bit: 0xFFF8 (3 undefined bits)
00628 * - 10-bit: 0xFFFC (2 undefined bits)
00629 * - 11-bit: 0xFFFE (1 undefined bit)

```

```

00631 * - 12-bit: 0xFFFF (no undefined bits)
00632 *
00633 * Algorithm steps:
00634 * 1. Validate handle and output pointer are non-null and handle is initialized
00635 * 2. Read raw scratchpad bytes via internal_ds18b20_read_scratchpad_raw()
00636 * 3. Combine temperature LSB and MSB bytes into uint16_t
00637 * 4. Apply resolution mask from internal_ds18b20_get_temp_mask()
00638 * 5. Cast to int16_t and validate range [k_ds18b20_temp_min_raw, k_ds18b20_temp_max_raw]
00639 * 6. Write validated result to *raw_temp
00640 *
00641 * @param[in,out] handle Sensor handle (must be initialized)
00642 * @param[out] raw_temp Signed 16-bit raw temperature in 1/16 degC units,
00643 *         valid range [k_ds18b20_temp_min_raw, k_ds18b20_temp_max_raw];
00644 *         undefined on error
00645 *
00646 * @return rx_err_t Error code
00647 * @retval k_rx_ok Raw temperature read, masked, and validated successfully
00648 * @retval k_rx_err_null_ptr handle or raw_temp is nullptr
00649 * @retval k_rx_err_invalid_state handle not initialized
00650 * @retval k_rx_err_out_of_range raw value outside physical sensor range
00651 * @retval k_rx_err_crc_failure Scratchpad CRC check failed
00652 * @retval k_rx_err_bus_error 1-Wire bus communication failure
00653 *
00654 * @pre handle must be initialized via rx_ds18b20_init()
00655 * @pre rx_ds18b20_trigger_conversion() must have been called and conversion time elapsed
00656 * @post *raw_temp is in [k_ds18b20_temp_min_raw, k_ds18b20_temp_max_raw] on k_rx_ok
00657 * @post Resolution mask has been applied; undefined bits are cleared
00658 *
00659 * @note Not thread-safe; caller must serialize access to the same handle
00660 * @warning Returns stale data if conversion time has not elapsed after trigger
00661 *
00662 * @par Example:
00663 * @code
00664 * int16_t raw = 0;
00665 * rx_err_t err = rx_ds18b20_read_temperature_raw(&sensor, &raw);
00666 * if (err == k_rx_ok) {
00667 *     float celsius = (float)raw / 16.0f;
00668 * }
00669 * @endcode
00670 *
00671 * @see rx_ds18b20_read_temperature() Convenience wrapper returning float Celsius
00672 * @see rx_ds18b20_trigger_conversion() Must be called before reading
00673 * @see rx_ds18b20_set_resolution() Configure ADC resolution and mask
00674 *
00675 * @since Version 1.0.0
00676 *
00677 * @par NASA Power of 10 Compliance:
00678 * - Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions (range, mask)
00679 */
00680 rx_err_t rx_ds18b20_read_temperature_raw(rx_ds18b20_handle_t* handle, int16_t* raw_temp)
00681 {
00682     CHECK_DS18B20_HANDLE(handle, s_tag);
00683     RX_CHECK_NULL_PTR(raw_temp, s_tag, "raw_temp is nullptr");
00684
00685     /* Read scratchpad */
00686     uint8_t scratchpad[k_ds18b20_scratchpad_bytes];
00687     rx_err_t err = internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
00688     if (err != k_rx_ok) {
00689         return err;
00690     }
00691
00692     /* Extract temperature (LSB + MSB) */
00693     uint16_t temp = (uint16_t)scratchpad[k_ds18b20_scratch_temp_lsb];
00694     temp |= ((uint16_t)scratchpad[k_ds18b20_scratch_temp_msb]) « k_ds18b20_shift_byte;
00695
00696     /* Apply resolution mask to clear undefined bits */
00697     const uint16_t mask = internal_ds18b20_get_temp_mask(handle->resolution);
00698     temp &= mask;
00699
00700     /* Post-condition: Validate temperature is within sensor range */
00701     *raw_temp = (int16_t)temp;
00702     if (*raw_temp < k_ds18b20_temp_min_raw || *raw_temp > k_ds18b20_temp_max_raw) {
00703         rx_log_error(s_tag, "Temperature out of sensor range (-55degC to +125degC)");
00704         return k_rx_err_out_of_range;
00705     }
00706
00707     return k_rx_ok;
00708 }
00709 /**
00710 * @brief Set ADC measurement resolution on DS18B20 sensor
00711 *
00712 * @details
00713 * Updates the configuration register in the DS18B20 scratchpad to change
00714 * the ADC resolution. The existing TH and TL alarm threshold bytes are
00715 * preserved by reading the current scratchpad first, then writing back
00716 * only the updated configuration byte.
00717 */

```

```

00718 *
00719 * Higher resolution provides finer temperature steps but increases
00720 * conversion time:
00721 * - 9-bit: 0.5 degC steps, 94 ms conversion
00722 * - 10-bit: 0.25 degC steps, 188 ms conversion
00723 * - 11-bit: 0.125 degC steps, 375 ms conversion
00724 * - 12-bit: 0.0625 degC steps, 750 ms conversion
00725 *
00726 * Algorithm steps:
00727 * 1. Validate handle is initialized and resolution is a legal enum value
00728 * 2. Read current 9-byte scratchpad to preserve TH/TL alarm bytes
00729 * 3. Convert resolution enum to configuration register byte via
00730 *    internal_ds18b20_resolution_to_config()
00731 * 4. Write scratchpad with new config byte (TH/TL unchanged)
00732 * 5. Update handle->resolution to reflect new setting
00733 *
00734 * @param[in,out] handle Sensor handle (must be initialized); resolution field updated on success
00735 * @param[in] resolution New resolution setting (k_ds18b20_resolution bit through
00736 *    k_ds18b20_resolution bit); all other values are invalid
00737 *
00738 * @return rx_err_t Error code
00739 * @retval k_rx_ok Resolution updated in scratchpad and handle
00740 * @retval k_rx_err_null_ptr handle is nullptr
00741 * @retval k_rx_err_invalid_state handle not initialized
00742 * @retval k_rx_err_invalid_arg resolution is not a valid ds18b20_resolution_t value
00743 * @retval k_rx_err_crc_failure Scratchpad read CRC check failed
00744 * @retval k_rx_err_bus_error 1-Wire bus communication failure during read or write
00745 *
00746 * @pre handle must be initialized via rx_ds18b20_init()
00747 * @pre resolution must be one of k_ds18b20_resolution bit/10bit/11bit/12bit
00748 * @post handle->resolution reflects the newly configured value on k_rx_ok
00749 * @post DS18B20 scratchpad configuration register updated with new resolution bits
00750 *
00751 * @note Not thread-safe; caller must serialize access to the same handle
00752 * @warning Change is not persistent across power cycles unless rx_ds18b20_save_config() is called
00753 *
00754 * @par Example:
00755 * @code
00756 * rx_err_t err = rx_ds18b20_set_resolution(&sensor, k_ds18b20_resolution bit);
00757 * if (err == k_rx_ok) {
00758 *     // Save to EEPROM so it persists across power cycles
00759 *     rx_ds18b20_save_config(&sensor);
00760 * }
00761 * @endcode
00762 *
00763 * @see rx_ds18b20_get_resolution() Read back the currently configured resolution
00764 * @see rx_ds18b20_save_config() Persist resolution change to EEPROM
00765 * @see rx_ds18b20_get_conversion_time_ms() Get required conversion time for new resolution
00766 *
00767 * @since Version 1.0.0
00768 *
00769 * @par NASA Power of 10 Compliance:
00770 * - Rule 5: 2 preconditions (null/initialized, valid resolution), 2 postconditions
00771 */
00772 rx_err_t rx_ds18b20_set_resolution(rx_ds18b20_handle_t* handle,
00773                                   const ds18b20_resolution_t resolution)
00774 {
00775     CHECK_DS18B20_HANDLE(handle, s_tag);
00776
00777     if (resolution != k_ds18b20_resolution bit && resolution != k_ds18b20_resolution bit &&
00778         resolution != k_ds18b20_resolution bit && resolution != k_ds18b20_resolution bit) {
00779         rx_log_error(s_tag, "Invalid resolution value");
00780         return k_rx_err_invalid_arg;
00781     }
00782
00783     /* Read current scratchpad */
00784     uint8_t scratchpad[k_ds18b20_scratchpad_bytes];
00785     rx_err_t err = internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
00786     if (err != k_rx_ok) {
00787         return err;
00788     }
00789
00790     /* Generate new config byte */
00791     const uint8_t config = internal_ds18b20_resolution_to_config(resolution);
00792
00793     /* Write scratchpad with new config */
00794     ds18b20_scratchpad_write_t write_cfg;
00795     write_cfg.th = scratchpad[k_ds18b20_scratch_th_reg];
00796     write_cfg.tl = scratchpad[k_ds18b20_scratch_tl_reg];
00797     write_cfg.config = config;
00798     err = internal_ds18b20_write_scratchpad(handle, &write_cfg);
00799     if (err != k_rx_ok) {
00800         return err;
00801     }
00802
00803     /* Update handle */
00804     handle->resolution = resolution;

```

```

00805
00806     return k_rx_ok;
00807 }
00808
00809 /**
00810  * @brief Get the currently configured ADC resolution from DS18B20 handle
00811  *
00812  * @details
00813  * Returns the resolution value stored in the handle, which reflects the
00814  * last successfully applied resolution (set during initialization or via
00815  * rx_ds18b20_set_resolution()). This is a cached read from the handle
00816  * state; it does not issue a 1-Wire bus transaction.
00817  *
00818  * Algorithm steps:
00819  * 1. Validate handle and output pointer are non-null
00820  * 2. Verify handle is initialized
00821  * 3. Copy handle->resolution to *resolution
00822  *
00823  * @param[in] handle Sensor handle (must be initialized, non-null)
00824  * @param[out] resolution Receives the currently configured ds18b20_resolution_t value;
00825  *                          undefined on error
00826  *
00827  * @return rx_err_t Error code
00828  * @retval k_rx_ok Resolution successfully returned
00829  * @retval k_rx_err_null_ptr handle or resolution pointer is nullptr
00830  * @retval k_rx_err_invalid_state handle not initialized (CHECK_DS18B20_HANDLE fails)
00831  *
00832  * @pre handle must be initialized via rx_ds18b20_init()
00833  * @pre resolution must be a valid non-null pointer
00834  * @post *resolution contains the current resolution value on k_rx_ok
00835  * @post handle state is unchanged
00836  *
00837  * @note Not thread-safe; caller must ensure handle is not concurrently modified
00838  * @note Returns cached value from handle -- does not issue 1-Wire bus transaction
00839  *
00840  * @par Example:
00841  * @code
00842  * ds18b20_resolution_t res;
00843  * rx_err_t err = rx_ds18b20_get_resolution(&sensor, &res);
00844  * if (err == k_rx_ok) {
00845  *     uint32_t conv_ms = rx_ds18b20_get_conversion_time_ms(&sensor);
00846  *     rx_log_info("app", "Resolution %d, conv time %u ms", (int)res, (unsigned)conv_ms);
00847  * }
00848  * @endcode
00849  *
00850  * @see rx_ds18b20_set_resolution() Change the ADC resolution
00851  * @see rx_ds18b20_get_conversion_time_ms() Get conversion time for current resolution
00852  *
00853  * @since Version 1.0.0
00854  *
00855  * @par NASA Power of 10 Compliance:
00856  * - Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions
00857  */
00858 rx_err_t rx_ds18b20_get_resolution(const rx_ds18b20_handle_t* handle,
00859                                   ds18b20_resolution_t* resolution)
00860 {
00861     CHECK_DS18B20_HANDLE(handle, s_tag);
00862     RX_CHECK_NULL_PTR(resolution, s_tag, "resolution is nullptr");
00863     *resolution = handle->resolution;
00864     return k_rx_ok;
00865 }
00866
00867 /**
00868  * @brief Save current scratchpad configuration to DS18B20 EEPROM
00869  *
00870  * @details
00871  * Sends the Copy Scratchpad (0x48) command over the 1-Wire bus, which
00872  * instructs the DS18B20 to copy the TH alarm, TL alarm, and configuration
00873  * register bytes from volatile scratchpad RAM to non-volatile EEPROM. The
00874  * saved values persist across power cycles and are automatically restored
00875  * on the next power-up via the Recall E2 sequence.
00876  *
00877  * Algorithm steps:
00878  * 1. Validate handle is initialized and non-null
00879  * 2. Issue 1-Wire bus reset and verify device presence pulse
00880  * 3. Select the target device (Match ROM or Skip ROM depending on config)
00881  * 4. Write the Copy Scratchpad command byte (0x48)
00882  *
00883  * @param[in] handle Sensor handle (must be initialized, non-null)
00884  *
00885  * @return rx_err_t Error code
00886  * @retval k_rx_ok Copy Scratchpad command successfully issued
00887  * @retval k_rx_err_null_ptr handle is nullptr
00888  * @retval k_rx_err_invalid_state handle not initialized or device not present on bus
00889  * @retval k_rx_err_bus_error 1-Wire bus reset or write operation failed
00890  *
00891  */

```



```

00892 * @pre handle must be initialized via rx_ds18b20_init()
00893 * @pre DS18B20 device must be physically connected and externally powered
00894 *      (Copy Scratchpad is not guaranteed in parasitic power mode)
00895 * @post DS18B20 EEPROM contains current TH, TL, and configuration register values
00896 * @post Configuration will survive next power cycle
00897 *
00898 * @note Not thread-safe; caller must serialize access to the same handle
00899 * @warning Not reliable in parasitic power mode; use external power for EEPROM writes
00900 *
00901 * @par Example:
00902 * @code
00903 * // Configure resolution, then persist to EEPROM
00904 * rx_ds18b20_set_resolution(&sensor, k_ds18b20_resolution_bit);
00905 * rx_err_t err = rx_ds18b20_save_config(&sensor);
00906 * if (err != k_rx_ok) {
00907 *     rx_log_error("app", "Failed to save DS18B20 config to EEPROM");
00908 * }
00909 * @endcode
00910 *
00911 * @see rx_ds18b20_recall_config() Restore EEPROM contents to scratchpad
00912 * @see rx_ds18b20_set_resolution() Configure resolution before saving
00913 *
00914 * @since Version 1.0.0
00915 *
00916 * @par NASA Power of 10 Compliance:
00917 * - Rule 5: 2 preconditions (null/initialized, device present), 2 postconditions
00918 */
00919 rx_err_t rx_ds18b20_save_config(const rx_ds18b20_handle_t* handle)
00920 {
00921     CHECK_DS18B20_HANDLE(handle, s_tag);
00922
00923     /* Reset and check presence */
00924     bool presence = false;
00925     rx_err_t err = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
00926     if (err != k_rx_ok || !presence) {
00927         rx_log_error(s_tag, "Device not present for EEPROM save");
00928         return k_rx_err_invalid_state;
00929     }
00930
00931     /* Select device */
00932     err = internal_ds18b20_select_device(handle);
00933     if (err != k_rx_ok) {
00934         return err;
00935     }
00936
00937     /* Send Copy Scratchpad command */
00938     err =
00939         rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_copy_scratch);
00940     if (err != k_rx_ok) {
00941         rx_log_error(s_tag, "Failed to send copy scratchpad command");
00942         return err;
00943     }
00944
00945     return k_rx_ok;
00946 }
00947
00948 /**
00949 * @brief Recall configuration from DS18B20 EEPROM into scratchpad RAM
00950 *
00951 * @details
00952 * Sends the Recall E2 (0xB8) command over the 1-Wire bus, which instructs
00953 * the DS18B20 to copy the TH alarm, TL alarm, and configuration register
00954 * bytes from non-volatile EEPROM back into volatile scratchpad RAM. This
00955 * is the same operation the device performs automatically on power-up.
00956 *
00957 * Use this function to discard unsaved scratchpad changes and revert to the
00958 * last saved EEPROM state without cycling power.
00959 *
00960 * Algorithm steps:
00961 * 1. Validate handle is initialized and non-null
00962 * 2. Issue 1-Wire bus reset and verify device presence pulse
00963 * 3. Select the target device (Match ROM or Skip ROM depending on config)
00964 * 4. Write the Recall E2 command byte (0xB8)
00965 *
00966 * @param[in] handle Sensor handle (must be initialized, non-null)
00967 *
00968 * @return rx_err_t Error code
00969 * @retval k_rx_ok Recall E2 command successfully issued
00970 * @retval k_rx_err_null_ptr handle is nullptr
00971 * @retval k_rx_err_invalid_state handle not initialized or device not present on bus
00972 * @retval k_rx_err_bus_error 1-Wire bus reset or write operation failed
00973 *
00974 * @pre handle must be initialized via rx_ds18b20_init()
00975 * @pre DS18B20 device must be physically connected to the bus
00976 * @post DS18B20 scratchpad TH, TL, and configuration bytes reflect last EEPROM save
00977 * @post Any unsaved scratchpad changes since last rx_ds18b20_save_config() are discarded
00978 *

```

```

00979 * @note Not thread-safe; caller must serialize access to the same handle
00980 * @note The Recall E2 operation is performed automatically by the device on power-up
00981 *
00982 * @par Example:
00983 * @code
00984 * // Revert any unsaved configuration changes
00985 * rx_err_t err = rx_ds18b20_recall_config(&sensor);
00986 * if (err != k_rx_ok) {
00987 *     rx_log_error("app", "Failed to recall DS18B20 config from EEPROM");
00988 * }
00989 * @endcode
00990 *
00991 * @see rx_ds18b20_save_config() Persist scratchpad configuration to EEPROM
00992 * @see rx_ds18b20_set_resolution() Modify the scratchpad configuration
00993 *
00994 * @since Version 1.0.0
00995 *
00996 * @par NASA Power of 10 Compliance:
00997 * - Rule 5: 2 preconditions (null/initialized, device present), 2 postconditions
00998 */
00999 rx_err_t rx_ds18b20_recall_config(const rx_ds18b20_handle_t* handle)
01000 {
01001     CHECK_DS18B20_HANDLE(handle, s_tag);
01002
01003     /* Reset and check presence */
01004     bool presence = false;
01005     rx_err_t err = rx_bus_owewire_reset(handle->bus_manager, handle->bus_name, &presence);
01006     if (err != k_rx_ok || !presence) {
01007         rx_log_error(s_tag, "Device not present for EEPROM recall");
01008         return k_rx_err_invalid_state;
01009     }
01010
01011     /* Select device */
01012     err = internal_ds18b20_select_device(handle);
01013     if (err != k_rx_ok) {
01014         return err;
01015     }
01016
01017     /* Send Recall E2 command */
01018     err =
01019         rx_bus_owewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_recall_eeprom);
01020     if (err != k_rx_ok) {
01021         rx_log_error(s_tag, "Failed to send recall EEPROM command");
01022         return err;
01023     }
01024
01025     return k_rx_ok;
01026 }
01027
01028 /**
01029 * @brief Read DS18B20 power supply mode (external vs. parasitic)
01030 *
01031 * @details
01032 * Issues the Read Power Supply (0xB4) command over the 1-Wire bus and reads
01033 * back a single bit to determine whether the DS18B20 is operating on an
01034 * external voltage supply or in parasitic (bus-powered) mode.
01035 *
01036 * Per the DS18B20 datasheet: the sensor pulls the bus low for one read time
01037 * slot to indicate parasitic power mode (0), or releases the bus (1) to
01038 * indicate external power mode.
01039 *
01040 * Algorithm steps:
01041 * 1. Validate handle and output pointer are non-null; handle is initialized
01042 * 2. Issue 1-Wire bus reset and verify device presence pulse
01043 * 3. Select the target device (Match ROM or Skip ROM depending on config)
01044 * 4. Write the Read Power Supply command byte (0xB4)
01045 * 5. Read one bit from the bus (0 = parasitic, 1 = external)
01046 * 6. Write result to *external_power
01047 *
01048 * @param[in] handle Sensor handle (must be initialized, non-null)
01049 * @param[out] external_power Set to true if device uses external power supply,
01050 * false if device uses parasitic (bus-powered) mode; undefined on error
01051 *
01052 * @return rx_err_t Error code
01053 * @retval k_rx_ok Power mode successfully read
01054 * @retval k_rx_err_null_ptr handle or external_power pointer is nullptr
01055 * @retval k_rx_err_invalid_state handle not initialized or device not present on bus
01056 * @retval k_rx_err_bus_error 1-Wire bus reset, write, or read operation failed
01057 *
01058 * @pre handle must be initialized via rx_ds18b20_init()
01059 * @pre DS18B20 device must be physically connected to the bus
01060 * @post *external_power reflects the device power supply mode on k_rx_ok
01061 * @post handle state is unchanged
01062 *
01063 * @note Not thread-safe; caller must serialize access to the same handle
01064 * @warning Some operations (Copy Scratchpad) are unreliable in parasitic power mode
01065 */

```



```

01066 * @par Example:
01067 * @code
01068 * bool ext_power = false;
01069 * rx_err_t err = rx_ds18b20_read_power_mode(&sensor, &ext_power);
01070 * if (err == k_rx_ok && !ext_power) {
01071 *     rx_log_warn("app", "DS18B20 in parasitic mode: EEPROM writes may fail");
01072 * }
01073 * @endcode
01074 *
01075 * @see rx_ds18b20_save_config() EEPROM write (requires external power)
01076 * @see rx_ds18b20_init() Initialize sensor handle
01077 *
01078 * @since Version 1.0.0
01079 *
01080 * @par NASA Power of 10 Compliance:
01081 * - Rule 5: 2 preconditions (null checks/initialized, device present), 2 postconditions
01082 */
01083 rx_err_t rx_ds18b20_read_power_mode(const rx_ds18b20_handle_t* handle, bool* external_power)
01084 {
01085     CHECK_DS18B20_HANDLE(handle, s_tag);
01086     RX_CHECK_NULL_PTR(external_power, s_tag, "external_power is nullptr");
01087
01088     /* Reset and check presence */
01089     bool presence = false;
01090     rx_err_t err = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
01091     if (err != k_rx_ok || !presence) {
01092         rx_log_error(s_tag, "Device not present for power mode read");
01093         return k_rx_err_invalid_state;
01094     }
01095
01096     /* Select device */
01097     err = internal_ds18b20_select_device(handle);
01098     if (err != k_rx_ok) {
01099         return err;
01100     }
01101
01102     /* Send Read Power Supply command */
01103     err = rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_read_power);
01104     if (err != k_rx_ok) {
01105         rx_log_error(s_tag, "Failed to send read power command");
01106         return err;
01107     }
01108
01109     /* Read power bit (1 = external, 0 = parasitic) */
01110     bool power_bit = false;
01111     err = rx_bus_onewire_read_bit(handle->bus_manager, handle->bus_name, &power_bit);
01112     if (err != k_rx_ok) {
01113         rx_log_error(s_tag, "Failed to read power bit");
01114         return err;
01115     }
01116
01117     *external_power = power_bit;
01118     return k_rx_ok;
01119 }
01120
01121 /**
01122 * @brief Read the full 9-byte scratchpad register from DS18B20 sensor
01123 *
01124 * @details
01125 * Reads all nine bytes of the DS18B20 scratchpad memory and returns the raw
01126 * buffer to the caller. The scratchpad layout is:
01127 *
01128 * | Byte | Field          | Description          |
01129 * |-----|-----|-----|
01130 * | 0    | Temperature LSB | Low byte of temperature register |
01131 * | 1    | Temperature MSB | High byte of temperature register |
01132 * | 2    | TH Register     | High alarm threshold |
01133 * | 3    | TL Register     | Low alarm threshold  |
01134 * | 4    | Config Register | Resolution configuration byte |
01135 * | 5-7  | Reserved        | Reserved (0xFF, 0x10, 0x10) |
01136 * | 8    | CRC             | CRC-8 of bytes 0-7 |
01137 *
01138 * CRC is verified internally by internal_ds18b20_read_scratchpad_raw() before
01139 * the buffer is returned. This function is a thin public wrapper around the
01140 * internal helper.
01141 *
01142 * Algorithm steps:
01143 * 1. Validate handle and scratchpad buffer pointer are non-null
01144 * 2. Verify handle is initialized
01145 * 3. Delegate to internal_ds18b20_read_scratchpad_raw() which issues the
01146 *    1-Wire reset, Match/Skip ROM selection, Read Scratchpad (0xBE) command,
01147 *    reads 9 bytes, and verifies CRC-8
01148 *
01149 * @param[in,out] handle Sensor handle (must be initialized, non-null)
01150 * @param[out] scratchpad Buffer of exactly k_ds18b20_scratchpad_bytes (9) bytes
01151 *    to receive the raw scratchpad contents; undefined on error
01152 */

```

```

01153 * @return rx_err_t Error code
01154 * @retval k_rx_ok Scratchpad successfully read and CRC verified
01155 * @retval k_rx_err_null_ptr handle or scratchpad buffer is nullptr
01156 * @retval k_rx_err_invalid_state handle not initialized
01157 * @retval k_rx_err_crc_failure Scratchpad CRC-8 check failed (data corrupted)
01158 * @retval k_rx_err_bus_error 1-Wire bus communication failure
01159 *
01160 * @pre handle must be initialized via rx_ds18b20_init()
01161 * @pre scratchpad must point to a buffer of at least k_ds18b20_scratchpad_bytes bytes
01162 * @post scratchpad[0..8] contains valid data on k_rx_ok
01163 * @post CRC of scratchpad bytes 0-7 matches byte 8 on k_rx_ok
01164 *
01165 * @note Not thread-safe; caller must serialize access to the same handle
01166 * @note Use rx_ds18b20_read_temperature() for converted Celsius value
01167 *
01168 * @par Example:
01169 * @code
01170 * uint8_t raw[k_ds18b20_scratchpad_bytes];
01171 * rx_err_t err = rx_ds18b20_read_scratchpad(&sensor, raw);
01172 * if (err == k_rx_ok) {
01173 *     uint8_t config_reg = raw[4];
01174 * }
01175 * @endcode
01176 *
01177 * @see rx_ds18b20_read_temperature() High-level temperature read returning float Celsius
01178 * @see rx_ds18b20_read_temperature_raw() Read raw 16-bit temperature with masking
01179 *
01180 * @since Version 1.0.0
01181 *
01182 * @par NASA Power of 10 Compliance:
01183 * - Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions
01184 */
01185 rx_err_t rx_ds18b20_read_scratchpad(rx_ds18b20_handle_t* handle,
01186                                     uint8_t scratchpad[k_ds18b20_scratchpad_bytes])
01187 {
01188     CHECK_DS18B20_HANDLE(handle, s_tag);
01189     RX_CHECK_NULL_PTR(scratchpad, s_tag, "scratchpad is nullptr");
01190
01191     return internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
01192 }
01193
01194 /**
01195 * @brief Get temperature conversion time for current resolution
01196 *
01197 * Returns the required wait time after triggering conversion based on
01198 * the configured resolution (9-bit: 94ms, 10-bit: 188ms, 11-bit: 375ms, 12-bit: 750ms).
01199 *
01200 * @param[in] handle Sensor handle
01201 *
01202 * @return Conversion time in milliseconds, or invalid value if handle is nullptr/uninitialized
01203 */
01204 uint32_t rx_ds18b20_get_conversion_time_ms(const rx_ds18b20_handle_t* handle)
01205 {
01206     if (handle == nullptr || !handle->initialized) {
01207         return s_ds18b20_conversion_time_invalid_u32;
01208     }
01209
01210     switch (handle->resolution) {
01211         case k_ds18b20_resolution_bit:
01212             return k_ds18b20_conv_time_bit_ms;
01213         case k_ds18b20_resolution_bit:
01214             return k_ds18b20_conv_time_bit_ms;
01215         case k_ds18b20_resolution_bit:
01216             return k_ds18b20_conv_time_bit_ms;
01217         case k_ds18b20_resolution_bit:
01218             return k_ds18b20_conv_time_bit_ms;
01219         default:
01220             return s_ds18b20_conversion_time_invalid_u32;
01221     }
01222 }
01223
01224 /*
=====
01225 * Internal Helper Functions Implementation
01226 *
=====
01227 */
01228
01229 /**
01230 * @brief Validate DS18B20 configuration parameters before initialization
01231 *
01232 * @details
01233 * Comprehensive validation of configuration structure and handle state.
01234 * Called by rx_ds18b20_init() to enforce preconditions and prevent
01235 * misconfiguration.
01236 *
01237 * **Algorithm:**

```

```

01238 * 1. Check config pointer is not nullptr
01239 * 2. Check bus_manager pointer is not nullptr
01240 * 3. Check bus_name string is not nullptr
01241 * 4. Verify handle is not already initialized (prevent re-initialization)
01242 * 5. Validate resolution is in valid range (0-3)
01243 * 6. If ROM matching enabled, verify family code is 0x28 (DS18B20)
01244 *
01245 * **Validation Rules:**
01246 * ```
01247 * +-----+-----+-----+
01248 * | Field      | Check      | Error Code      |
01249 * +-----+-----+-----+
01250 * | config      | != nullptr  | k_rx_err_null_ptr |
01251 * | bus_manager | != nullptr  | k_rx_err_null_ptr |
01252 * | bus_name    | != nullptr  | k_rx_err_null_ptr |
01253 * | initialized | == false    | k_rx_err_invalid_state |
01254 * | resolution  | <= 3        | k_rx_err_invalid_arg |
01255 * | rom[0] (if ROM) | == 0x28    | k_rx_err_invalid_arg |
01256 * +-----+-----+-----+
01257 * ```
01258 *
01259 * @param[in] config Configuration structure to validate
01260 * @param[in] handle Handle to check for existing initialization
01261 *
01262 * @return k_rx_ok Configuration is valid
01263 * @retval k_rx_err_null_ptr config, bus_manager, or bus_name is nullptr
01264 * @retval k_rx_err_invalid_state handle already initialized
01265 * @retval k_rx_err_invalid_arg Invalid resolution or wrong family code
01266 *
01267 * @pre config != nullptr
01268 * @pre handle != nullptr (checked by caller)
01269 * @post If k_rx_ok returned, all fields are valid for initialization
01270 *
01271 * @note This is an internal helper - not exposed in public API
01272 * @warning Do not call after initialization - will return error
01273 *
01274 * @see rx_ds18b20_init()
01275 * @see ds18b20_resolution_t
01276 */
01277 static rx_err_t internal_ds18b20_validate_config(const rx_ds18b20_config_t* config,
01278                                                  const rx_ds18b20_handle_t* handle)
01279 {
01280     RX_CHECK_NULL_PTR(config, s_tag, "config is nullptr");
01281     RX_CHECK_NULL_PTR(config->bus_manager, s_tag, "bus_manager is nullptr");
01282     RX_CHECK_NULL_PTR(config->bus_name, s_tag, "bus_name is nullptr");
01283
01284     if (handle->initialized) {
01285         rx_log_warn(s_tag, "DS18B20 already initialized");
01286         return k_rx_err_invalid_state;
01287     }
01288
01289     if (config->resolution > k_ds18b20_resolution_bit) {
01290         rx_log_error(s_tag, "Invalid resolution setting");
01291         return k_rx_err_invalid_arg;
01292     }
01293
01294     if (config->use_rom_matching && config->rom[0] != s_ds18b20_family_code) {
01295         rx_log_error(s_tag, "Invalid DS18B20 family code");
01296         return k_rx_err_invalid_arg;
01297     }
01298
01299     return k_rx_ok;
01300 }
01301
01302 /**
01303 * @brief Verify DS18B20 device presence and establish communication
01304 *
01305 * @details
01306 * Three-step verification process to ensure sensor is physically connected
01307 * and responding correctly before completing initialization.
01308 *
01309 * **Algorithm:**
01310 * ```
01311 * @startuml
01312 * start
01313 * :Initialize OneWire bus;
01314 * if (Bus init successful?) then (yes)
01315 *   :Send reset pulse;
01316 *   if (Presence pulse detected?) then (yes)
01317 *     :Read scratchpad (with CRC);
01318 *     if (Scratchpad valid?) then (yes)
01319 *       :Return k_rx_ok;
01320 *     stop
01321 *   else (CRC fail)
01322 *     :Return error;
01323 *   endif
01324 * else (no presence)

```

```

01325 *      :Log "device not present";
01326 *      :Return k_rx_err_invalid_state;
01327 *      stop
01328 *    endif
01329 * else (bus init fail)
01330 *      :Return error;
01331 *    endif
01332 * @enduml
01333 * ```
01334 *
01335 * **Verification Steps:**
01336 * 1. **Bus Initialization** - Configure GPIO for 1-Wire timing
01337 * 2. **Presence Detection** - Send reset, check for presence pulse
01338 * 3. **Communication Test** - Read scratchpad with CRC validation
01339 *
01340 * **Timing:**
01341 * - Reset pulse: 480-960 us LOW
01342 * - Presence detect: 60-240 us after reset
01343 * - Scratchpad read: ~5ms (9 bytes + CRC)
01344 *
01345 * @param[in,out] handle DS18B20 handle with bus configuration
01346 *
01347 * @return k_rx_ok Device present and communicating
01348 * @retval k_rx_err_invalid_state No presence pulse detected
01349 * @retval k_rx_err_* Bus initialization or scratchpad read error
01350 *
01351 * @pre handle->bus_manager is valid
01352 * @pre handle->bus_name is valid
01353 * @post If k_rx_ok, device is ready for configuration
01354 *
01355 * @note This function is called during rx_ds18b20_init()
01356 * @warning Blocks for ~100ms total (bus init + scratchpad read)
01357 *
01358 * @par Performance:
01359 * Execution time: ~100ms (bus init 50ms + reset 1ms + scratchpad 5ms + margin)
01360 *
01361 * @see rx_bus_1wire_init()
01362 * @see rx_bus_1wire_reset()
01363 * @see internal_ds18b20_read_scratchpad_raw()
01364 */
01365 static rx_err_t internal_ds18b20_verify_device_presence(rx_ds18b20_handle_t* handle)
01366 {
01367     /* Initialize OneWire bus */
01368     rx_err_t err = rx_bus_1wire_init(handle->bus_manager, handle->bus_name);
01369     if (err != k_rx_ok) {
01370         rx_log_error(s_tag, "Failed to initialize OneWire bus");
01371         return err;
01372     }
01373
01374     /* Check device presence */
01375     bool presence = false;
01376     err = rx_bus_1wire_reset(handle->bus_manager, handle->bus_name, &presence);
01377     if (err != k_rx_ok) {
01378         rx_log_error(s_tag, "OneWire reset failed");
01379         return err;
01380     }
01381
01382     if (!presence) {
01383         rx_log_error(s_tag, "DS18B20 device not present on bus");
01384         return k_rx_err_invalid_state;
01385     }
01386
01387     /* Read scratchpad to verify communication */
01388     uint8_t scratchpad[k_ds18b20_scratchpad_bytes];
01389     err = internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
01390     if (err != k_rx_ok) {
01391         rx_log_error(s_tag, "Failed to read scratchpad during init");
01392         return err;
01393     }
01394
01395     return k_rx_ok;
01396 }
01397
01398 /**
01399 * @brief Select DS18B20 device using ROM matching or Skip ROM command
01400 *
01401 * @details
01402 * Sends appropriate ROM command based on bus configuration:
01403 * - **Match ROM (0x55)** - Address specific device on multi-drop bus
01404 * - **Skip ROM (0xCC)** - Broadcast to single device or all devices
01405 *
01406 * **Decision Flow:**
01407 * ```
01408 * @startuml
01409 * start
01410 * if (use_rom_matching?) then (yes)
01411 *     :Send Match ROM (0x55);

```

```

01412 * :Send 64-bit ROM code;
01413 * note right
01414 *   Addresses specific device
01415 *   by unique ROM ID
01416 * end note
01417 * else (no)
01418 * :Send Skip ROM (0xCC);
01419 * note right
01420 *   Single device: addresses only device
01421 *   Multiple devices: broadcasts to all
01422 * end note
01423 * endif
01424 * :Device selected;
01425 * :Ready for function command;
01426 * stop
01427 * @enduml
01428 * ```
01429 *
01430 * **ROM Matching (Multi-Drop Bus):**
01431 * - Used when multiple DS18B20 sensors on one bus
01432 * - Requires 64-bit ROM code (obtained via search or label)
01433 * - Ensures commands go to intended sensor only
01434 *
01435 * **Skip ROM (Single Device):**
01436 * - Faster - no need to send 64-bit ROM
01437 * - Safe when only one device on bus
01438 * - Caution: broadcasts if multiple devices present
01439 *
01440 * **Timing:**
01441 * - Match ROM: ~3ms (1 byte command + 8 bytes ROM)
01442 * - Skip ROM: ~1ms (1 byte command)
01443 *
01444 * @param[in] handle DS18B20 handle with ROM configuration
01445 *
01446 * @return k_rx_ok Device selected successfully
01447 * @retval k_rx_err_* Bus communication error
01448 *
01449 * @pre handle->use_rom_matching is configured
01450 * @pre If ROM matching: handle->rom contains valid 64-bit ID
01451 * @post Selected device ready for function commands
01452 *
01453 * @note Called before every DS18B20 function command
01454 * @warning Skip ROM on multi-drop bus affects ALL devices
01455 *
01456 * @par Example: ROM Code Format
01457 * ```
01458 * Byte 0: Family Code (0x28 for DS18B20)
01459 * Byte 1-6: Serial Number (48-bit unique ID)
01460 * Byte 7: CRC-8 checksum
01461 * ```
01462 *
01463 * @see rx_bus_onewire_match_rom()
01464 * @see rx_bus_onewire_skip_rom()
01465 */
01466 static rx_err_t internal_ds18b20_select_device(const rx_ds18b20_handle_t* handle)
01467 {
01468     if (handle->use_rom_matching) {
01469         /* Use Match ROM to address specific device */
01470         rx_err_t err = rx_bus_onewire_match_rom(handle->bus_manager, handle->bus_name, handle->rom);
01471         if (err != k_rx_ok) {
01472             rx_log_error(s_tag, "Failed to send Match ROM");
01473             return err;
01474         }
01475     } else {
01476         /* Use Skip ROM for single device or broadcast */
01477         rx_err_t err = rx_bus_onewire_skip_rom(handle->bus_manager, handle->bus_name);
01478         if (err != k_rx_ok) {
01479             rx_log_error(s_tag, "Failed to send Skip ROM");
01480             return err;
01481         }
01482     }
01483
01484     return k_rx_ok;
01485 }
01486
01487 /**
01488 * @brief Read DS18B20 scratchpad memory with CRC-8 validation
01489 *
01490 * @details
01491 * Reads all 9 bytes of scratchpad memory and validates data integrity using
01492 * CRC-8/MAXIM checksum. This is the primary method for reading temperature
01493 * and configuration data from the sensor.
01494 *
01495 * **Algorithm:**
01496 * ```
01497 * @startuml
01498 * start

```

```

01499 * :Send reset pulse;
01500 * if (Presence detected?) then (yes)
01501 *   :Select device (Match/Skip ROM);
01502 *   :Send Read Scratchpad (0xBE);
01503 *   :Read 9 bytes (8 data + 1 CRC);
01504 *   :Calculate CRC-8 of first 8 bytes;
01505 *   if (CRC matches byte 8?) then (yes)
01506 *     if (Reserved byte == 0xFF?) then (yes)
01507 *       :Return k_rx_ok;
01508 *     stop
01509 *   else (reserved invalid)
01510 *     :Log warning;
01511 *     :Return k_rx_ok anyway;
01512 *     note right: Non-fatal, continue
01513 *   endif
01514 *   else (CRC mismatch)
01515 *     :Log error;
01516 *     :Return k_rx_err_crc_mismatch;
01517 *     stop
01518 *   endif
01519 * else (no presence)
01520 *   :Return k_rx_err_invalid_state;
01521 * endif
01522 * @enduml
01523 * ```
01524 *
01525 * **Scratchpad Memory Map:**
01526 * ```
01527 * +-----+-----+-----+
01528 * | Byte | Field | Description |
01529 * +-----+-----+-----+
01530 * | 0 | Temp LSB | Temperature low byte |
01531 * | 1 | Temp MSB | Temperature high byte |
01532 * | 2 | TH Register | High alarm threshold |
01533 * | 3 | TL Register | Low alarm threshold |
01534 * | 4 | Config | Resolution bits (6:5) |
01535 * | 5 | Reserved | Always 0xFF |
01536 * | 6 | Reserved | Factory-programmed |
01537 * | 7 | Reserved | Count remain (legacy) |
01538 * | 8 | CRC | CRC-8/MAXIM checksum |
01539 * +-----+-----+-----+
01540 * ```
01541 *
01542 * **CRC-8/MAXIM Calculation:**
01543 * - Polynomial: 0x31 ( $x^8 + x^5 + x^4 + 1$ )
01544 * - Initial value: 0x00
01545 * - Input: First 8 bytes of scratchpad
01546 * - Result: Must match byte 8
01547 *
01548 * **Post-Conditions:**
01549 * - Scratchpad buffer contains valid sensor data
01550 * - CRC verified (data integrity guaranteed)
01551 * - Reserved byte checked (warning if unexpected)
01552 *
01553 * @param[in] handle DS18B20 sensor handle (must be initialized)
01554 * @param[out] scratchpad 9-byte buffer to receive scratchpad data
01555 *   Array must be at least k_ds18b20_scratchpad_bytes (9) bytes
01556 *
01557 * @return k_rx_ok Scratchpad read successful, CRC valid
01558 * @retval k_rx_err_null_ptr handle or scratchpad is nullptr
01559 * @retval k_rx_err_invalid_state Device not present on bus
01560 * @retval k_rx_err_crc_mismatch CRC validation failed (corrupted data)
01561 * @retval k_rx_err_* Bus communication error
01562 *
01563 * @pre handle != nullptr
01564 * @pre handle->initialized == true
01565 * @pre scratchpad != nullptr
01566 * @pre scratchpad size >= 9 bytes
01567 * @post If k_rx_ok: scratchpad contains validated sensor data
01568 * @post If error: scratchpad contents undefined
01569 *
01570 * @note Reserved byte validation is non-fatal (log warning only)
01571 * @warning Do not use scratchpad data if CRC check fails
01572 *
01573 * @par Performance:
01574 * Execution time: ~5ms (reset 1ms + select 1ms + read 3ms)
01575 *
01576 * @par Example: Manual Scratchpad Access
01577 * @code
01578 * uint8_t scratchpad[9];
01579 * rx_err_t err = internal_ds18b20_read_scratchpad_raw(&sensor, scratchpad);
01580 * if (err == k_rx_ok) {
01581 *   int16_t temp_raw = scratchpad[0] | (scratchpad[1] << 8);
01582 *   uint8_t config = scratchpad[4];
01583 *   uint8_t resolution = (config >> 5) & 0x03;
01584 * }
01585 * @endcode

```

```

01586 *
01587 * @see rx_crc8_maxim()
01588 * @see internal_ds18b20_select_device()
01589 */
01590 static rx_err_t internal_ds18b20_read_scratchpad_raw(const rx_ds18b20_handle_t* handle,
01591                                                     uint8_t scratchpad[k_ds18b20_scratchpad_bytes])
01592 {
01593     bool presence = false;
01594     rx_err_t err = k_rx_ok;
01595     uint8_t crc_calc = 0;
01596     uint8_t crc_device = 0;
01597
01598     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
01599     RX_CHECK_NULL_PTR(scratchpad, s_tag, "scratchpad is nullptr");
01600
01601     /* Reset and check presence */
01602     err = rx_bus_owewire_reset(handle->bus_manager, handle->bus_name, &presence);
01603     if (err != k_rx_ok || !presence) {
01604         rx_log_error(s_tag, "Device not present for scratchpad read");
01605         return k_rx_err_invalid_state;
01606     }
01607
01608     /* Select device */
01609     err = internal_ds18b20_select_device(handle);
01610     if (err != k_rx_ok) {
01611         return err;
01612     }
01613
01614     /* Send Read Scratchpad command */
01615     err =
01616         rx_bus_owewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_read_scratch);
01617     if (err != k_rx_ok) {
01618         rx_log_error(s_tag, "Failed to send read scratchpad command");
01619         return err;
01620     }
01621
01622     /* Read 9 bytes (8 data + 1 CRC) */
01623     err = rx_bus_owewire_read(handle->bus_manager,
01624                               handle->bus_name,
01625                               scratchpad,
01626                               k_ds18b20_scratchpad_bytes);
01627     if (err != k_rx_ok) {
01628         rx_log_error(s_tag, "Failed to read scratchpad data");
01629         return err;
01630     }
01631
01632     /* Validate CRC */
01633     crc_calc = rx_crc8_maxim(scratchpad, k_ds18b20_crc_bytes);
01634     crc_device = scratchpad[k_ds18b20_scratch_crc];
01635
01636     if (crc_calc != crc_device) {
01637         rx_log_error(s_tag, "Scratchpad CRC mismatch");
01638         return k_rx_err_crc_mismatch;
01639     }
01640
01641     /* Post-condition: Validate scratchpad structure (reserved byte should be 0xFF) */
01642     if (scratchpad[k_ds18b20_scratch_reserved1] != s_ds18b20_reserved_byte_value) {
01643         rx_log_warn(s_tag, "Scratchpad reserved byte unexpected value");
01644     }
01645
01646     return k_rx_ok;
01647 }
01648
01649 /**
01650 * @brief Write DS18B20 scratchpad registers (TH, TL, Config)
01651 *
01652 * @details
01653 * Writes the three user-modifiable scratchpad registers: high alarm (TH),
01654 * low alarm (TL), and configuration (resolution). This is the only way to
01655 * change sensor resolution and alarm thresholds.
01656 *
01657 * **Algorithm:**
01658 * ```
01659 * @startuml
01660 * start
01661 * :Send reset pulse;
01662 * if (Presence detected?) then (yes)
01663 *   :Select device (Match/Skip ROM);
01664 *   :Send Write Scratchpad (0x4E);
01665 *   :Write TH byte;
01666 *   :Write TL byte;
01667 *   :Write Config byte;
01668 *   :Return k_rx_ok;
01669 * note right
01670 *   Changes take effect immediately
01671 *   but are volatile (lost on power cycle)
01672 * end note

```

```

01673 * stop
01674 * else (no presence)
01675 * :Return k_rx_err_invalid_state;
01676 * endif
01677 * @enduml
01678 * ```
01679 *
01680 * **Write Sequence:**
01681 * 1. Reset bus and check presence
01682 * 2. Select device (Match ROM or Skip ROM)
01683 * 3. Send Write Scratchpad command (0x4E)
01684 * 4. Write 3 bytes in order: TH, TL, Config
01685 *
01686 * **Configuration Register Encoding:**
01687 * ```
01688 * Bit: 7 6 5 4 3 2 1 0
01689 * +---+---+---+---+---+---+---+
01690 * | 0 |R1|R0| 1 | 1 | 1 | 1 | 1 |
01691 * +---+---+---+---+---+---+---+
01692 * +---+---+
01693 * Resolution
01694 *
01695 * R1 R0 | Resolution | Conversion Time
01696 * +---+---+---+---+
01697 * | 0 | 0 | 9-bit | 94 ms
01698 * | 0 | 1 | 10-bit | 188 ms
01699 * | 1 | 0 | 11-bit | 375 ms
01700 * | 1 | 1 | 12-bit | 750 ms
01701 * ```
01702 *
01703 * **Persistence:**
01704 * - Scratchpad writes are **volatile** (RAM only)
01705 * - Lost on power cycle or reset
01706 * - Use Copy Scratchpad (0x48) to save to EEPROM
01707 *
01708 * @param[in] handle DS18B20 sensor handle (must be initialized)
01709 * @param[in] scratchpad Write parameters (TH, TL, config)
01710 *
01711 * @return k_rx_ok Scratchpad written successfully
01712 * @retval k_rx_err_null_ptr handle or scratchpad is nullptr
01713 * @retval k_rx_err_invalid_state Device not present on bus
01714 * @retval k_rx_err_* Bus communication error
01715 *
01716 * @pre handle != nullptr
01717 * @pre handle->initialized == true
01718 * @pre scratchpad != nullptr
01719 * @post Sensor resolution/alarms updated (volatile, RAM only)
01720 *
01721 * @note Changes are immediate but volatile (lost on power cycle)
01722 * @warning No CRC verification - ensure bus integrity
01723 *
01724 * @par Performance:
01725 * Execution time: ~3ms (reset 1ms + select 1ms + write 1ms)
01726 *
01727 * @par Example: Change Resolution to 10-bit
01728 * @code
01729 * uint8_t scratchpad_data[9];
01730 * internal_ds18b20_read_scratchpad_raw(&sensor, scratchpad_data);
01731 *
01732 * ds18b20_scratchpad_write_t write_cfg = {
01733 * .th = scratchpad_data[2], // Preserve TH
01734 * .tl = scratchpad_data[3], // Preserve TL
01735 * .config = (0x01 << 5) | 0x1F // 10-bit: R1=0, R0=1
01736 * };
01737 * internal_ds18b20_write_scratchpad(&sensor, &write_cfg);
01738 *
01739 * // Save to EEPROM to make permanent
01740 * rx_ds18b20_save_config(&sensor);
01741 * @endcode
01742 *
01743 * @see rx_ds18b20_save_config() Make changes permanent
01744 * @see internal_ds18b20_select_device()
01745 * @see ds18b20_scratchpad_write_t
01746 */
01747 static rx_err_t internal_ds18b20_write_scratchpad(const rx_ds18b20_handle_t* handle,
01748 const ds18b20_scratchpad_write_t* scratchpad)
01749 {
01750 RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
01751 RX_CHECK_NULL_PTR(scratchpad, s_tag, "scratchpad is nullptr");
01752
01753 /* Reset and check presence */
01754 bool presence = false;
01755 rx_err_t err = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
01756 if (err != k_rx_ok || !presence) {
01757 rx_log_error(s_tag, "Device not present for scratchpad write");
01758 return k_rx_err_invalid_state;
01759 }

```



```

01760
01761 /* Select device */
01762 err = internal_ds18b20_select_device(handle);
01763 if (err != k_rx_ok) {
01764     return err;
01765 }
01766
01767 /* Send Write Scratchpad command */
01768 err =
01769     rx_bus_owewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_write_scratch);
01770 if (err != k_rx_ok) {
01771     rx_log_error(s_tag, "Failed to send write scratchpad command");
01772     return err;
01773 }
01774
01775 /* Write 3 bytes (TH, TL, Config) */
01776 uint8_t write_buf[k_ds18b20_scratchpad_write_bytes];
01777 write_buf[k_ds18b20_write_idx_th] = scratchpad->th;
01778 write_buf[k_ds18b20_write_idx_tl] = scratchpad->tl;
01779 write_buf[k_ds18b20_write_idx_config] = scratchpad->config;
01780
01781 err = rx_bus_owewire_write(handle->bus_manager,
01782                             handle->bus_name,
01783                             write_buf,
01784                             k_ds18b20_scratchpad_write_bytes);
01785 if (err != k_rx_ok) {
01786     rx_log_error(s_tag, "Failed to write scratchpad data");
01787     return err;
01788 }
01789
01790 return k_rx_ok;
01791 }
01792
01793 /**
01794  * @brief Convert resolution enum to config register value
01795  *
01796  * @param[in] resolution Resolution mode
01797  *
01798  * @return Configuration register value
01799  */
01800 static uint8_t internal_ds18b20_resolution_to_config(const ds18b20_resolution_t resolution)
01801 {
01802     /* Resolution bits are R1:R0 at bits 6:5 */
01803     const uint8_t config = (uint8_t)resolution « k_ds18b20_config_r0_bit;
01804
01805     return config;
01806 }
01807
01808 /**
01809  * @brief Get temperature mask for resolution
01810  *
01811  * Lower bits are undefined for resolutions < 12-bit and must be masked.
01812  *
01813  * @param[in] resolution Resolution mode
01814  *
01815  * @return Temperature mask
01816  */
01817 static uint16_t internal_ds18b20_get_temp_mask(const ds18b20_resolution_t resolution)
01818 {
01819     switch (resolution) {
01820         case k_ds18b20_resolution_bit:
01821             return k_ds18b20_temp_mask_bit;
01822         case k_ds18b20_resolution_bit:
01823             return k_ds18b20_temp_mask_bit;
01824         case k_ds18b20_resolution_bit:
01825             return k_ds18b20_temp_mask_bit;
01826         case k_ds18b20_resolution_bit:
01827             return k_ds18b20_temp_mask_bit;
01828         default:
01829             return k_ds18b20_temp_mask_bit;
01830     }
01831 }
01832
01833 /**
01834  * @brief Convert raw temperature to Celsius
01835  *
01836  * DS18B20 stores temperature as 16-bit signed value in 1/16degC units.
01837  * Divide by 16 to get Celsius.
01838  *
01839  * @param[in] raw_temp Raw 16-bit temperature value
01840  *
01841  * @return Temperature in degrees Celsius
01842  */
01843 static float internal_ds18b20_raw_to_celsius(const int16_t raw_temp)
01844 {
01845     if (raw_temp < k_ds18b20_temp_min_raw || raw_temp > k_ds18b20_temp_max_raw) {
01846         rx_log_error(s_tag, "Raw temperature out of range - returning NaN sentinel");

```

```
01847     return NAN;
01848 }
01849
01850 /* Convert from 1/16degC to degC */
01851 float result = (float)raw_temp / s_temp_conversion_divisor;
01852
01853 /* Post-condition: Validate result is finite (not NaN or Inf) */
01854 if (!isfinite(result)) {
01855     rx_log_error(s_tag, "Computed Celsius not finite - returning NaN sentinel");
01856     return NAN;
01857 }
01858
01859 return result;
01860 }
```